COMPUTER SCIENCE

Portfolio - Period 3

Francisco Pimentel

Student No. 5708915

francisco.ponte.pimentel@nhlstenden.student.com

NHL Stenden University of Applied Sciences

January 2026

Table of Contents

Images Index *5*

Introduction *8*

Computer Science *9*

Week 1 ..... **9**

Indoor 9
Playback Speed 10
Making Faces 11
Einstein 12
Deep Thought 13
Home Federal Savings Bank 14
File Extensions 15
Math Interpreter 16
Meal Time 17
CamelCase 18
Coke Machine 19
Just Setting Up my Twttr 20
Vanity Plates 21
Nutrition Facts 22
Tip Calculator 23
Smart Café Helper 24
Vanity Flask 26

Week 2 ..... **30**

Manic Monday Functions and Variables 30
Fuel Gauge 32
Felipe's Taqueria 33
Grocery List 34
Outdated 35
Survival Simulator 36

Week 3 ..... **41**

Manic Monday - Exceptions and Libraries 41
Testing my Twttr 43
Back to the Bank 44
Re-requesting a Vanity Plate 45
Refueling 46
Emojize 48
Frank, Ian and Glen's Letters 49
Adieu, Adieu 50
Guessing Game 51
Little Professor 52

Week 4 ..... **54**

Numb3rs 54
Watch on Youtube 55
Working 9 to 5 56

Regular, um, Expressions ... 58
Response Validation ... 59

Week 5 ... 60
Scavenger Hunt ... 60
CS50 P-Shirt ... 64

Week 6 ... 66
Manic Monday - Regex & File I/O ... 66
Tic-Tac-Toe ... 68
Current Challenges in AI - Based on User Interviews ... 72

Week 7 ... 74
Knights ... 74
The Misleading Minesweeper ... 76
Design Thinking Assignment - Math Animator (Solo Innovator Challenge) ... 82

Week 8 ... 84
Math Animator - Solo Project ... 84

Embedded Systems ... 86

Hardware Platform ... 86
Raspberry Pi Pico 2W ... 86

Software Environments ... 87
MicroPython ... 87
CircuitPython ... 87

Eletronic Components ... 88
LED (Light Emitting Diode) ... 88
RGB LED (Common Cathode) ... 88
4-Digit 7-Segment Display ... 89
LM35 Temperature Sensor ... 89
DS1302 Real-Time Clock (RTC) ... 90
16×2 I2C LCD Display ... 90
LDR (Light Dependent Resistor) ... 91
Analog Joystick ... 91
Voltage Divider ... 91
RFID Module (RC522) ... 92

Web Server (Socket-Based) ... 92

Embedded Systems - Assignements ... 93

Week 1 ... 93
Blink ... 93
Mario Pico ... 95

Week 2 ... 96
7-Segment Voltmeter (0...10V) ... 96

Week 3 ... 99
RTC and Temperature on LCD-display ... 99

Week 4 ... 102
Dino Cheater (HID) ... 102
Analog Joystick (HID) ... 104

Week 5 ..... **106**
WiFi Scanner 106
Controlling Stuff (Webserver) 107

Weeks 6 / 7 ..... **111**
Attendance (RFID, RTC, Web and LCD) 111

Self-Reflection ***121***

Links ***122***

Assignements **122**

Solo Project **122**

Youtube Video **122**

Images Index

Figure 1 - Indoor | Result 9
Figure 2 - Playback Speed | Result 10
Figure 3 - Making Faces | Result 11
Figure 4 - Einstein | Result 12
Figure 5 - Deep Thought | Result 13
Figure 6 - Bank | Result 14
Figure 7 - File Extensions | Result 15
Figure 8 - Math Interpreter | Result 16
Figure 9 - Meal Time | Result 17
Figure 10 - CamelCase | Result 18
Figure 11 - Coke Machine | Result 19
Figure 12 - Twitter | Result 20
Figure 13 - Vanity Plates | Result 21
Figure 14 - Nutrition Facts | Result 22
Figure 15 - Tip Calculator | Result 23
Figure 16 - Smart Café Helper | Result 25
Figure 17 - Website | Valid Input 28
Figure 18 - Website | Invalid Input 29
Figure 19 - Website | History 29
Figure 20 - Manic Monday Functions and Variables | Result 30
Figure 21 - Fuel Gauge | Result 32
Figure 22 - Felipe's Taqueria | Result 33
Figure 23 - Grocery List | Result 34
Figure 24 - Outdated | Result 36
Figure 25 - Manic Monday - Exceptions and Libraries | Result 42
Figure 26 - Manic Monday - Exceptions and Libraries | Pytest 42
Figure 27 - Testing my twttr | Result 43
Figure 28 - Back to the Bank | Result 44
Figure 29 - Re-requesting a Vanity Plate | Result 46
Figure 30 - Refueling | Result 47
Figure 31 - Emojize | Result 48
Figure 32 - Frank, Ian and Glen’s Letters | Result 49
Figure 33 - Adieu, Adieu | Result 50
Figure 34 - Guessing Game | Result 51
Figure 35 - Little Professor | Result 53
Figure 36 - Numb3rs | Result 54
Figure 37 - Numb3rs | Test Result 55
Figure 38 - Watch on Youtube | Result 56
Figure 39 - Working 9 to 5 | Result 57

Figure 40 - Working 9 to 5 | Test Result 58
Figure 41 - Regular, um, Expressions | Result 58
Figure 42 - Regular, um, Expressions | Test Result 59
Figure 43 - Response Validation | Result 59
Figure 44 - Validate Email Address | Result 60
Figure 45 - FizzBuzz | Result 61
Figure 46 - Palindrome Checker | Result 62
Figure 47 - Find The Longest Word | Result 62
Figure 48 - Unitesting A Calculator | Result 63
Figure 49 - CS50 P-Shirt | Result 64
Figure 50 - CS50 P-Shirt | After Figure 51 - CS50 P-Shirt | Before 65
Figure 52 - Tic-Tac-Toe | Win Result 71
Figure 53 - Tic-Tac-Toe | Tie Result 72
Figure 54 - Knights | Result 76
Figure 55 - The Misleading Minesweeper | Start 81
Figure 56 - The Misleading Minesweeper | Beginning 81
Figure 57 - The Misleading Minesweeper | Lost 81
Figure 58 - The Misleading Minesweeper | Win 82
Figure 59 - Blink | Real View 94
Figure 60 - Blink | Circuit View 94
Figure 61 - Mario More | Pico Version | Result 95
Figure 62 - 7-Segment Voltmeter | Circuit View 98
Figure 63 - 7-Segment Voltmeter | Real View 98
Figure 64 - RTC and Temperature | Circuit View 101
Figure 65 - RTC and Temperature | Real View 101
Figure 66 - Dino Cheater (HID) | Real View 103
Figure 67 - Dino Cheater (HID) | Circuit View 103
Figure 68 - Analog Joystick (HID) | Real View 105
Figure 69 - Analog Joystick (HID) | Circuit View 105
Figure 70 - WiFi Scanner | Result 106
Figure 71 - Controlling Stuff (Webserver) | Circuit View 110
Figure 72 - Controlling Stuff (Webserver) | Real View 110
Figure 73 - Controlling Stuff (Webserver) | Web page 110
Figure 74 - Attendance (RFID, RTC, Web and LCD) | Circuit View 117
Figure 75 - Attendance (RFID, RTC, Web and LCD) | Real View 117
Figure 76 - Attendance (RFID, RTC, Web and LCD) | Clock-in 118
Figure 77 - Attendance (RFID, RTC, Web and LCD) | While Clock-in 118
Figure 78 - Attendance (RFID, RTC, Web and LCD) | Clock-out 118
Figure 79 - Attendance (RFID, RTC, Web and LCD) | Error 119
Figure 80 - Attendance (RFID, RTC, Web and LCD) | Clock-in 119
Figure 81 - Attendance (RFID, RTC, Web and LCD) | Clock-out 120
Figure 82 - Attendance (RFID, RTC, Web and LCD) | Unknown 120

Introduction

This portfolio documents my individual work throughout the third semester of the Computer Science program at NHL Stenden University of Applied Sciences. It covers two main areas of study: Computer Science (CS) and Embedded Systems, reflecting the practical and theoretical skills developed over the course of eight weeks.

The Computer Science section follows the CS50P curriculum and progressively builds competency in Python programming. Starting from basic input/output and string manipulation in Week 1, the assignments advanced through functions, loops, dictionaries, file I/O, regular expressions, unit testing with pytest, web development with Flask, object-oriented design, and AI concepts such as logic-based reasoning and game-playing algorithms. Each assignment is documented with its source code and the corresponding test results, including check50 scores where applicable.

The Embedded Systems section covers programming and circuit design for the Raspberry Pi Pico 2W microcontroller using MicroPython and CircuitPython. The assignments range from basic LED control (Blink) to more complex projects involving 7-segment displays, real time clocks, temperature sensor, LCD screen, HID device, Wi-Fi scanning, and a web-controlled RGB LED. Each project includes the program code, circuit diagrams, and real-world photographs of the hardware setup.
Throughout this semester, I aimed to write clean, well-commented code and to test my implementations thoroughly. Where unit tests were required, I wrote them using pytest and ensured all tests passed before submission. For the embedded projects I paid particular attention to reproducibility, documenting writing diagrams alongside the physical builds.

This portfolio is a reflection of my learning process and the skills I have developed during Period 3.

Computer Science

Week 1

Week 1 introduces the core building blocks of Python: reading input, manipulating strings, performing arithmetic, and controlling program flow with conditionals and loops. Every assignment in this week depends entirely on built-in Python features, which supports the language fundamentals before moving on to functions and modules.

Indoor

A minimal program that reads a string from standard input and prints it back in all lower-case. The objective is to practise reading user input and applying the str.lower() method.

```
string = input("")
print(f"{string.lower()}")
```

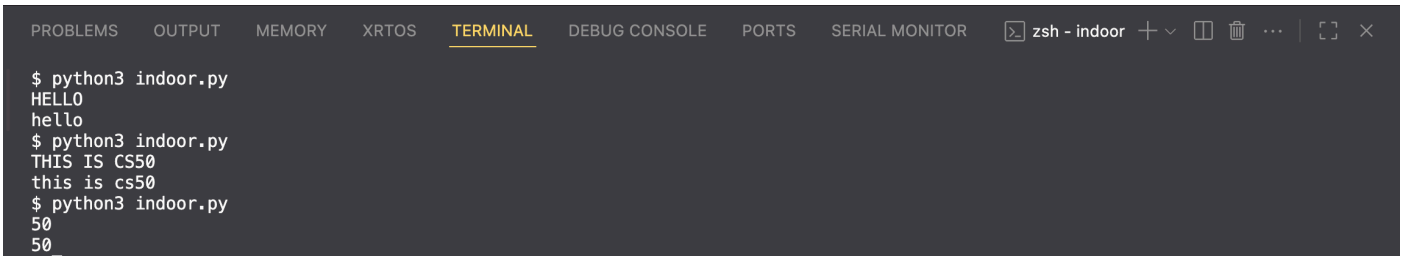
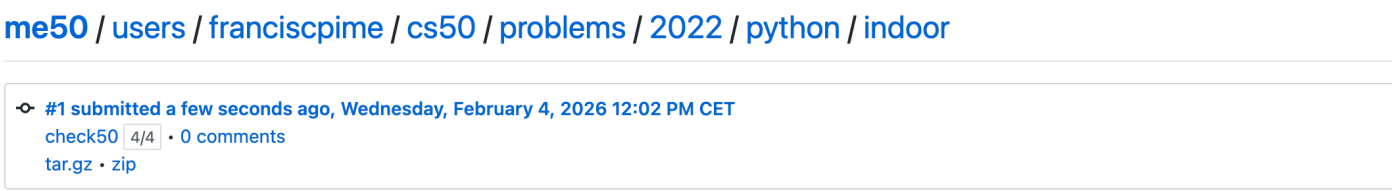


Figure 1 - Indoor | Result



Playback Speed

Reads a message from the user and replaces every space character with "..." to simulate a slowed-down playback effect. The solution iterates character-by-character through the string using a for loop and accumulates the result in a new variable.

```
string = input("Message: ")
transformed = ""
for i in range(len(string)):
    if string[i] == " ":
        transformed += "..."
    else:
        transformed += string[i]
print(transformed)
```

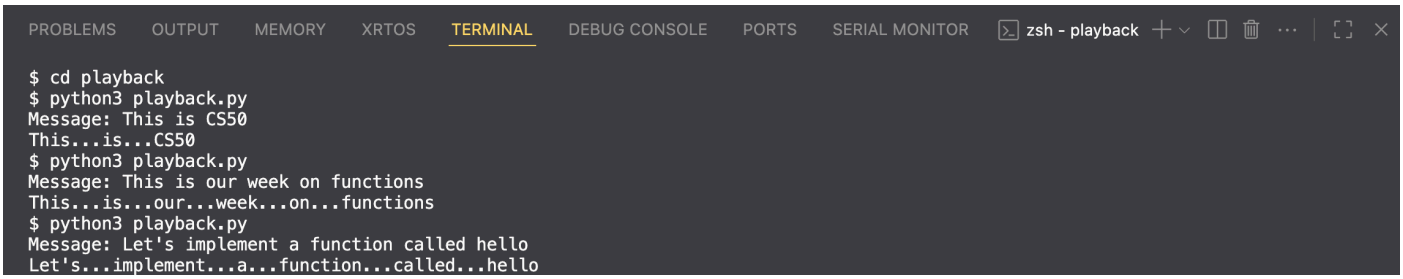
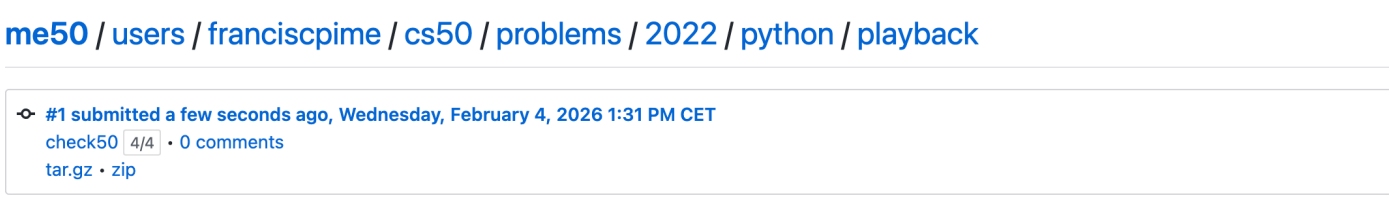


Figure 2 - Playback Speed | Result



Making Faces

Converts ASCII emoticons such as ":)" or ":(" into their corresponding Unicode emoji characters. The conversion table is stored in a Python dictionary, and str.replace() is applied iteratively, demonstrating how dictionaries serve as lookup tables for transformations.

```
def main():
    message = input("Write a message: ")
    print(convert(message))
def convert(message):
    emojis = {
        ":)": "🙂",
        ":(": "🙁",
        ":D": "😄",
        ";)": "😉",
        ":/": "😐",
        "xD": "😵",
        ":O": "😮"
    }
    for c in emojis:
        message = message.replace(c, emojis[c])
    return message
main()
```

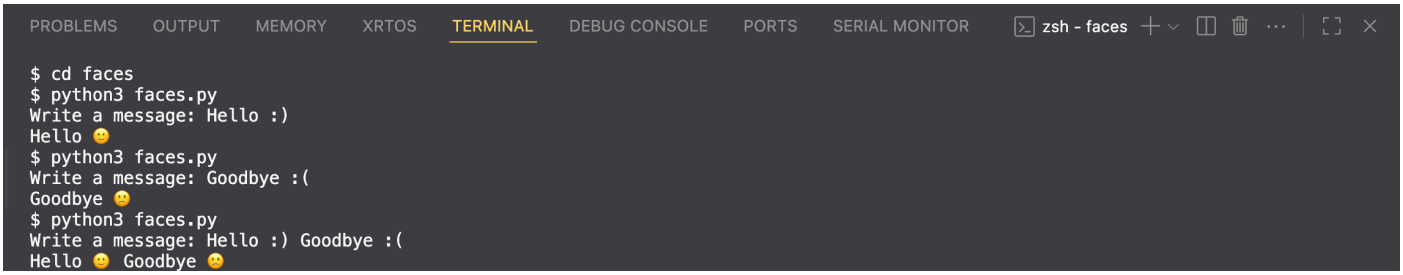
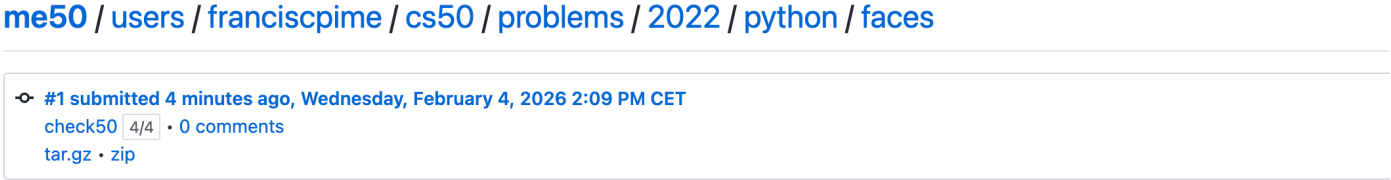


Figure 3 - Making Faces | Result



Einstein

Implements the mass-energy equivalence formula $E = mc^2$. The user inputs a mass in kilograms and the program outputs the equivalent energy in Joules formatted in scientific notation. This exercises integer input, the pow() function, and f-string formatting with the :.2e specifier.

```
mass = int(input("m: "))
c = 300000000
energy = mass * pow(c,2)
print(f"E: {energy:.2e}")
```

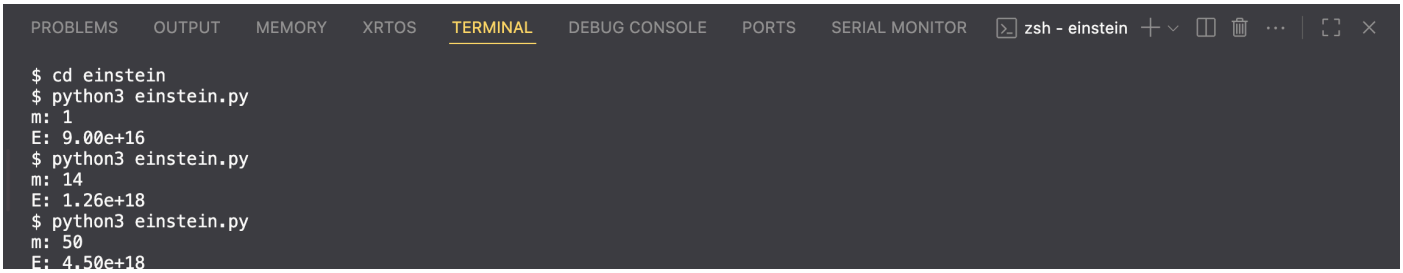


Figure 4 - Einstein | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / einstein

#1 submitted a few seconds ago, Wednesday, February 4, 2026 2:20 PM CET
check50 4/4 • 0 comments
tar.gz • zip

Deep Thought

A humorous reference to The Hitchhiker's Guide to the Galaxy. The program accepts multiple valid representations of the answer "42" (numeric, written-out) and prints "Yes" or "No" accordingly. Demonstrates case normalisation with str.lower() and multi-condition boolean logic using or.

```
answer = input("What is the Answer to the Great Question of Life, the Universe and Everyth
ing? ").lower()
if answer == "forty-two" or answer == "forty two" or answer.strip() == "42":
    print("Yes")
else:
    print("No")
```

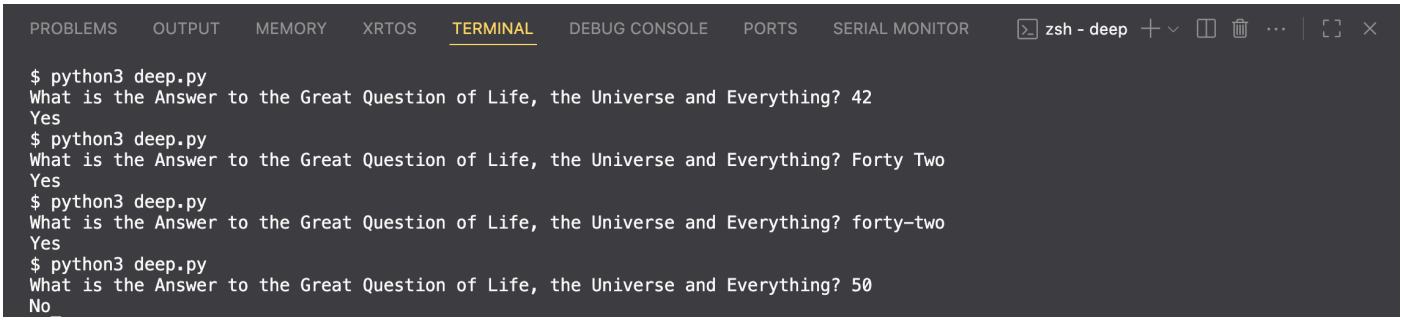
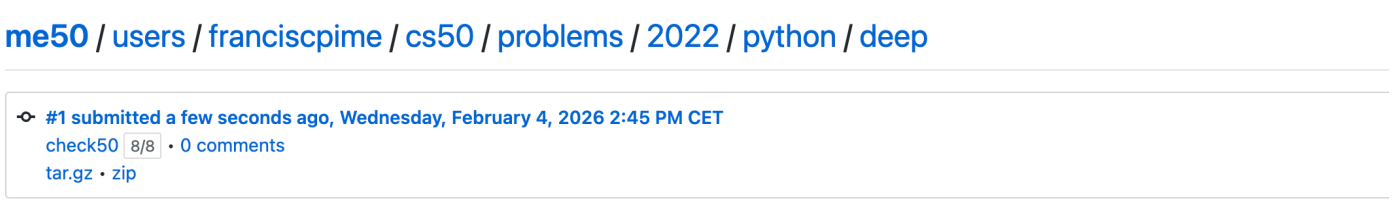


Figure 5 - Deep Thought | Result



Home Federal Savings Bank

Implements a teller greeting fee system: greetings starting with "hello" are free, those starting with "h" incur a $20 fee, and all others cost $100.

```
greeting = input("Greeting: ").lower()
if greeting.strip() == "hello" or greeting == "hello there" or greeting =="hello, newman":
    print("$0")
elif greeting[0] == "h":
    print("$20")
else:
    print("$100")
```

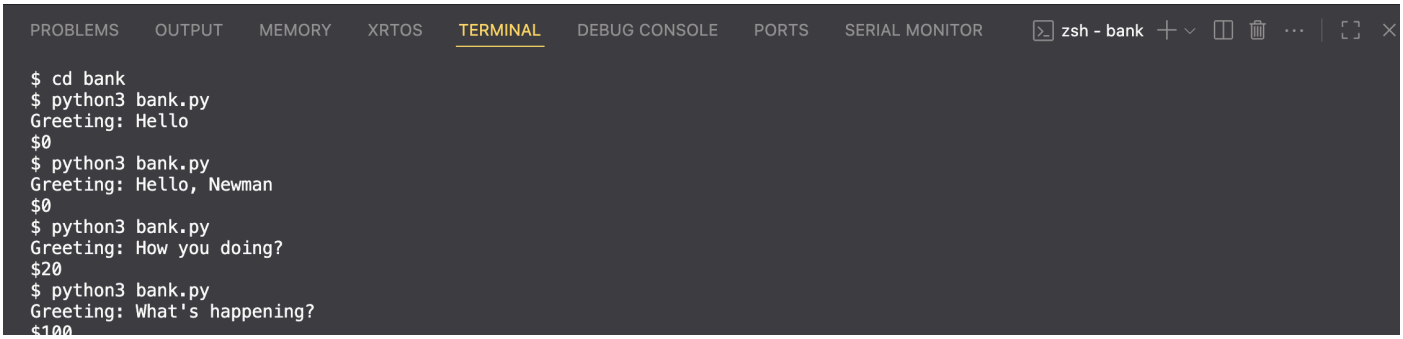


Figure 6 - Bank | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / bank

#1 submitted a few seconds ago, Wednesday, February 4, 2026 3:00 PM CET
check50 7/7 · 0 comments
tar.gz · zip

File Extensions

Maps common file extensions to their MIME types. The program converts the filename to lowercase, then uses a chain of elif blocks checking str.endswith() to determine the correct MIME type. Unknown extensions default to "application/octet-stream".

```
filename = input("File name: ").lower().strip()
if filename.endswith(".gif"):
    print("image/gif")
elif filename.endswith(".jpg") or filename.endswith(".jpeg"):
    print("image/jpeg")
elif filename.endswith(".png"):
    print("image/png")
elif filename.endswith(".pdf"):
    print("application/pdf")
elif filename.endswith(".txt"):
    print("text/plain")
elif filename.endswith(".zip"):
    print("application/zip")
elif filename.endswith(".html") or filename.endswith(".htm"):
    print("text/html")
else:
    print("application/octet-stream")
```

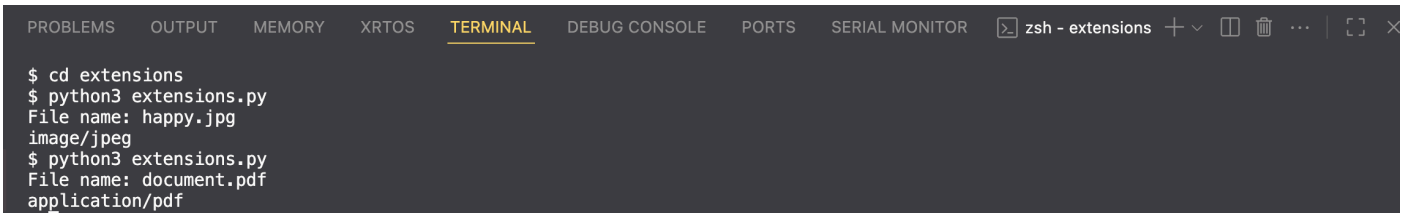


Figure 7 - File Extensions | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / extensions

#1 submitted a few seconds ago, Wednesday, February 4, 2026 3:27 PM CET
check50 14/14 • 0 comments
tar.gz • zip

Math Interpreter

Parses a simple arithmetic expression entered as a string (e.g. "3 + 4") by splitting on whitespace to extract the two operands and the operator. Supports +, -, *, and / and prints the result formatted to one decimal place. Introduces the concept of parsing user-supplied expressions.

```
expression = input("Expression: ")
x, y, z = expression.split()
n1 = int(x)
n2 = int(z)
if y == "+":
    expression = n1 + n2
    print(f"{expression:.1f}")
elif y == "-":
    expression = n1 - n2
    print(f"{expression:.1f}")
elif y == "*":
    expression = n1 * n2
    print(f"{expression:.1f}")
elif y == "/":
    expression = n1 / n2
    print(f"{expression:.1f}")
```

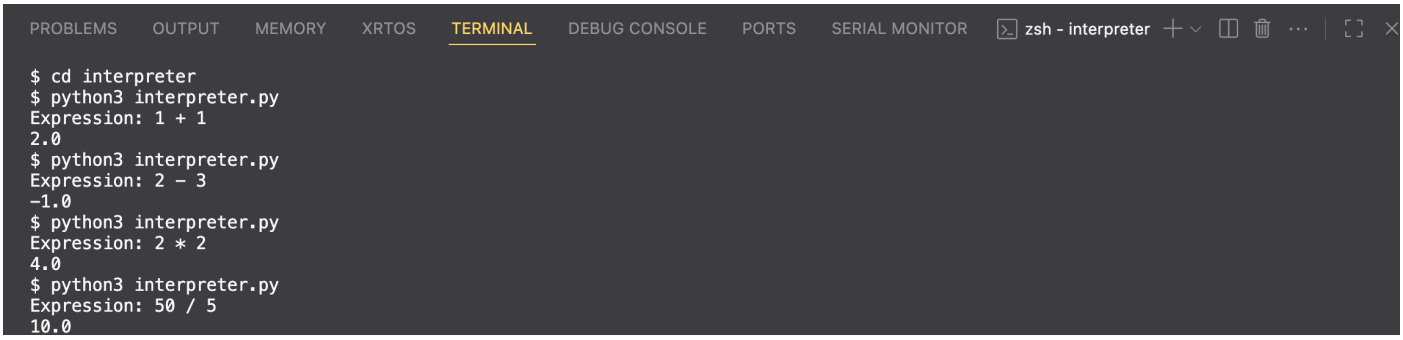
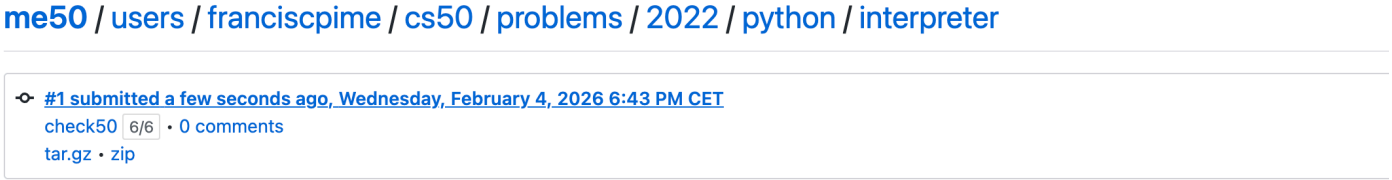


Figure 8 - Math Interpreter | Result



Meal Time

Determines whether a given time corresponds to breakfast (07:00 to 08:00), lunch (12:00 to 13:00), or dinner (18:00 to 19:00). A separate convert() function converts "HH:MM" into a decimal hours value, which is then compared using inclusive range checks. Demonstrates function decomposition and the use of __name__ == "__main__".

```python
def main():
    time = input("What time is it? ")
    hours = convert(time)
    if 7.0 <= hours <= 8.0:
        print("Breakfast time")
    elif 12.0 <= hours <= 13.0:
        print("Lunch time")
    elif 18.0 <= hours <= 19.0:
        print("Dinner time")
def convert(time):
    hours, minutes = time.split(":")
    return int(hours) + int(minutes) / 60
if __name__ == "__main__":
    main()
```

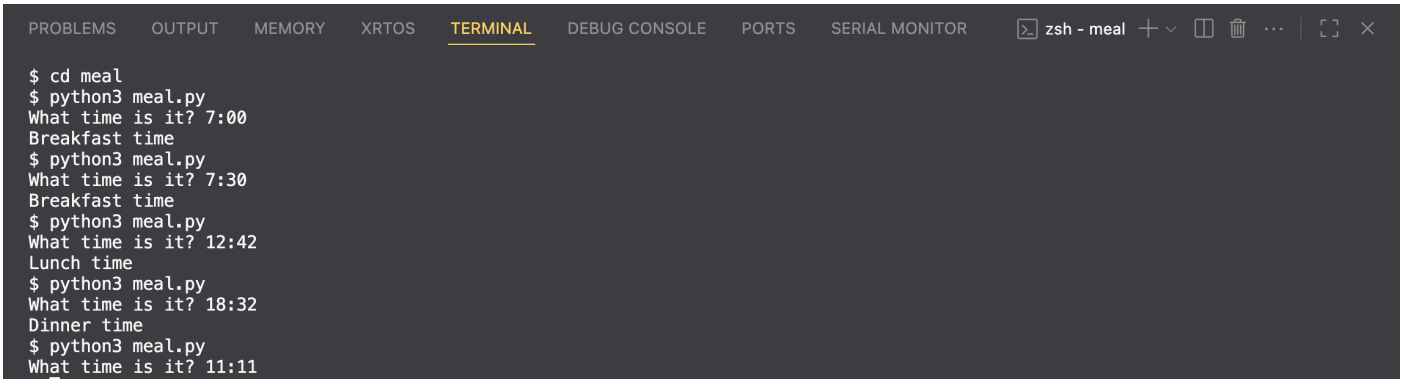
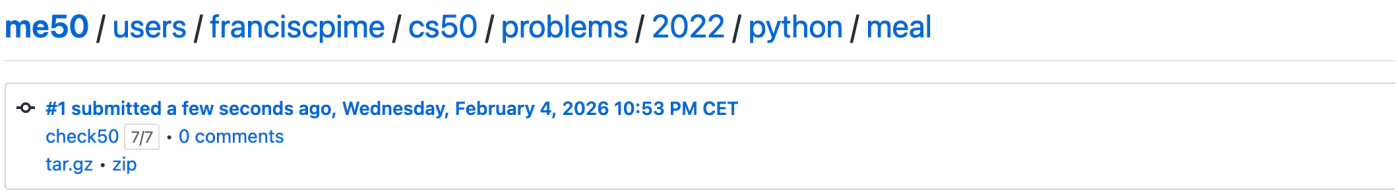


Figure 9 - Meal Time | Result



CamelCase

Converts a camelCase variable name to snake_case by iterating through each character and inserting an underscore followed by the lower-cased letter whenever an uppercase character is encountered. Uses str.isupper() and str.lower() and demonstrates character-level string processing.

```
variable = input("camelCase: ")

for c in variable:
    if c.isupper():
        print("_" + c.lower(), end="")
    else:
        print(c, end="")

print()
```

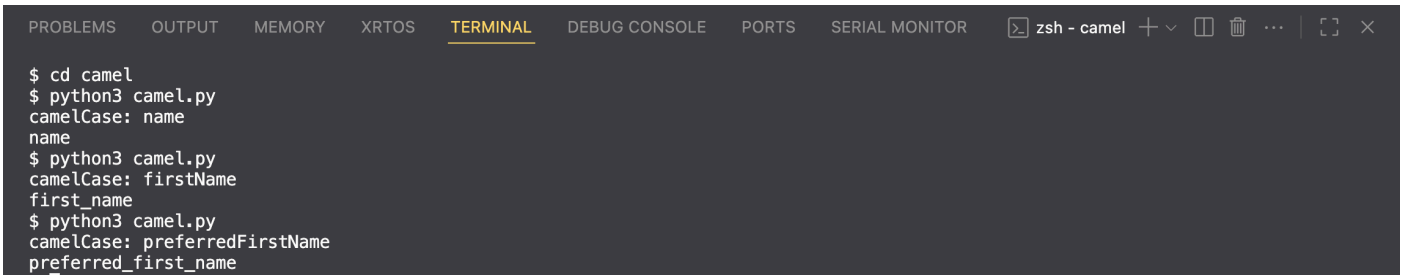
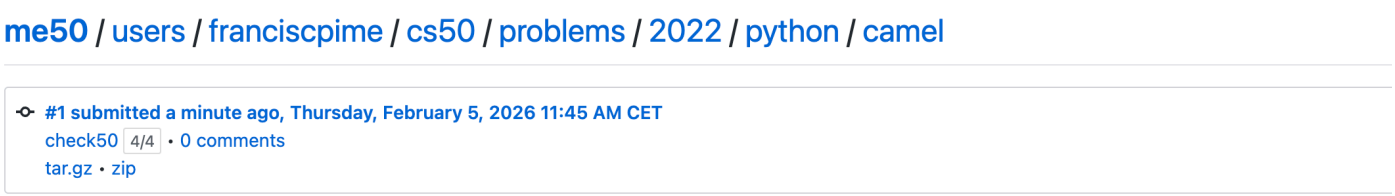


Figure 10 - CamelCase | Result



Coke Machine

Simulates a vending machine that accepts coins of 5, 10, and 25 cents until the $0.50 price is met or exceeded, then prints any change owed. Uses a while loop that terminates when the running total reaches zero, ignoring invalid coin values. Introduces loop-based accumulation and the abs() function for computing change.

```
price = 50
while price > 0:
    print((f"Amount Due: {price}"))
    amount = int(input("Insert coin: "))
    if amount == 25:
        price -= 25
    elif amount == 10:
        price -= 10
    elif amount == 5:
        price -= 5
change = abs(price)
print(f"Change Owed: {change}")
```

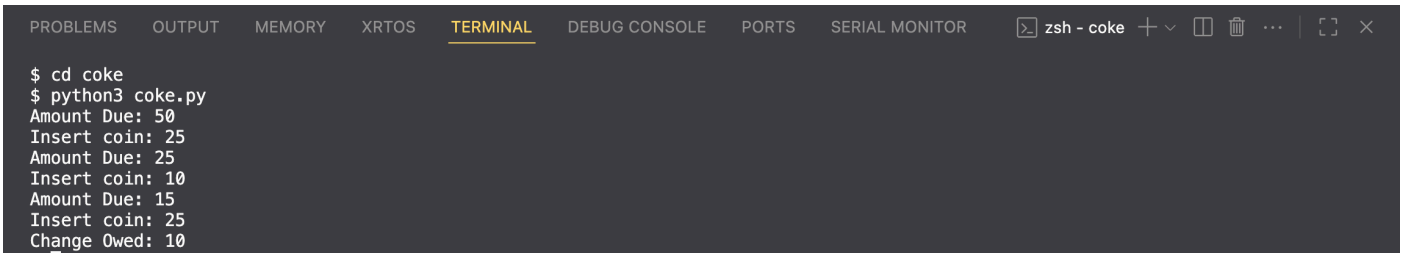
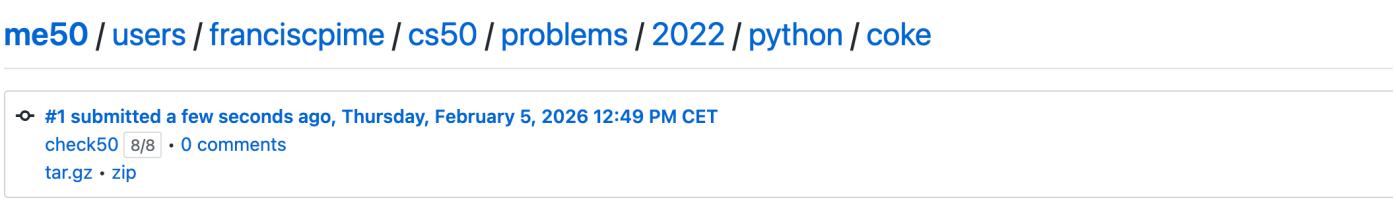


Figure 11 - Coke Machine | Result



Just Setting Up my Twttr

Removes all vowels from user input, replicating the style of early SMS character limits. The program defines the vowel set as a string and uses a for loop with a membership check to filter out vowels before printing. Introduces the concept of filtering characters from a string.

```
text = input("Input: ")
vowels = "aeiouAEIOU"
for c in text:
    if c not in vowels:
        print(c, end="")
print()
```

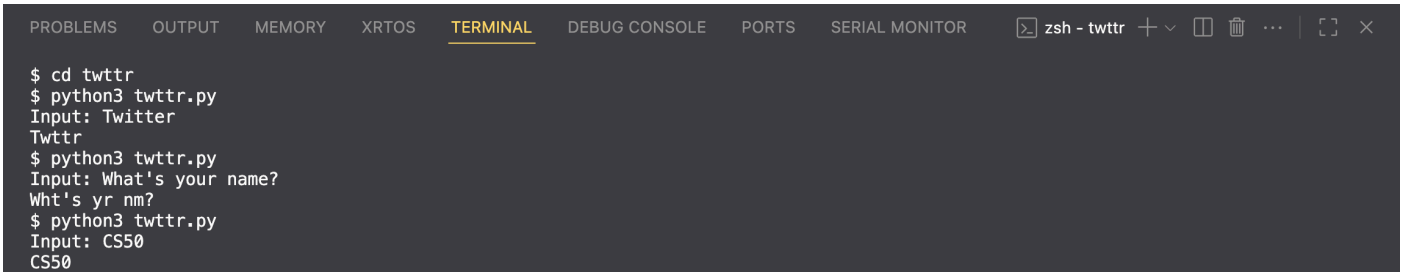
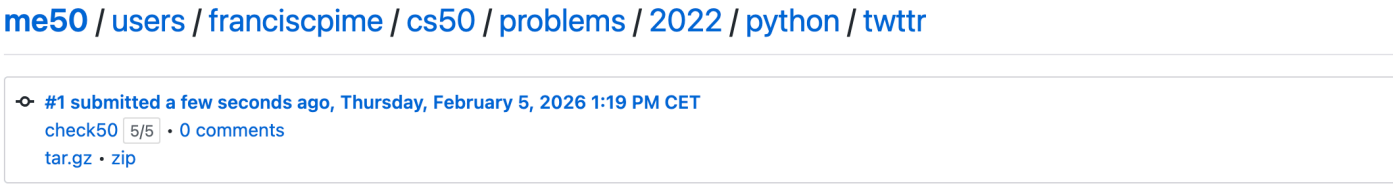


Figure 12 - Twitter | Result



Vanity Plates

Validates Massachusetts vanity licence plates according to a set of rules: length between 2 and 6, first two characters must be letters, only alphanumeric characters allowed, numbers must appear at the end, and the first digit cannot be 0. The logic is encapsulated in an is_valid() function, demonstrating early-return guard clauses and a stateful loop.

```
def main():
    plate = input("Plate: ")
    if is_valid(plate):
        print("Valid")
    else:
        print("Invalid")
def is_valid(s):
    # size
    if not (2 <= len(s) <= 6):
        return False
    # first 2 characters are letters
    if not s[0].isalpha() or not s[1].isalpha():
        return False
    # just letters and numbers
    if not s.isalnum():
        return False
    # numbers in the end
    number_started = False
    for c in s:
        if c.isdigit():
            # first number can't be 0
            if not number_started:
                if c == "0":
                    return False
                number_started = True
        else:
            if number_started:
                return False
    return True
main()
```

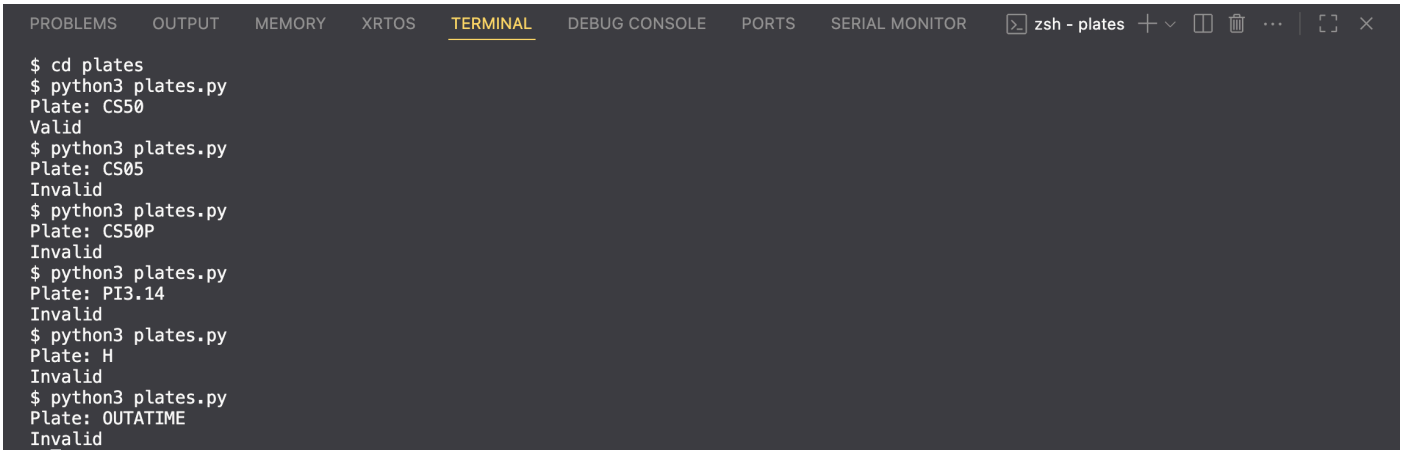


Figure 13 - Vanity Plates | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / plates

#1 submitted a few seconds ago, Thursday, February 5, 2026 3:40 PM CET
check50 10/10 • 0 comments
tar.gz • zip

Nutrition Facts

Looks up the calorie count of a piece of fruit using a dictionary. Input is normalised with strip().lower() before the lookup, so capitalisation differences are handled transparently. Demonstrates dictionary key lookup and the importance of input normalisation.

```
item = input("Item: ").strip().lower()
fruits = {
    "apple" : "130",
    "Avocado" : "50",
    "banana" : "110",
    "cantaloupe" : "50",
    "grapefruit" : "60",
    "grapes" : "90",
    "honeydew melon" : "50",
    "Kiwifruit" : "90",
    "lemon" : "15",
    "lime" : "20",
    "nectarine" : "60",
    "orange" : "80",
    "peach" : "60",
    "pear" : "100",
    "pineapple" : "50",
    "plums" : "70",
    "strawberries" : "50",
    "Sweet Cherries" : "100",
    "tangerine" : "50",
    "watermelon" : "80"
}
if item in fruits:
    print(f"Calories: {fruits[item]}")
else:
    print()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR  zsh - nutrition

$ cd nutrition
$ python3 nutrition.py
Item: Apple
Calories: 130
$ python3 nutrition.py
Item: Avocado
Calories: 50
$ python3 nutrition.py
Item: Sweet Cherries
Calories: 100
$ python3 nutrition.py
Item: Tomato
```

Figure 14 - Nutrition Facts | Result

Tip Calculator

Calculates a restaurant tip given a meal cost and a desired tip percentage. Two helper functions, dollars_to_float() and percent_to_float(), handle input parsing by stripping the "$" and "%" symbols respectively. Demonstrates single-responsibility functions and formatted output with :.2f.

```
def main():
    dollars = dollars_to_float(input("How much was the meal? "))
    percent = percent_to_float(input("What percentage would you like to tip? "))
    tip = dollars * percent
    print(f"Leave ${tip:.2f}")
def dollars_to_float(d):
    dollars = d.strip("$")
    d = float(dollars)
    return d
def percent_to_float(p):
    percent = p.strip("%")
    p = float(percent)
    return p / 100
main()
```

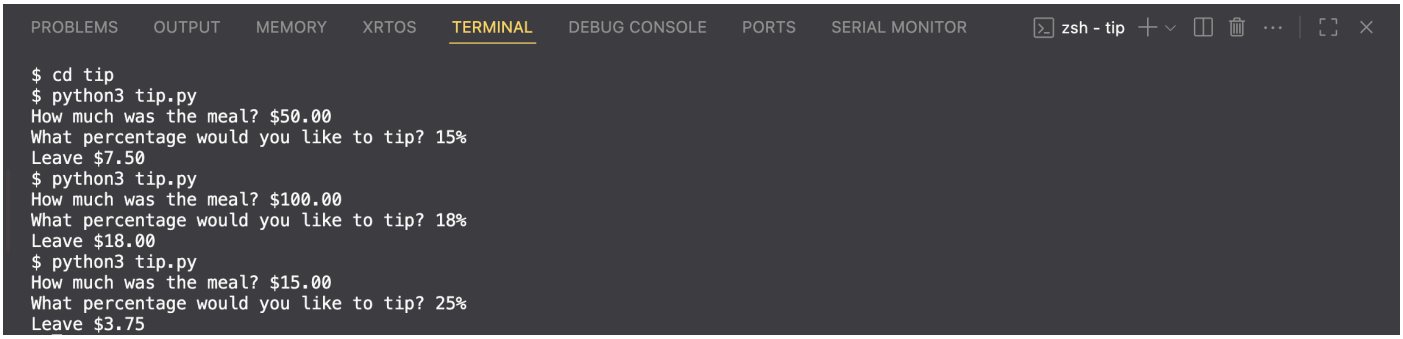


Figure 15 - Tip Calculator | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / tip

#1 submitted a few seconds ago, Monday, February 2, 2026 3:48 PM CET
check50 4/4 • 0 comments
tar.gz • zip

Smart Café Helper

A creative assignment that integrates several Week 1 concepts into a single café ordering program. The program greets the customer, optionally transforms the welcome message character-by-character (Playback Speed style), allows multiple item orders mapped to emojis (Making Faces style), offers an optional E = mc² energy calculation (Einstein style), tallies a running total, and finally applies a tip (Tip Calculator style). Each feature is encapsulated in its own function, resulting in a clean main() control flow.

```
def main():
    welcome_message()
    total = 0
    while True:
        order = input("Please enter your order or type 'done' to finish: ")
        if order.lower() == "done":
            break
        show_order(order)
        energy_calculation()
        total += get_price()
    print(f"Your total is ${total:.2f}.")
    tip_calculation(total)
    print("Thank you for visiting Smart Café!")
def welcome_message():
    print("Welcome to Smart Café!")
    answer = input("Would you like to customize the welcome message? (yes/no): ").lower()

    if answer == "yes":
        message = input("Enter your custom message: ")
        transformed = ""
        for i in range(len(message)):                    # Loop through each character
            transformed += message[i]                    # Add character
            if i != len(message) - 1:                    # Avoid adding dots at the end
                transformed += "..."                     # Add dots between characters
        print(f"Transformed welcome message: {transformed}")
def show_order(order):
    emojis = {                          # Dictionary of keywords and emojis
        "coffee": "☕",
        "tea": "🍵",
        "cake": "🍰",
        "cookie": "🍪",
        "sushi": "🍱"
    }
    order_lower = order.lower()
    if order_lower in emojis:
        print(f"Okay, I will prepare {order_lower} {emojis[order_lower]}")
    else:
        print(f"Okay, I will prepare {order_lower}")
def energy_calculation():
    answer = input("Would you like to calculate energy? (yes/no): ").lower()
    if answer == "yes":
        weight = float(input("Enter the weight in grams: "))
        mass = weight / 1000                             # Convert grams to kilograms
        c = 299792458                                    # Speed of light
```

```
        energy = mass * pow(c, 2)                    # Calculate energy using E = mc^2
        print(f"Energy: {energy:.2e} Joules.")
def get_price():
    price = float(input("Enter the price of this item: $"))
    return price
def tip_calculation(total):
    tip = int(input("How much tip would you like to add? (10, 15, 20): "))
    final_total = total + (total * tip / 100)
    print(f"With tip, your total is: ${final_total:.2f}")
main()
```

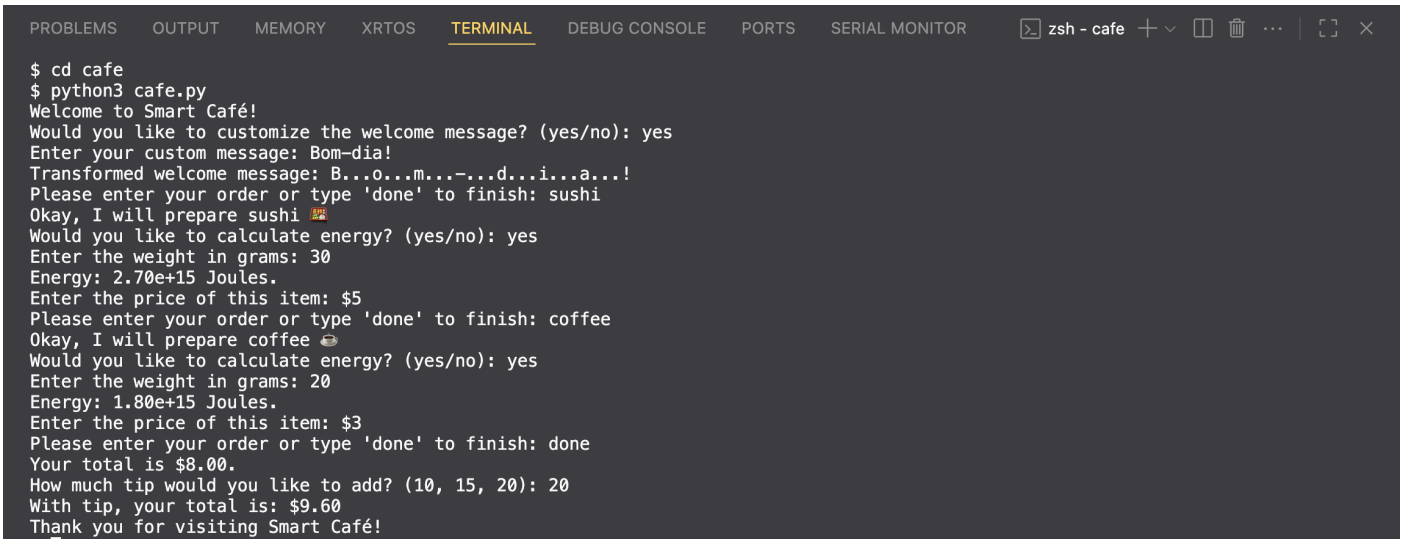


Figure 16 - Smart Café Helper | Result

Vanity Flask

A web application built with Flask and SQLite (via the cs50 library) that exposes the Vanity Plates validator through a browser interface. The is_valid() logic is reused from the earlier assignment unchanged. Two routes are defined: "/" handles both GET (render empty form) and POST (validate and store result) requests, while "/history" queries the database and renders all previous checks in a table. This assignment introduces HTTP request handling, Jinja2 templating, and persistent storage with SQL.

```
from flask import Flask, render_template, request
from cs50 import SQL
app = Flask(__name__)
# Connect the application to the SQLite database file plates.db
db = SQL("sqlite:///plates.db")
def is_valid(s):
    # Plate must have between 2 and 6 characters
    if not (2 <= len(s) <= 6):
        return False
    # First two characters must be letters
    if not s[0].isalpha() or not s[1].isalpha():
        return False
    # Plate can only contain letters and numbers
    if not s.isalnum():
        return False
    # Flag to track when the number sequence begins
    number_started = False
    # Loop through every character in the plate
    for c in s:
        # If the character is a digit
        if c.isdigit():
            # If this is the first digit found
            if not number_started:
                # The first digit cannot be 0
                if c == "0":
                    return False
                # Mark that numbers have started
                number_started = True
        else:
            # If we already started numbers and now see a letter >> invalid
            if number_started:
                return False
    return True
@app.route("/", methods=["GET", "POST"])
def index():
    # If the user submitted the form
    if request.method == "POST":
        # Get the plate value typed in the input field named "plate"
        plate = request.form.get("plate")
        # Check if the plate is valid using our function
        valid = is_valid(plate)
        # Insert the result into the database
        db.execute(
            "INSERT INTO plates (plate, valid) VALUES (?, ?)",
```

```
            plate,
            1 if valid else 0                                   # 1 = valid  0 = invalid
        )
        # Reload page showing the result
        return render_template("index.html", result=valid, plate=plate)
    # If user just opened the page (GET request), show empty form
    return render_template("index.html")
@app.route("/history")
def history():
    # Get all previous checks ordered from newest to oldest
    plates = db.execute("SELECT * FROM plates ORDER BY id DESC")
    # Send the list to the HTML page
    return render_template("history.html", plates=plates)
if __name__ == "__main__":
    app.run(debug=True)
```

```
<!DOCTYPE html>
<html>
<head>
    <title>Vanity Plates</title>
</head>
<body>
    <h1>Vanity Plates Checker</h1>
    <a href="/">Check Plate</a>
    <a href="/history">History</a>
    <hr>
    <h2>Check a plate</h2>
    <!-- Form sends data to the same route ("/") using POST method -->
    <form method="post">
        <input type="text" name="plate">
        <button type="submit">Check</button>
    </form>
    <br>
        <!-- Only show this block after a form submission -->
        {% if result is defined %}
            <!-- If plate is valid -->
            {% if result %}
                <p>Valid plate: {{ plate }}</p>
            <!-- If plate is invalid -->
            {% else %}
                <p>Invalid plate: {{ plate }}</p>
            {% endif %}
        {% endif %}
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
    <title>History</title>
</head>
<body>
    <h1>History</h1>
    <a href="/">Check Plate</a>
    <a href="/history">History</a>
    <hr>
    <!-- Table displaying all previous checks -->
    <table border="5">
```

```
        <!-- Table header row -->
        <tr>
            <th>Plate</th>
            <th>Result</th>
        </tr>
        <!-- Loop through all records sent from Flask -->
        {% for plate in plates %}
        <tr>
            <!-- Show plate text -->
            <td>{{ plate.plate }}</td>
            <!-- Show validation result -->
            <td>
                <!-- valid stored as 1 -->
                {% if plate.valid == 1 %}
                    Valid
                <!-- valid stored as 0 -->
                {% else %}
                    Invalid
                {% endif %}
            </td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>
```



Figure 17 - Website | Valid Input



Figure 18 - Website | Invalid Input



Figure 19 - Website | History

Week 2

Week 2 intensifies the use of functions as the primary unit of code organisation and introduces more advanced control flow patterns: while loops with exception handling (try/except), dictionaries as data structures, and the EOFError pattern for reading unbounded input. Unit testing with Python's unittest module is also introduced through the Manic Monday exercise.

Manic Monday Functions and Variables

The ask() function reads a line from standard input and returns it; the titlize() function converts any string to title case by tracking word boundaries manually. A suite of three unittest test cases verifies input handling, lower-case conversion, and mixed-case conversion. This assignment establishes the pattern of separating I/O from logic.

```
def ask() -> str:
 text = input("Write: ")
 return text
def titlize(s: str) -> str:
 result = ""
 new_word = True
 for c in s:
  if c == " ":
   result += c
   new_word = True
  else:
   if new_word:
    result += c.upper()
    new_word = False
   else:
    result += c.lower()
 return result
def main():
 # do not change code below
 s = ask()
 s = titlize(s)
 print(f"The title is: {s}")
if __name__ == "__main__":
 # do not change code below
 main()
```

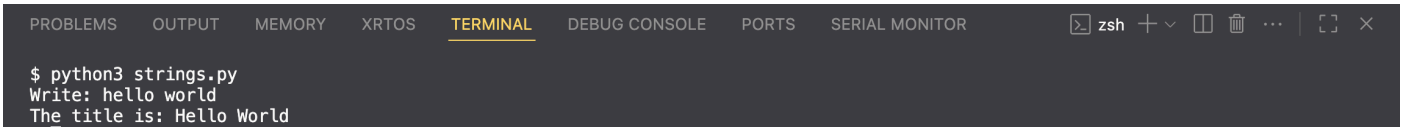


Figure 20 - Manic Monday Functions and Variables | Result

```python
import unittest
from unittest.mock import patch
import io
import sys
import strings as tob
def prompt_input(s: str, fn) -> any:
 # fn is function call with user input
 # s = the user input after function call fn
 with io.StringIO(s) as handle:
  stdin = sys.stdin
  sys.stdin = handle
  # function call with return value
  return_val = fn()
  sys.stdin = stdin
  return return_val

def prompt_output(fn=None) -> str:
 # catches prompt output after running function fn
 with patch('sys.stdout', new = io.StringIO()) as mo:
  if not fn is None:
   fn()
  s = mo.getvalue()
 return s
class Testing(unittest.TestCase):
 def test_ask(self):
  a = "dit is een input test"
  aa = prompt_input(a, tob.ask)
  b = "dit is een input test"
  self.assertEqual(aa, b)

 def test_make_title_lower(self):
  a = "dit is een output test"
  aa = tob.titlize(a)
  b = "Dit Is Een Output Test"
  self.assertEqual(aa, b)

 def test_make_title_upper(self):
  a = "dit is EEN OUTPUT test MET Capitals"
  aa = tob.titlize(a)
  b = "Dit Is Een Output Test Met Capitals"
  self.assertEqual(aa, b)
if __name__ == '__main__':
 unittest.main()
```

```
$ pytest test_strings.py
============================= test session starts ==============================
platform darwin -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period
collected 3 items

test_strings.py ...                                                      [100%]

============================== 3 passed in 0.03s ===============================
```

Fuel Gauge

Converts a fraction (e.g. "3/4") to a percentage and prints "F" for ≥ 99 %, "E" for ≤ 1 %, or the percentage otherwise. Input is validated inside a while True loop using try/except to catch both ValueError (non-numeric input) and ZeroDivisionError. Demonstrates robust input validation and exception-based control flow.

```python
while True:
    try:
        fraction = input("Fraction: ")
        x, z = fraction.split("/")
        n1 = int(x)
        n2 = int(z)
        # validação obrigatória
        if n1 < 0 or n2 <= 0 or n1 > n2:
            continue
        percent = round((n1 / n2) * 100)
        if percent >= 99:
            print("F")
        elif percent <= 1:
            print("E")
        else:
            print(f"{percent}%")
        break
    except (ValueError, ZeroDivisionError):
        pass
```

```
$ cd fuel
$ python3 fuel.py
Fraction: 3/4
75%
$ python3 fuel.py
Fraction: 1/4
25%
$ python3 fuel.py
Fraction: 4/4
F
$ python3 fuel.py
Fraction: 0/4
E
$ python3 fuel.py
Fraction: 4/0
Fraction: three/four
Fraction: 1.5/3
Fraction: -3/4
Fraction: 5/4
Fraction: 2/3
67%
```

Figure 21 - Fuel Gauge | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / fuel

#1 submitted a few seconds ago, Monday, February 16, 2026 10:52 AM CET
check50 15/15 • 0 comments
tar.gz • zip

Felipe's Taqueria

Simulates an ordering system for a Mexican restaurant. A dictionary maps menu items to prices. The program uses .title() to normalise input capitalisation, accumulates a running total, and terminates on EOFError (Ctrl+D), a common Unix pattern for signalling end of input. Unknown items are silently ignored.

```
total = 0
while True:
    items = {
        "Baja Taco": 4.25,
        "Burrito": 7.50,
        "Bowl": 8.50,
        "Nachos": 11.00,
        "Quesadilla": 8.50,
        "Super Burrito": 8.50,
        "Super Quesadilla": 9.50,
        "Taco": 3.00,
        "Tortilla Salad": 8.00
    }
    try:
        item = input("Item: ").title()
        if item in items:
            total += items[item]
            print(f"Total ${total:.2f}")
    except EOFError:
        print()
        break
```

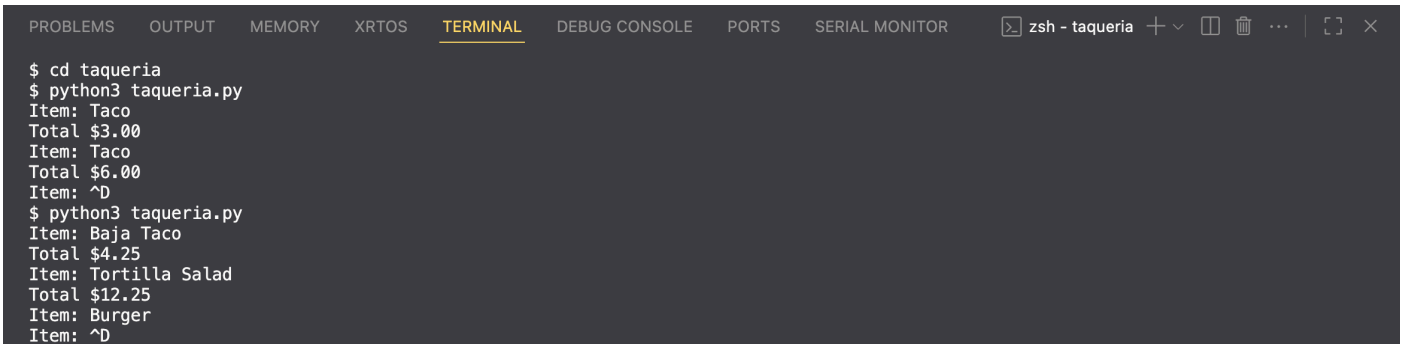
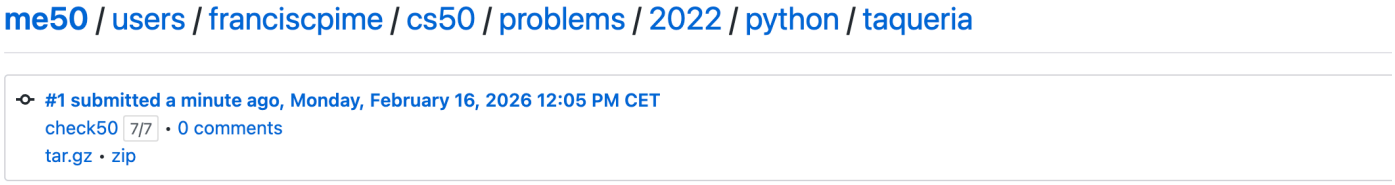


Figure 22 - Felipe's Taqueria | Result



Grocery List

Reads grocery items from stdin until EOFError and produces a sorted, de-duplicated count list in upper case. Items are stored in a dictionary mapping item name to count; the output is generated by iterating over the sorted keys. Demonstrates dictionary-based frequency counting and sorted() with string keys.

```
groceries = {}
try:
    while True:
        item = input().upper()
        if item in groceries:
            groceries[item] += 1
        else:
            groceries[item] = 1
except EOFError:
    pass
    print()
for item in sorted(groceries):
    print(groceries[item], item)
```

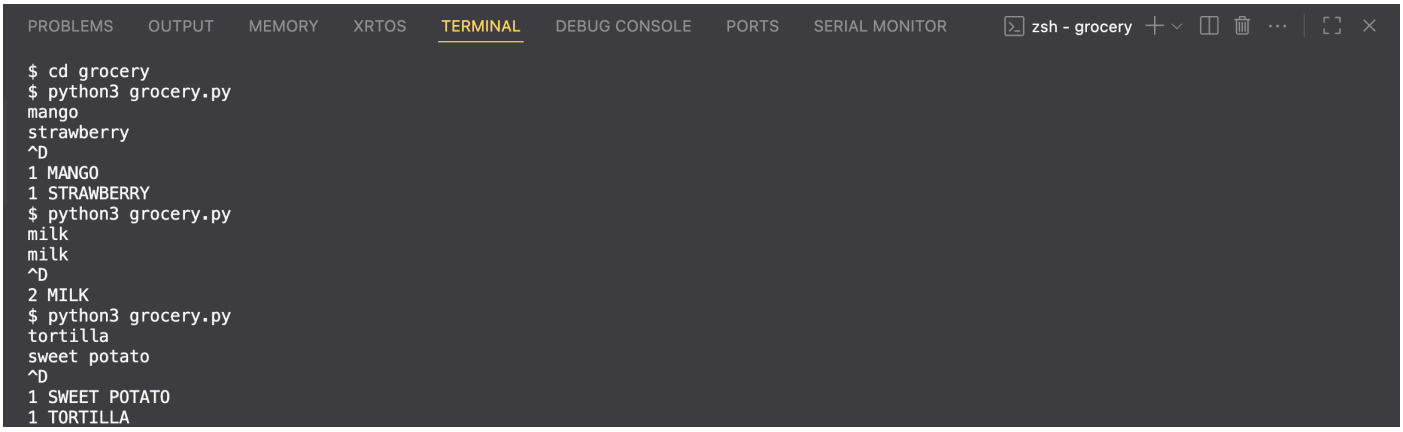
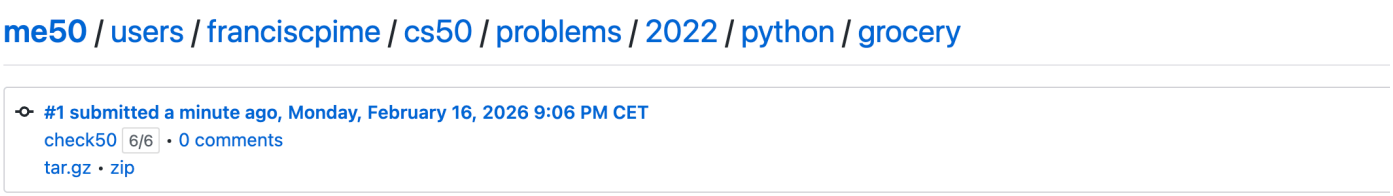


Figure 23 - Grocery List | Result



Outdated

Converts dates from two formats, "Month D, YYYY" (textual) and "M/D/YYYY" (numeric), into the ISO 8601 format "YYYY-MM-DD". A months dictionary maps month names to their numeric equivalents. The program validates both the month range and day range, and loops until valid input is provided.

```
months = {
    "January": 1,
    "February": 2,
    "March": 3,
    "April": 4,
    "May": 5,
    "June": 6,
    "July": 7,
    "August": 8,
    "September": 9,
    "October": 10,
    "November": 11,
    "December": 12
}
while True:
    try:
        date = input("Date: ").strip()
        # formato numérico
        if "/" in date:
            month, day, year = date.split("/")
            month = int(month)
            day = int(day)
            year = int(year)
        # formato textual
        else:
            month, day, year = date.split()
            # TEM de ter vírgula
            if not day.endswith(","):
                continue
            day = day.replace(",", "")
            month = months[month]
            day = int(day)
            year = int(year)
        # validações
        if not (1 <= month <= 12 and 1 <= day <= 31):
            continue
        print(f"{year:04}-{month:02}-{day:02}")
        break
    except (ValueError, KeyError):
        continue
```

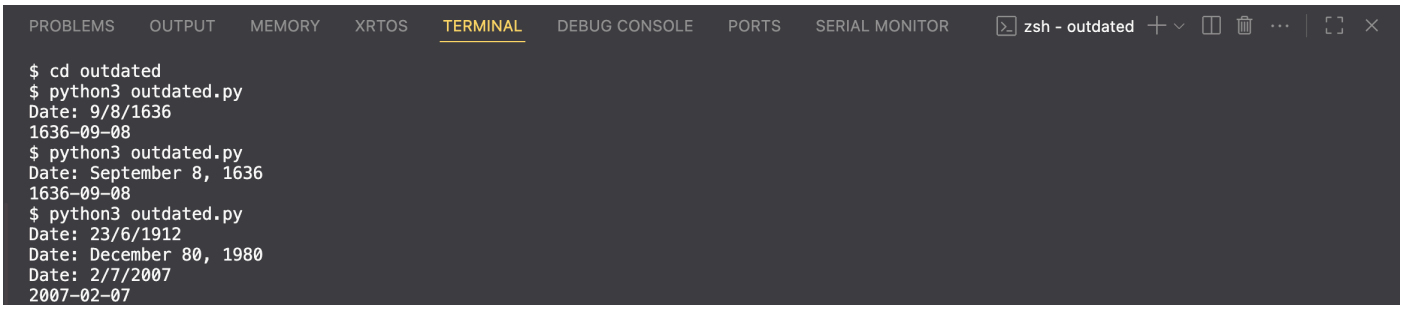
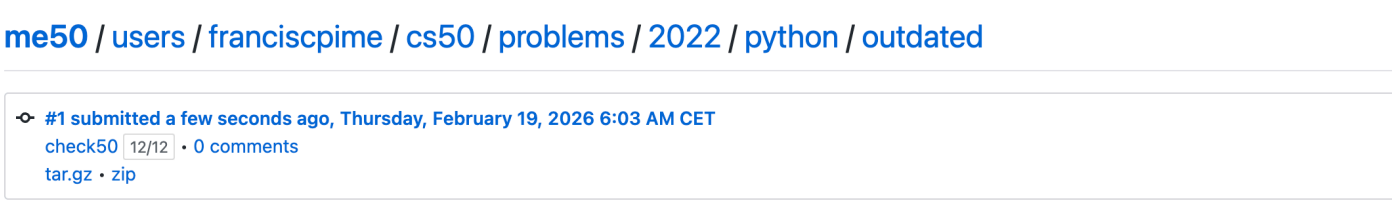


Figure 24 - Outdated | Result



Survival Simulator

A creative assignment that wraps several Week 1 and Week 2 concepts into a text-based game. The player navigates a supermarket by completing ten mini-challenges, each of which reproduces a prior assignment (File Extensions, Math Interpreter, Coke Machine, Guessing Game, etc.) within the game context. A progress dictionary tracks completion status, and a visual progress bar is rendered in the terminal. The game ends either with escape (all challenges complete) or game-over (premature exit attempt).

```
import random
progress = {
    "extension": False,
    "math": False,
    "meal": False,
    "coke": False,
    "camel": False,
    "outdated": False,
    "guessing": False,
    "bitcoin": False,
    "emojize": False,
    "felipe": False
}
WIDTH = 60
def line():
    print("=" * WIDTH)
def title(text):
    line()
    print(text.center(WIDTH))
    line()
def progress_bar(completed, total):
    percent = completed / total
    bars = int(percent * 30)
    bar = "🟥" * bars + "-" * (30 - bars)
    print(f"Progress: [{bar}] {completed}/{total}".center(WIDTH))
def pause():
```

```
    input("\nPress Enter to continue...")
def main():
    title("SURVIVAL SUPERMARKET SIMULATOR")
    print()
    print("Escape by completing all challenges!".center(WIDTH))
    print()
    menu()
def menu():
    while True:
        completed = sum(progress.values())
        total = len(progress)
        title("MAIN MENU")
        progress_bar(completed, total)
        print()
        options = [
            "1   | File Extensions",
            "2   | Math Interpreter",
            "3   | Nutrition Facts",
            "4   | Coke Machine",
            "5   | CamelCase Decoder",
            "6   | Outdated Products",
            "7   | Guessing Game",
            "8   | Bitcoin Investment",
            "9   | Emojize",
            "10  | Felipe's Taqueria",
            "11  | Exit"
        ]
        print("=" * WIDTH)
        for option in options:
            print("  " + option)
        print("=" * WIDTH)
        choice = input("\nSelect option (1-11): ")
        if choice == "1":
            extension()
        elif choice == "2":
            math_game()
        elif choice == "3":
            meal()
        elif choice == "4":
            coke()
        elif choice == "5":
            camelCase()
        elif choice == "6":
            outdated()
        elif choice == "7":
            guessing()
        elif choice == "8":
            bitcoin()
        elif choice == "9":
            emojize()
        elif choice == "10":
            felipe()
        elif choice == "11":
            completed = sum(progress.values())
            total = len(progress)
            if completed == total:
                title("ESCAPE SUCCESSFUL")
                print("YOU COMPLETED ALL CHALLENGES!".center(WIDTH))
                print("🎉 CONGRATULATIONS! 🎉".center(WIDTH))
                print("You escaped the supermarket.".center(WIDTH))
```

```
            else:
                title("GAME OVER")
                print("You tried to escape too early.".center(WIDTH))
                print(f"You completed {completed}/{total} challenges.".center(WIDTH))
                print("You lost.".center(WIDTH))
            break
def extension():
    title("FILE EXTENSIONS")
    file = input("Enter file name: ")
    print()
    if file.lower().endswith(".pdf"):
        print("PDF file detected. Recipe unlocked!".center(WIDTH))
        progress["extension"] = True
    else:
        print("Unknown file type.".center(WIDTH))
    pause()
def math_game():
    title("MATH INTERPRETER")
    print("Safe unlocks if result equals 14\n".center(WIDTH))
    while True:
        expression = input("Expression: ")
        allowed = "0123456789+-*/ "
        if any(c not in allowed for c in expression):
            print("Invalid characters!".center(WIDTH))
            continue
        try:
            result = eval(expression)
            if result == 14:
                print("Correct! Safe opened.".center(WIDTH))
                progress["math"] = True
                break
            else:
                print("Wrong result.".center(WIDTH))
        except:
            print("Invalid expression!".center(WIDTH))
    pause()
def meal():
    title("NUTRITION TABLE")
    foods = {
        "apple": 95,
        "banana": 105,
        "cookie": 200,
        "sandwich": 300,
        "water": 0
    }
    print("Food".ljust(20) + "Calories".rjust(10))
    print("-" * 30)
    for f, c in foods.items():
        print(f.ljust(20) + str(c).rjust(10))
    print("\nTarget: 500 calories\n")
    total = 0
    while total < 500:
        food = input("Choose food: ").lower()
        if food in foods:
            total += foods[food]
            print(f"Total: {total}".center(WIDTH))
        else:
            print("Invalid food.".center(WIDTH))
        if total > 500:
            print("Too many calories!".center(WIDTH))
```

```
            pause()
            return
    print("Perfect balance achieved!".center(WIDTH))
    progress["meal"] = True
    pause()
def coke():
    title("COKE MACHINE")
    price = 50
    while price > 0:
        print(f"Amount Due: {price}".center(WIDTH))
        try:
            amount = int(input("Insert coin (5/10/25): "))
        except:
            continue
        if amount in [5, 10, 25]:
            price -= amount
    change = abs(price)
    print(f"Change Owed: {change}".center(WIDTH))
    progress["coke"] = True
    pause()
def camelCase():
    title("CAMELCASE DECODER")
    message = input("Enter message: ")
    transformed = ""
    for i, c in enumerate(message):
        if c.isupper() and i != 0:
            transformed += " " + c
        else:
            transformed += c
    print("\nDecoded:".center(WIDTH))
    print(transformed.center(WIDTH))
    progress["camel"] = True
    pause()
def outdated():
    title("OUTDATED PRODUCTS")
    items = ["Milk", "Apple", "Lettuce", "Bread", "Yogurt", "Meat",
             "Fish", "Cereals", "Tomato", "Banana", "Cheese"]
    print(", ".join(items))
    print("\nDairy products are expired.\n")
    expired = {"Milk", "Yogurt", "Cheese"}
    answer = input("Expired items (comma separated): ")
    user_set = set(item.strip().title() for item in answer.split(","))
    if user_set == expired:
        print("Correct!".center(WIDTH))
        progress["outdated"] = True
    else:
        print("Not correct.".center(WIDTH))
    pause()
def guessing():
    title("GUESSING GAME")
    answer = 7
    attempts = 3
    for i in range(attempts):
        try:
            guess = int(input(f"Attempt {i+1}/3: "))
        except:
            continue
        if guess < answer:
            print("Too low!".center(WIDTH))
        elif guess > answer:
```

```
            print("Too high!".center(WIDTH))
        else:
            print("Correct!".center(WIDTH))
            progress["guessing"] = True
            pause()
            return
    print("You failed.".center(WIDTH))
    pause()
def bitcoin():
    title("BITCOIN INVESTMENT")
    price = random.randint(20000, 60000)
    print(f"Current price: ${price}\n".center(WIDTH))
    decision = input("Buy? (yes/no): ").lower()
    future = price + random.randint(-10000, 15000)
    print(f"\nLater price: ${future}".center(WIDTH))
    if decision == "yes":
        if future > price:
            print("Profit!".center(WIDTH))
        else:
            print("You lost money.".center(WIDTH))
    else:
        print("No investment made.".center(WIDTH))
    progress["bitcoin"] = True
    pause()
def emojize():
    title("EMOJIZE")
    print("Clue: 🧠 🦙 🦁\n")
    print("The word formed is a COUNTRY.\n")
    attempts = 0
    while True:
        answer = input("Country: ").lower().strip()
        attempts += 1
        if answer == "brazil":
            print("Correct!".center(WIDTH))
            progress["emojize"] = True
            pause()
            return
        if attempts == 1:
            print("\nHint: It is in South America.".center(WIDTH))
        elif attempts == 2:
            print("\nHint: It starts with 'B' and has 6 letters.".center(WIDTH))
        else:
            print("\nWrong.".center(WIDTH))
def felipe():
    title("FELIPE'S TAQUERIA")
    ingredients = ["tortilla", "beef", "chicken",
                   "cheese", "lettuce", "tomato"]
    print("Available ingredients:\n")
    print(", ".join(ingredients))
    print()
    while True:
        item = input("Add ingredient (or done): ").lower()
        if item == "done":
            break
        if item in ingredients:
            print(f"{item} added.".center(WIDTH))
        else:
            print("Not available.".center(WIDTH))
    print("\nYour taco is ready!".center(WIDTH))
    progress["felipe"] = True
```

```
    pause()
main()
```

Week 3

Week 3 formalises testing practices by switching from unittest to pytest and by requiring that each significant function be covered by an independent test module. The assignments also introduce third-party libraries (pyjokes, emoji, pyfiglet) and practice passing command-line arguments with sys.argv.

Manic Monday - Exceptions and Libraries

Extends the Manic Monday pattern by introducing command-line argument parsing. The program reads an optional language flag (e.g. -fr) from sys.argv and fetches a programmer joke in the specified language using the pyjokes library. The read_arg() function handles edge cases such as missing flags, non-existent languages, and uppercase input. Seven pytest test cases verify all edge cases, achieving 100% pass on the first run.

```
import sys
import pyjokes
LANGUAGES = ["cs", "de", "en", "es", "eu", "fr", "gl", "hu", "it", "lt", "pl", "ru", "sv"]
def read_arg():

    default = "en"
    # If no extra argument is entered
    if len(sys.argv) < 2:
         return default
    # If the extra argument is in uppercase, handle as if lowercase
    arg = sys.argv[1].lower()
    # Must start with a "-"" followed by 2 letters for the language
    if not (arg.startswith("-") and len(arg) == 3):
         return default

    # Remove "-"
    lang = arg[1:]
    if lang not in LANGUAGES:           # If a non-existing language is entered
         return default
    return lang
def get_single_joke(lang):
    try:
         joke = pyjokes.get_joke(language=lang)
         return joke
    except Exception:
         return "Bad joke"
def main():
    lang = read_arg()
    joke = get_single_joke(lang)
    print(joke)
if __name__ == "__main__":
 main()
```

```python
import sys
import jokes as tob
def test_args_no():
 sys.argv = ["",]
 l = tob.read_arg()
 assert l == "en"
def test_args_more():
 sys.argv = ["", "-de", "-fr"]
 l = tob.read_arg()
 assert l == "de"
 sys.argv = ["", "de", "-fr"]
 l = tob.read_arg()
 assert l == "en"
def test_args_upper():
 sys.argv = ["", "-FR"]
 l = tob.read_arg()
 assert l == "fr"
 sys.argv = ["", "-Fr"]
 l = tob.read_arg()
 assert l == "fr"
def test_args_minus():
 sys.argv = ["", "de"]
 l = tob.read_arg()
 assert l == "en"
 sys.argv = ["", "-fr"]
 l = tob.read_arg()
 assert l == "fr"
def test_args_list():
 sys.argv = ["", "-kl"]
 l = tob.read_arg()
 assert l == "en"
 sys.argv = ["", "-nl"]
 l = tob.read_arg()
 assert l == "en"
def test_joke_ok():
 j = tob.get_single_joke("de")
 assert type(j) == str
def test_joke_wrong():
 j = tob.get_single_joke("kl")
 assert j.lower() == "Bad joke".lower()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR                    zsh

(venv) $ python3 jokes.py
Optimist: The glass is half full. Pessimist: The glass is half empty. Programmer: The glass is twice as large as necessary.
(venv) $ python3 jokes.py -fr
Comment comparer HTML et HTML5? Est-ce que cela fonctionne avec Internet Explorer? Non? C'est du HTML5.
(venv) $ python3 jokes.py -es
¿Qué es un terapeuta? 1024 Gigapeutas.
(venv) $
```

Figure 25 - Manic Monday - Exceptions and Libraries | Result

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR                    zsh

(venv) $ python -m pytest test_jokes.py
============================================================ test session starts ============================================================
platform darwin -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period
collected 7 items

test_jokes.py .......                                                                                                          [100%]

============================================================ 7 passed in 0.01s =============================================================
```

Figure 26 - Manic Monday - Exceptions and Libraries | Pytest

Testing my Twttr

Refactors the "Just Setting Up My Twttr" solution so that the core logic lives in a testable shorten() function. Five pytest tests cover lower-case vowels, digits (which should be preserved), spaces, upper-case vowels, and punctuation marks, providing complete branch coverage.

```
def main():
    message = input("Input: ")
    print(f"Output: {shorten(message)}")

def shorten(word):
    vowels = "aeiouAEIOU"
    transformed = ""
    for c in word:
        if c not in vowels:
            transformed += c
        else:
            transformed += ""

    return transformed
if __name__ == "__main__":
    main()
```

```
import twttr
from twttr import shorten
def test_vowels_lower():
    assert shorten("Francisco") == "Frncsc"
def test_numbers():
    assert shorten("1") == "1"
def test_space():
    assert shorten(" ") == " "
def test_vowels_upper():
    assert shorten("HOLA") == "HL"
def test_punctuation():
    assert shorten("Hi!") == "H!"
```

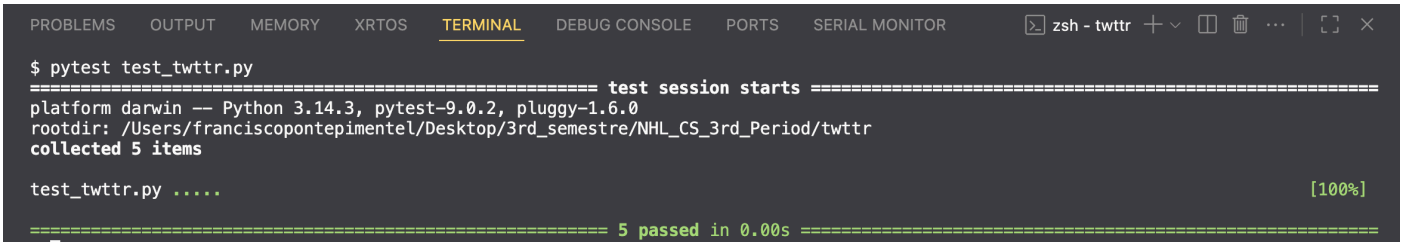
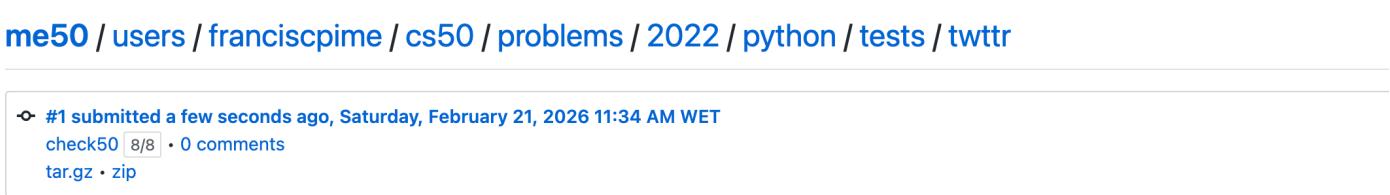


Figure 27 - Testing my twttr | Result



Back to the Bank

Refactors the "Home Federal Savings Bank" solution with a value() function and four pytest tests. The tests cover the exact "hello" case, a greeting starting with "h" but not "hello", a non-H greeting, and a complete sentence starting with "Hello", confirming that startswith() correctly handles multi-word inputs.

```
def main():
    greeting = input("Greeting: ")
    print(value(greeting))
def value(greeting):
    if greeting.strip().lower().startswith("hello"):
        return 0
    elif greeting.strip().lower().startswith("h"):
        return 20
    else:
        return 100
if __name__ == "__main__":
    main()
```

```
import bank
from bank import value
def test_specific():
    assert value("hello") == 0
def test_start_h():
    assert value("holla") == 20
def test_others():
    assert value("Salut") == 100
def test_complete_sentence():
    assert value("Hello my friend") == 0
```

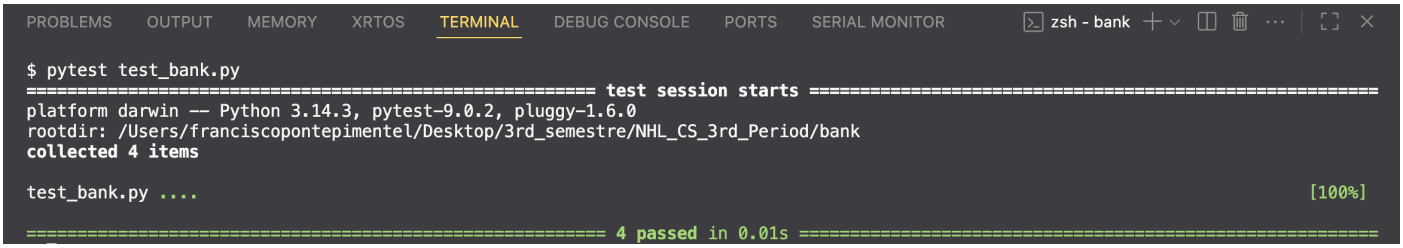


Figure 28 - Back to the Bank | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / tests / bank

#1 submitted a minute ago, Saturday, February 21, 2026 12:42 PM WET
check50 5/5 • 0 comments
tar.gz • zip

Re-requesting a Vanity Plate

Adds a pytest test suite (six test functions) to the Vanity Plates is_valid() function. Tests cover: too short / too long strings, first two characters not being letters, non-alphanumeric characters, a leading zero in the numeric segment, numbers followed by letters, and valid plates. Demonstrates the importance of negative test cases alongside positive ones.

```
def main():
    plate = input("Plate: ")
    if is_valid(plate):
        print("Valid")
    else:
        print("Invalid")
def is_valid(s):
    # size
    if not (2 <= len(s) <= 6):
        return False
    # first 2 characters are letters
    if not s[0].isalpha() or not s[1].isalpha():
        return False
    # just letters and numbers
    if not s.isalnum():
        return False
    # numbers in the end
    number_started = False
    for c in s:
        if c.isdigit():
            # first number can't be 0
            if not number_started:
                if c == "0":
                    return False
                number_started = True
        else:
            if number_started:
                return False
    return True
if __name__ == "__main__":
    main()
```

```
import plates
from plates import is_valid
def test_size():
    assert is_valid("A") == False
    assert is_valid("ABCDEFG") == False
def test_letters():
    assert is_valid("A1") == False
    assert is_valid("1A") == False
def test_isalnum():
    assert is_valid(" ") == False
    assert is_valid("AA@12") == False
def test_zero():
    assert is_valid("AA0") == False
def test_number_placement():
    assert is_valid("AA1A") == False
    assert is_valid("AB12C") == False
```

```
def test_valid_cases():
    assert is_valid("CS50") == True
    assert is_valid("HELLO") == True
    assert is_valid("AB123") == True
```

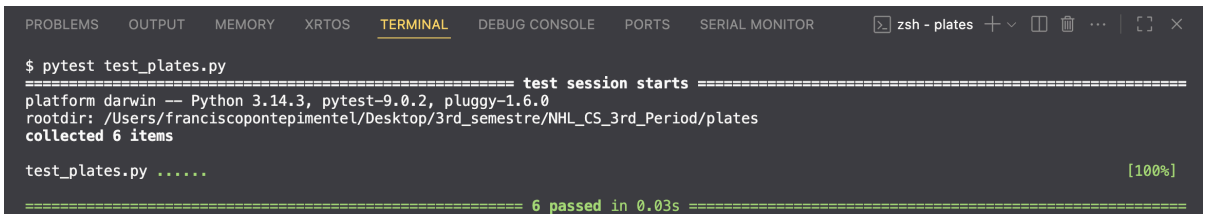


Figure 29 - Re-requesting a Vanity Plate | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / tests / plates

#1 submitted a few seconds ago, Thursday, February 19, 2026 9:53 PM WET
check50 7/7 • 0 comments
tar.gz • zip

Refueling

Restructures the Fuel Gauge solution by extracting convert() (fraction >> percentage) and gauge() (percentage >> label) as separately testable functions. Four pytest tests exercise: a normal conversion, ValueError for invalid fractions (negative, non-numeric, denominator < numerator), ZeroDivisionError for zero denominator, and the boundary values 99 (>> "F") and 1 (>> "E").

```
def main():
    while True:
        try:
            fraction = input("Fraction: ")
            percentage = convert(fraction)
            result = gauge(percentage)
            print(result)
            break
        except (ValueError, ZeroDivisionError):
            pass
def convert(fraction):
    x, z = fraction.split("/")
    n1 = int(x)
    n2 = int(z)
    if n2 == 0:
        raise ZeroDivisionError
    if n1 < 0 or n2 < 0 or n1 > n2:
        raise ValueError
    percentage = round((n1 / n2) * 100)
    return percentage
def gauge(percentage):
    if percentage <= 1:
        return "E"
    elif percentage >= 99:
        return "F"
    else:
        return f"{percentage}%"
if __name__ == "__main__":
    main()
```

```python
import pytest
from fuel import convert, gauge
def test_normal():
    assert convert("1/2") == 50
    assert gauge(50) == "50%"
def test_value_error():
    with pytest.raises(ValueError):
        convert("3/2")
    with pytest.raises(ValueError):
        convert("cat/dog")
    with pytest.raises(ValueError):
        convert("5/-1")

    with pytest.raises(ValueError):
        convert("-5/1")
    with pytest.raises(ValueError):
        convert("1.5/2")
def test_zero_division():
    with pytest.raises(ZeroDivisionError):
        convert("1/0")

def test_special():
    assert gauge(99) == "F"
    assert gauge(1) == "E"
```

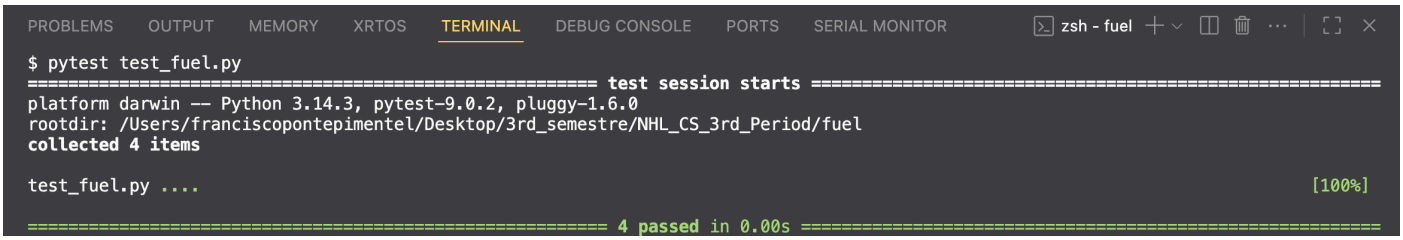


Figure 30 - Refueling | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / tests / fuel

#1 submitted a few seconds ago, Saturday, February 21, 2026 10:36 PM WET
check50 9/9 • 0 comments
tar.gz • zip

Emojize

Uses the emoji third-party library to convert alias strings such as ":smile_cat:" into their Unicode emoji equivalents. The language="alias" parameter enables the colon-delimited alias syntax. Demonstrates importing and using a third-party library with a single functional call.

```
import emoji
text = input("Input: ")
print("Output: ", emoji.emojize(text, language='alias'))
```

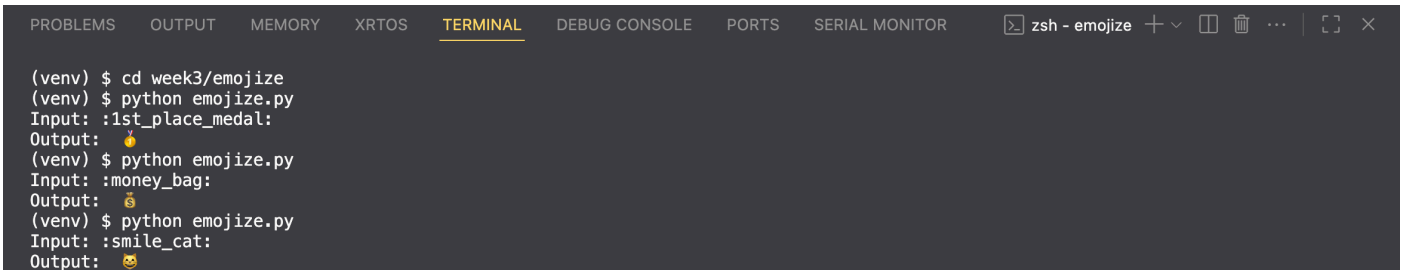
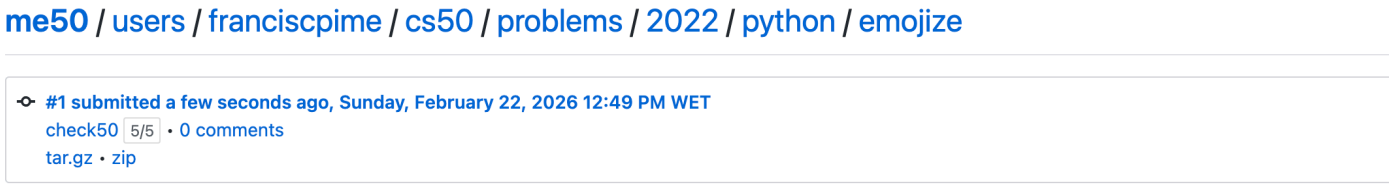


Figure 31 - Emojize | Result



Frank, Ian and Glen's Letters

Uses the pyfiglet library to render user-supplied text in large ASCII art fonts. If no argument is given, a random font is selected; if -f <font> is provided, the specified font is used. Invalid argument combinations cause the program to exit with a usage error via sys.exit(). Demonstrates defensive argument parsing and third-party ASCII art rendering.

```
import sys
import random
from pyfiglet import Figlet
def main():
    figlet = Figlet()
    fonts = figlet.getFonts()
    # 0 arguments
    if len(sys.argv) == 1:
        font = random.choice(fonts)
    # 2 arguments
    elif len(sys.argv) == 3:
        if sys.argv[1] in ["-f", "--font"] and sys.argv[2] in fonts:
            font = sys.argv[2]
        else:
            sys.exit("Invalid usage")
    # Other case
    else:
        sys.exit("Invalid usage")
    figlet.setFont(font=font)
    text = input("Input: ")
    print(figlet.renderText(text))
if __name__ == "__main__":
    main()
```

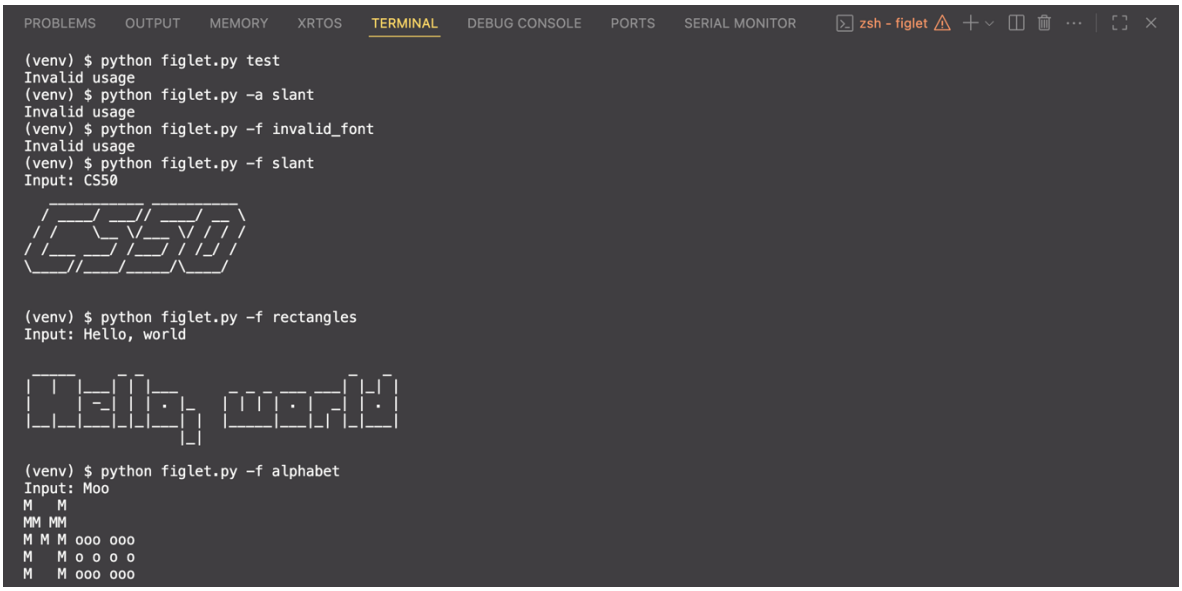


Figure 32 - Frank, Ian and Glen's Letters | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / figlet

#1 submitted 4 minutes ago, Monday, February 23, 2026 9:43 AM WET
check50 8/8 • 0 comments
tar.gz • zip

Adieu, Adieu

Collects names from the user until EOFError and then constructs a grammatically correct farewell. One name: "Adieu, adieu, to X"; two names: "… to X and Y"; three or more: "… to X, Y, and Z" using the Oxford comma. The join() slice pattern names[:-1] cleanly handles the comma-separated prefix.

```
names = []
while True:
    try:
        name = input("Name: ")
        names.append(name)
    except EOFError:
        print()
        break
if len(names) == 1:
    print(f"Adieu, adieu, to {names[0]}")
elif len(names) == 2:
    print(f"Adieu, adieu, to {names[0]} and {names[1]}")
elif len(names) >= 3:
    print(f"Adieu, adieu, to {', '.join(names[:-1])}, and {names[-1]}")
```

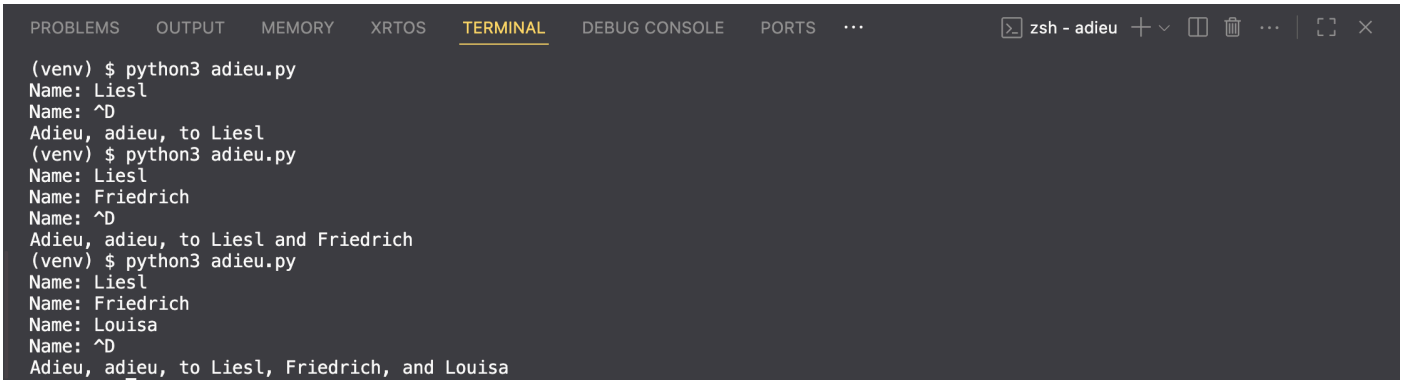


Figure 33 - Adieu, Adieu | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / adieu

#1 submitted a few seconds ago, Tuesday, February 24, 2026 7:51 PM WET
check50 9/9 • 0 comments
tar.gz • zip

Guessing Game

Generates a random integer between 1 and a user-specified upper bound, then prompts the user to guess. Feedback of "Too small!" or "Too large!" is given after each incorrect guess, and the game terminates on a correct answer. Uses random.randint() and demonstrates nested validation loops with exception handling for non-integer input.

```
import random
while True:
    try:
        level = int(input("Level: "))
        if level < 0:
            continue
        else:
            break
    except ValueError:
        pass
number = random.randint(1, level)
while True:
    try:
        guess = int(input("Guess: "))
        if guess <= 0:
            continue
        elif guess < number:
            print("Too small!")
        elif guess > number:
            print("Too large!")
        else:
            print("Just right!")
            break
    except ValueError:
        pass
```

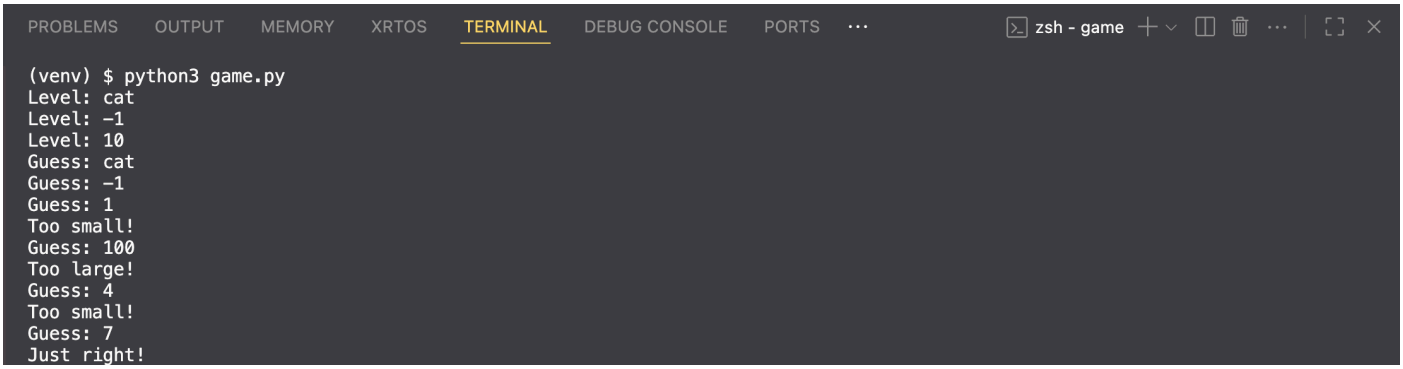
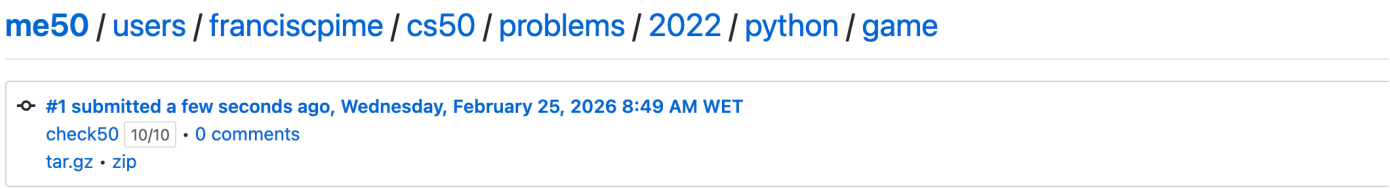


Figure 34 - Guessing Game | Result



Little Professor

Generates ten random addition problems at a chosen difficulty level (1-digit, 2-digit, or 3-digit numbers). The player has three attempts per problem; after three wrong answers the correct answer is revealed. A final score is printed. Demonstrates function decomposition with get_level() and generate_integer(), and the use of a nested loop with a tries counter.

```
import random
def main():
    level = get_level()
    score = 0
    for i in range(10):
        x = generate_integer(level)
        y = generate_integer(level)
        tries = 0
        answer = x + y
        while tries < 3:
            try:
                operation = int(input(f"{x} + {y} = "))
                if operation == answer:
                    score +=1
                    break
                else:
                    print("EEE")
                    tries += 1
            except ValueError:
                print("EEE")
                tries += 1

        if tries == 3:
            print(f"{x} + {y} = {answer}")

    print(score)

def get_level():
    while True:
        try:
            level = input("Level: ")
            right = "123"
            if level not in right:
                continue
            else:
                break

        except ValueError:
            pass
    return level

def generate_integer(level):
    level = int(level)

    if level == 1:
        number = random.randint(0, 9)
    elif level == 2:
        number = random.randint(10, 99)
    elif level == 3:
```

```
        number = random.randint(100, 999)
    return number
if __name__ == "__main__":
    main()
```

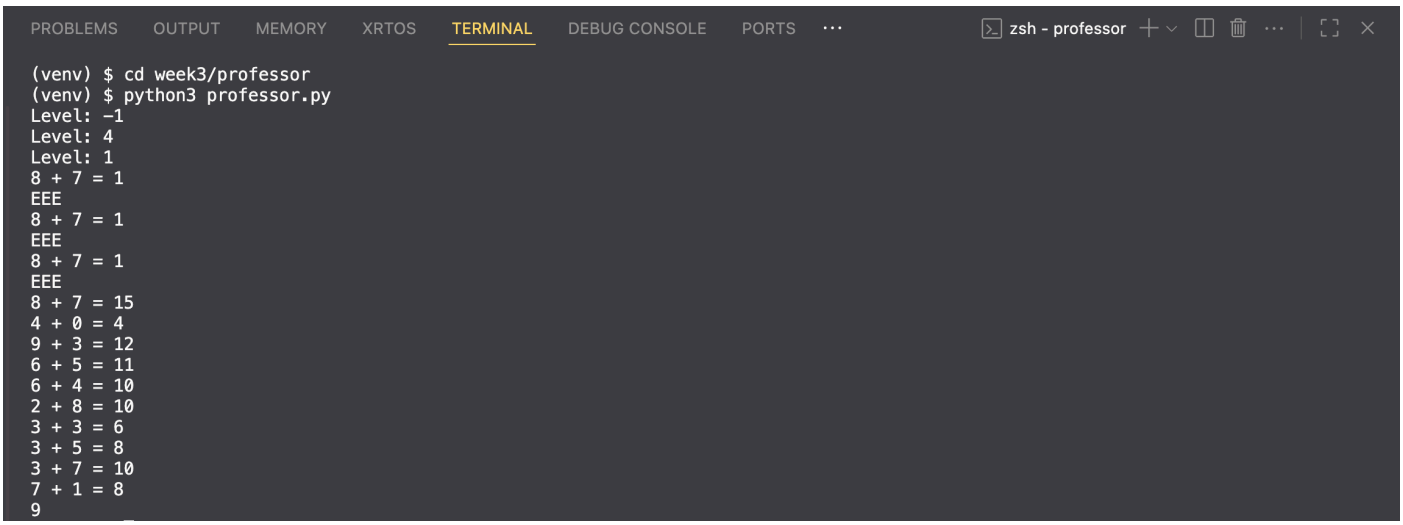


Figure 35 - Little Professor | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / professor

#1 submitted a minute ago, Friday, February 27, 2026 11:29 PM WET
check50 14/14 • 0 comments
tar.gz • zip

Week 4

Week 4 introduces the re module and the use of regular expressions to validate, search, and extract structured data from strings. Regex patterns make input validation concise and precise, replacing chains of string methods with declarative patterns.

Numb3rs

Validates IPv4 addresses by splitting on "." and checking each octet is a digit between 0 and 255, without a leading zero. While the validation logic intentionally avoids regex (to practise manual parsing), four pytest tests confirm edge cases: non-numeric input, octets > 255, leading zeros, and extra segments.

```python
import re
import sys
def main():
    print(validate(input("IPv4 Address: ")))
def validate(ip):
    parts = ip.split(".")
    if len(parts) != 4:
        return False
    for part in parts:
        if not part.isdigit():
            return False
        if len(part) > 1 and part.startswith("0"):
            return False
        number = int(part)
        if number > 255:
            return False
    return True
if __name__ == "__main__":
    main()
```

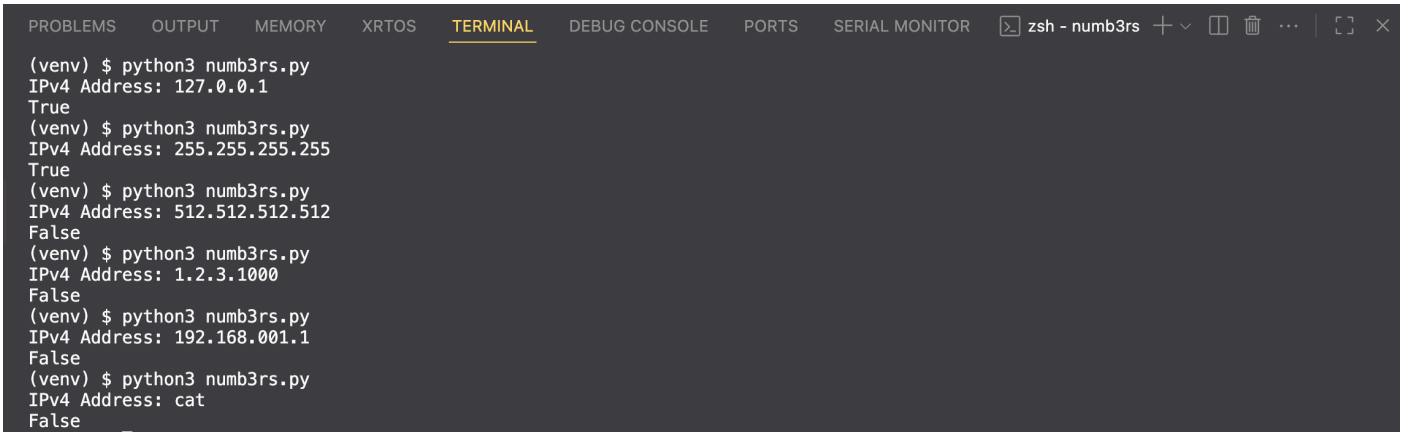


Figure 36 - Numb3rs | Result

```python
from numb3rs import validate
def test_string():
    assert validate("cat") == False
def test_more_255():
    assert validate("123.167.344.3") == False
def test_starts_with_0():
    assert validate("003.23.2.1") == False
def test_extra_number():
    assert validate("123.23.4.1.56") == False
```

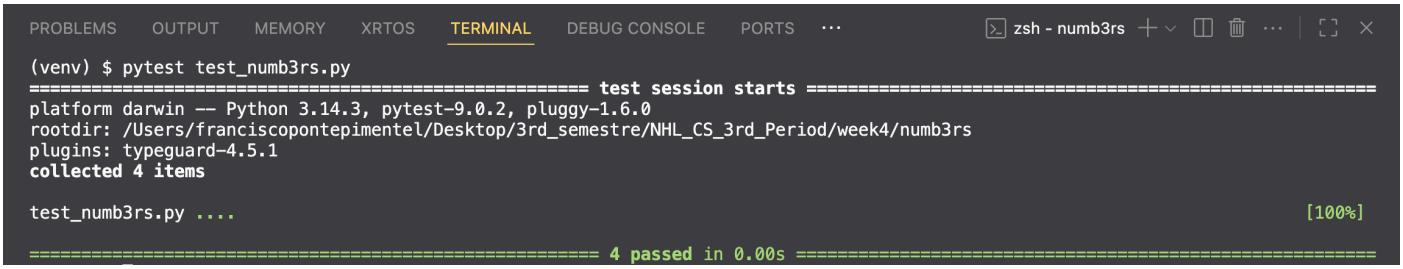
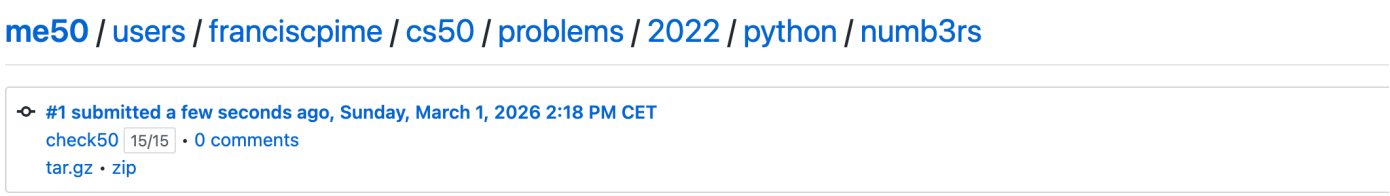


Figure 37 - Numb3rs | Test Result



Watch on Youtube

Extracts a YouTube video ID from an iframe embed snippet and converts it to a short youtu.be URL. A single re.search() call with a pattern that matches both http and https and the optional www. subdomain captures the video ID in group 1. Returns None for non-YouTube iframes.

```python
import re
import sys
def main():
    print(parse(input("HTML: ")))
def parse(s):
    match = re.search(
        r'<iframe[^>]*src="https?://(?:www\.)?youtube\.com/embed/([^"]+)"',
        s
    )
    if match:
        return f"https://youtu.be/{match.group(1)}"
    else:
        return None
if __name__ == "__main__":
    main()
```

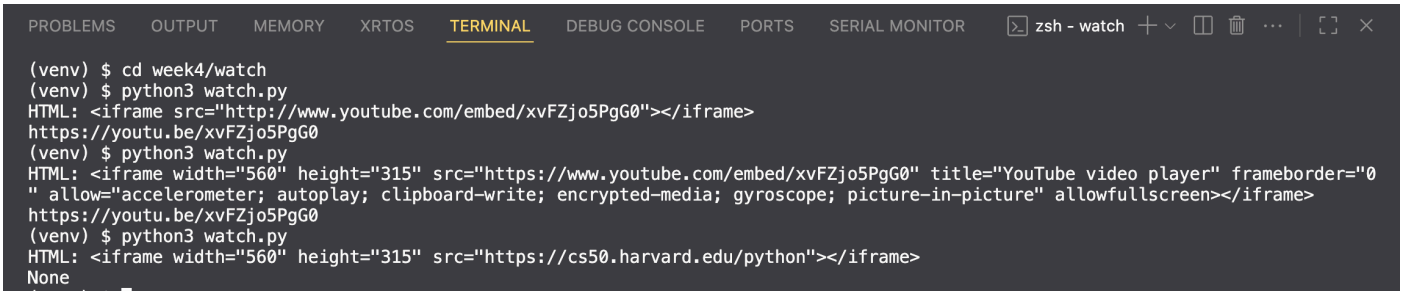


Figure 38 - Watch on Youtube | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / watch

#1 submitted a few seconds ago, Wednesday, March 4, 2026 8:32 PM CET
check50 11/11 • 0 comments
tar.gz • zip

Working 9 to 5

Converts a time range string such as "9 AM to 5 PM" or "9:30 AM to 5:45 PM" into 24-hour format ("09:00 to 17:00"). A single compiled regex pattern with optional minute groups captures both formats. The function raises ValueError on invalid formats or out-of-range values. Five pytest test functions cover hour-only, hour-and-minute, midnight/noon edge cases, invalid format, and invalid time values.

```
import re
def main():
    user_input = input("Hours: ")
    print(convert(user_input))
def convert(time_range):
    # Match formats like:
    # 9 AM to 5 PM
    # 9:30 AM to 5:45 PM
    pattern = r"^(\d{1,2})(?::(\d{2}))? (AM|PM) to (\d{1,2})(?::(\d{2}))? (AM|PM)$"
    match = re.search(pattern, time_range)
    # If the format doesn't match, raise an error
    if not match:
        raise ValueError
    # Extract captured groups
 start_hour, start_minute, start_period, end_hour, end_minute, end_period = match.groups()
    start_hour = int(start_hour)
    end_hour = int(end_hour)
    # If minutes are missing, assume 00
    start_minute = int(start_minute) if start_minute else 0
    end_minute = int(end_minute) if end_minute else 0
    # Validate time values
    if start_hour > 12 or end_hour > 12:
        raise ValueError
    if start_minute >= 60 or end_minute >= 60:
        raise ValueError
    # Convert start time to 24-hour format
    if start_period == "PM" and start_hour != 12:
```

```
        start_hour += 12
    if start_period == "AM" and start_hour == 12:
        start_hour = 0
    # Convert end time to 24-hour format
    if end_period == "PM" and end_hour != 12:
        end_hour += 12
    if end_period == "AM" and end_hour == 12:
        end_hour = 0
    return f"{start_hour:02}:{start_minute:02} to {end_hour:02}:{end_minute:02}"
if __name__ == "__main__":
    main()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  ···   zsh - working

(venv) $ cd week4/working
(venv) $ python3 working.py
Hours: 9 AM to 5 PM
09:00 to 17:00
(venv) $ python3 working.py
Hours: 9:00 AM to 5:00 PM
09:00 to 17:00
(venv) $ python3 working.py
Hours: 10 AM to 8:50 PM
10:00 to 20:50
(venv) $ python3 working.py
Hours: 10:30 PM to 8 AM
22:30 to 08:00
(venv) $ python3 working.py
Hours: 9:60 AM to 5:60 PM
Traceback (most recent call last):
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 51, in <module>
    main()
    ~~~~^^
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 5, in main
    print(convert(user_input))
          ~~~~~~~^^^^^^^^^^^^
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 33, in convert
    raise ValueError
ValueError
(venv) $ python3 working.py
Hours: 9 AM - 5 PM
Traceback (most recent call last):
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 51, in <module>
    main()
    ~~~~^^
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 5, in main
    print(convert(user_input))
          ~~~~~~~^^^^^^^^^^^^
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 17, in convert
    raise ValueError
ValueError
(venv) $ python3 working.py
Hours: 09:00 AM - 17:00 PM
Traceback (most recent call last):
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 51, in <module>
    main()
    ~~~~^^
  File "/Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working/working.py", line 5, in main
    print(convert(user_input))
          ~~~~~~~^^^^^^^^^^^^
```

Figure 39 - Working 9 to 5 | Result

```
import pytest
from working import convert
def test_hours_only():
    assert convert("9 AM to 5 PM") == "09:00 to 17:00"
    assert convert("10 PM to 8 AM") == "22:00 to 08:00"
def test_hours_and_minutes():
    assert convert("9:30 AM to 5:45 PM") == "09:30 to 17:45"
    assert convert("12:15 PM to 1:05 AM") == "12:15 to 01:05"
def test_midnight_and_noon():
    assert convert("12 AM to 12 PM") == "00:00 to 12:00"
    assert convert("12 PM to 12 AM") == "12:00 to 00:00"
def test_invalid_format():
    with pytest.raises(ValueError):
```

```
        convert("9AM to 5PM")
    with pytest.raises(ValueError):
        convert("9 AM - 5 PM")
def test_invalid_time():
    with pytest.raises(ValueError):
        convert("13 AM to 5 PM")
    with pytest.raises(ValueError):
        convert("9:60 AM to 5 PM")
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR  zsh - working

(venv) $ pytest test_working.py
============================================== test session starts ==============================================
platform darwin -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/franciscopontepimentel/Desktop/3rd_semestre/NHL_CS_3rd_Period/week4/working
plugins: typeguard-4.5.1
collected 5 items

test_working.py .....                                                                          [100%]

=============================================== 5 passed in 0.01s ===============================================
```

Figure 40 - Working 9 to 5 | Test Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / working

#1 submitted a few seconds ago, Wednesday, March 4, 2026 10:44 PM CET
check50 19/19 • 0 comments
tar.gz • zip

Regular, um, Expressions

Counts occurrences of the word "um" as a standalone word (not inside longer words like "album") in a given text. re.findall() with the \bum\b pattern and the re.IGNORECASE flag handles all cases. Five pytest tests verify single occurrence, multiple occurrences, case-insensitivity, non-word-boundary rejection, and mixed text.

```
import re
import sys
def main():
    text = input("Text: ")
    print(count(text))
def count(text):
    # Find occurrences of "um" as a standalone word (case-insensitive)
    matches = re.findall(r"\bum\b", text, re.IGNORECASE)
    # Return how many were found
    return len(matches)
if __name__ == "__main__":
    main()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  SERIAL MONITOR  zsh - um

(venv) $ python3 um.py
Text: um
1
(venv) $ python3 um.py
Text: um?
1
(venv) $ python3 um.py
Text: Um, thanks for the album.
1
(venv) $ python3 um.py
Text: Um, thanks, um...
2
```

Figure 41 - Regular, um, Expressions | Result

```python
from um import count
def test_single_um():
    assert count("um") == 1
def test_multiple_um():
    assert count("um, um, um") == 3
def test_case_insensitive():
    assert count("Um, thanks for the album.") == 1
def test_not_inside_words():
    assert count("album yummy umbrella") == 0
def test_mixed():
    assert count("Um, thanks, um...") == 2
```

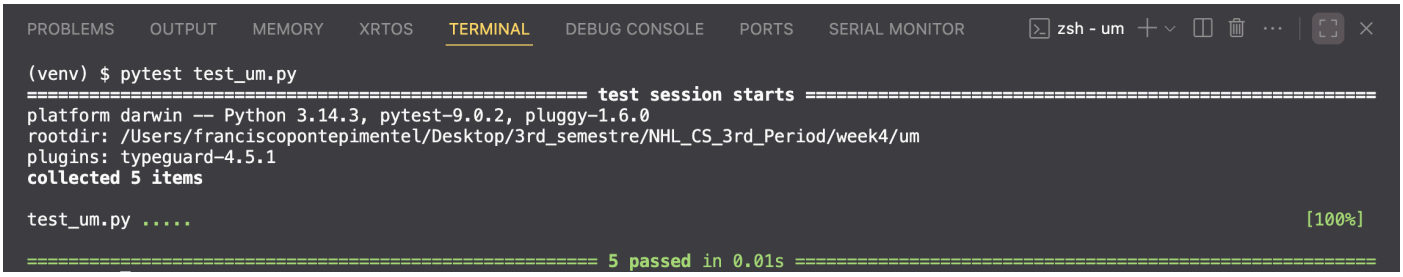
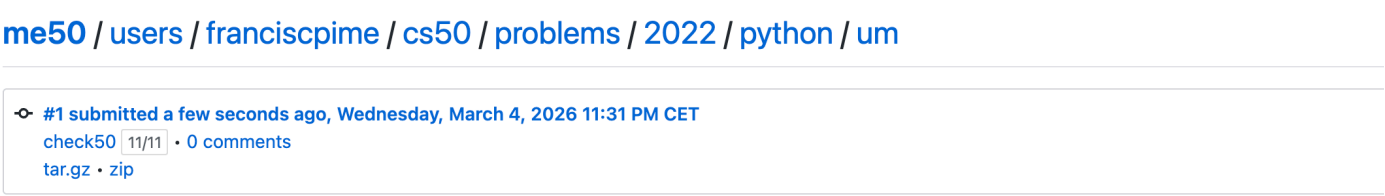


Figure 42 - Regular, um, Expressions | Test Result



Response Validation

Uses the validators third-party library to check whether a string is a syntactically valid email address. Demonstrates that many real-world validation problems are best solved by a well-tested library rather than a custom regex.

```python
import validators
def main():
    email = input("What's your email address? ")
    if validators.email(email):
        print("Valid")
    else:
        print("Invalid")
if __name__ == "__main__":
    main()
```

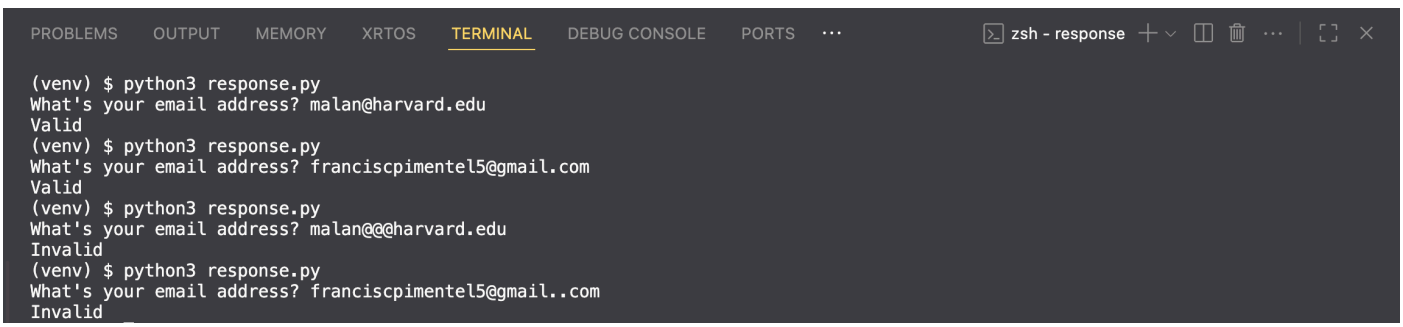


Figure 43 - Response Validation | Result

me50 / users / franciscpime / cs50 / problems / 2022 / python / response

#1 submitted a few seconds ago, Wednesday, March 4, 2026 11:43 PM CET
check50 6/6 • 0 comments
tar.gz • zip

Week 5

Week 5 is a scavenger hunt consisting of several self-contained mini-challenges designed to consolidate skills from previous weeks. Each challenge reinforces a specific concept: email validation, FizzBuzz logic, palindrome detection, string analysis, unit testing, and image manipulation with Pillow.

Scavenger Hunt

Validate Email Address

A lightweight email validator that checks for exactly one "@" symbol, non-empty local and domain parts, and the presence of a "." in the domain. Demonstrates incremental validation with early returns.

```python
def main() -> None:
    email = input("Enter ur email: ")
    if is_valid_email(email):
        print("Nice")
    else:
        print("Bad")

def is_valid_email(email: str) -> bool:
    if email.count("@") != 1:
        return False
    part1, part2 = email.split("@")
    if len(part1) < 1 or len(part2) < 1:
        return False
    if "." not in part2:
        return False

    return True
if __name__ == "__main__":
    main()
```

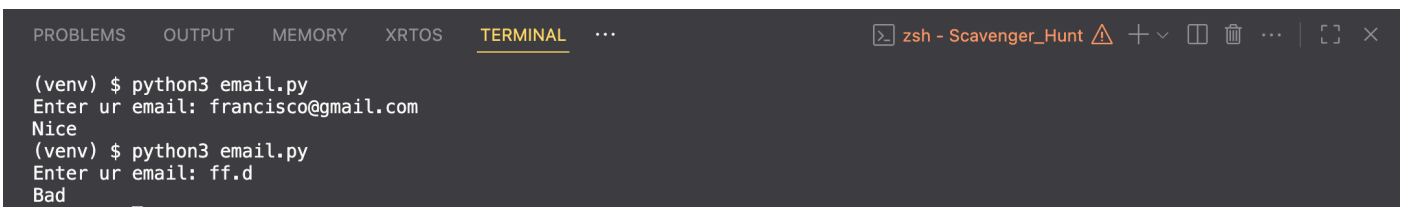


Figure 44 - Validate Email Address | Result

FizzBuzz

Classic FizzBuzz: returns "FizzBuzz" for multiples of both 3 and 5, "Fizz" for multiples of 3, "Buzz" for multiples of 5, and the number as a string otherwise. The logic is in a separate fizzbuzz() function for testability.

```
def main() -> None:
    n = int(input("Write a number: "))
    print(fizzbuzz(n))
def fizzbuzz(n: int) -> str:
    #your code here
    if n % 3 == 0 and n % 5 == 0:
        return "FizzBuzz"
    elif n % 3 == 0:
        return "Fizz"
    elif n % 5 == 0:
        return "Buzz"
    else:
        return str(n)

if __name__ == '__main__':
    main()
```

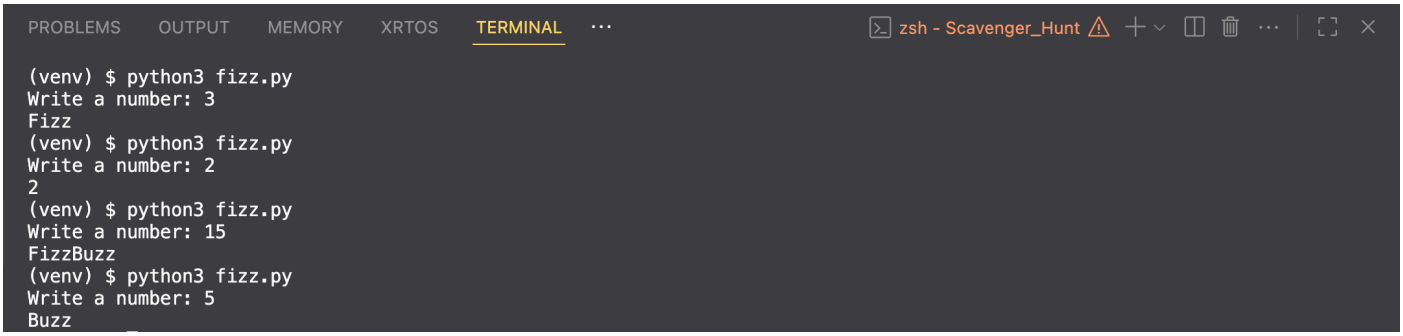


Figure 45 - FizzBuzz | Result

Palindrome Checker

Checks whether a string reads the same forwards and backwards using Python's slice notation s[::-1]. Demonstrates the concise reversed-string comparison technique.

```
def main() -> None:
    s = input(" Write something: ")
    if is_palindrome(s):
        print("Nice")
    else:
        print("Bad")
def is_palindrome(s: str) -> bool:
    # your code here
    s.strip(" ")
    s.lower()
    if s == s[::-1]:
        return True
    elif s[0] == s[-1]:
        return True
    else:
```

```
        return False
if __name__ == "__main__":
    main()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  ···   zsh - Scavenger_Hunt

(venv) $ python3 palindrome.py
 Write something: uau
Nice
(venv) $ python3 palindrome.py
 Write something: hey
Bad
```

Figure 46 - Palindrome Checker | Result

Find The Longest Word

Splits a sentence on whitespace and iterates through the resulting list, keeping track of the longest word seen so far. Returns the last longest word encountered (ties go to the later word).

```
def main() -> None:
    sentence = input("Write: ")
    print(longest_word(sentence))
def longest_word(sentence: str) -> str:
    # your code here
    words = sentence.split()
    bigger = ""

    for word in words:
        if len(word) >= len(bigger):
            bigger = word

    return bigger
if __name__ == "__main__":
    main()
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  ···   zsh - Scavenger_Hunt

(venv) $ python3 longest.py
Write: My name is Francisco
Francisco
(venv) $ python3 longest.py
Write: I really enjoy playing video games
playing
```

Figure 47 - Find The Longest Word | Result

Unitesting A Calculator

Tests a four-function calculator module (add, subtract, multiply, divide) using Python's assert statement inside a try/except block. Includes a division-by-zero guard that returns an error string rather than raising an exception.

```
def add(a, b):
 return a + b
def subtract(a, b):
 return a - b
def multiply(a, b):
 return a * b
def divide(a, b):
 if b == 0:
  return "Error: Division by zero"
 return a / b
```

```
from calculator import *
try:
    # your code here
    assert add(1, 2) == 3
    assert subtract(8, 4) == 4
    assert multiply(2, 3) == 6
    assert divide(6, 3) == 2
    assert divide(2, 0) == "Error: Division by zero"
    print("All tests passed!")
except AssertionError as e:
    print(e)
```

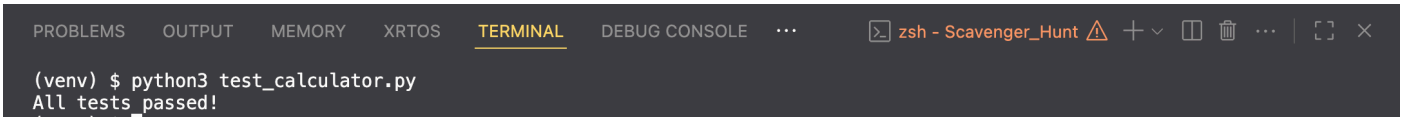


Figure 48 - Unitesting A Calculator | Result

CS50 P-Shirt

Uses the Pillow library (PIL) to overlay the CS50 shirt PNG on a user-supplied photo. ImageOps.fit() resizes and crops the input image to exactly match the shirt dimensions before the overlay is composited using Image.paste() with the shirt as its own mask (preserving transparency). Command-line argument validation ensures correct file count, valid extensions, and matching input/output formats.

```python
import sys
from PIL import Image, ImageOps
# Ensure the correct number of command-line arguments
if len(sys.argv) < 3:
    sys.exit("Too few command-line arguments")
elif len(sys.argv) > 3:
    sys.exit("Too many command-line arguments")
# Validate extensions
elif not sys.argv[1].endswith((".jpg", ".jpeg", ".png")):
    sys.exit("Invalid input")
elif not sys.argv[2].endswith((".jpg", ".jpeg", ".png")):
    sys.exit("Invalid output")
# Ensure extensions match
elif sys.argv[1].split(".")[-1] != sys.argv[2].split(".")[-1]:
    sys.exit("Input and output have different extensions")
try:
    input_image = Image.open(sys.argv[1])
except FileNotFoundError:
    sys.exit("Input does not exist")
# Open the shirt overlay image
shirt = Image.open("shirt.png")
# Resize and crop the input image to match the shirt's dimensions
fitted = ImageOps.fit(input_image, shirt.size)
# Overlay the shirt image on top of the resized image using transparency
fitted.paste(shirt, shirt)
# Save the final image to the output file specified by the user
fitted.save(sys.argv[2])
```

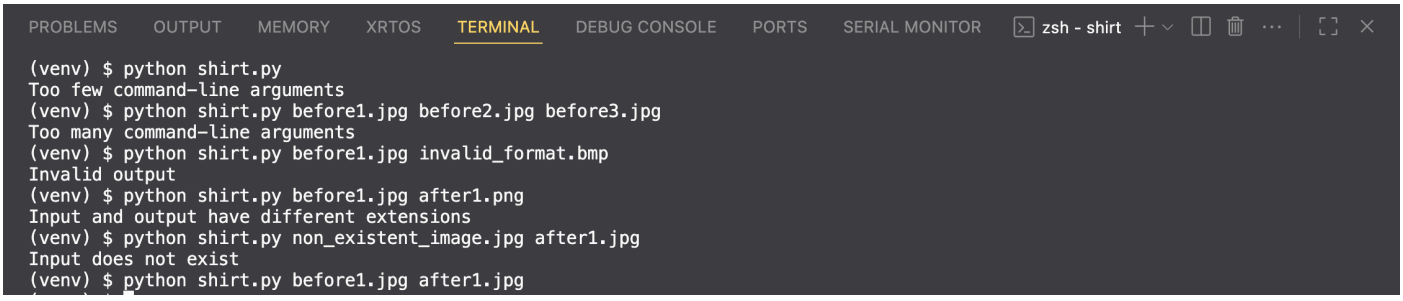


Figure 49 - CS50 P-Shirt | Result



Figure 50 - CS50 P-Shirt | After



Figure 51 - CS50 P-Shirt | Before

me50 / users / franciscpime / cs50 / problems / 2022 / python / shirt

#2 submitted a few seconds ago, Sunday, March 15, 2026 10:18 AM CET
check50 13/13 • 0 comments
tar.gz • zip

#1 submitted 19 hours ago, Saturday, March 14, 2026 3:04 PM CET
check50 13/13 • 0 comments
tar.gz • zip

Week 6

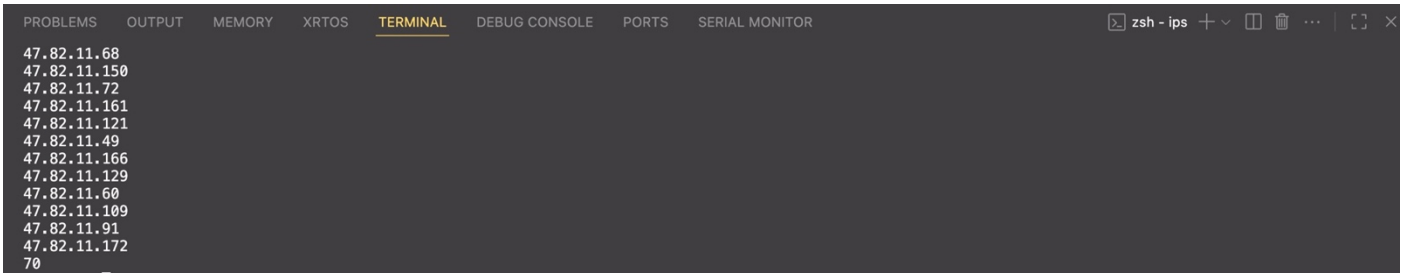
Week 6 focuses on reading and writing files and on applying regular expressions to structured text data. It also introduces AI concepts through a Tic-Tac-Toe implementation driven by the Minimax algorithm.

Manic Monday - Regex & File I/O

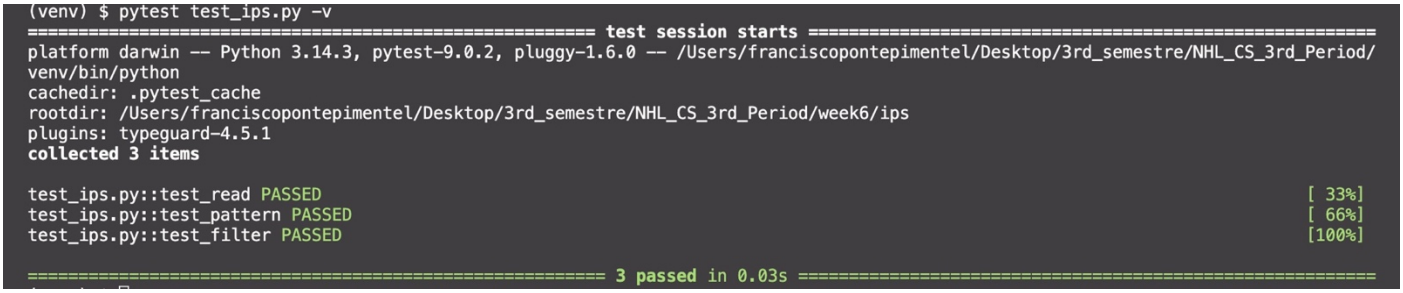
Reads a file of IP addresses (one per line, with duplicates), extracts only those in the range 47.82.11.0 to 47.82.11.255 using a carefully crafted regex, and prints them together with the total count. Three pytest tests verify: that 360 unique IPs are read from the file, that the regex pattern correctly accepts and rejects specific addresses, and that the filter function returns between 68 and 70 valid IPs.

```
import re
def main():
 # do not change below code
 all_ips = read_ips()
 correct_ips = filter_ips(all_ips)
 for p in correct_ips:
  print(p)
 print(len(correct_ips))
def read_ips():
 # read all content of file as string
 with open("ips.txt", "r") as file:
  content = file.read()
 # split by whitespace/newlines
 ips = content.split()
 # put all ips in a list
 # filter out double ips, only unique ips in the list
 unique_ips = list(set(ips))
 # return list
 return unique_ips
def get_pattern():
 # uses regex to determine of IP address is a valid IP in the range:
 # 47.82.11.0 - 47.82.11.255
    # return the pattern, so something like r"^47.....$"
 return r"^47\.82\.11\.(\d|[1-9]\d|1\d\d|2[0-4]\d|25[0-5])$"
def filter_ips(all_ips):
 correct = list()
 # receives a list with a lot of possible IP addresses
 pattern = get_pattern()
 for ip in all_ips:
  if re.match(pattern, ip):
   correct.append(ip)
 # remove duplicates
 correct = list(set(correct))

 # returns a list with valid IP addresses
 return correct
if __name__ == "__main__":
 main()
```




```python
import ips as ips
import re
def test_read():
 l = ips.read_ips()
 assert len(l) == 360
def test_pattern():
 pp = ips.get_pattern()
 p = "47.82.11.68"
 assert re.match(pp, p) is not None
 p = "47.83.11.68"
 assert re.match(pp, p) is None
 p = "47.82.11.280"
 assert re.match(pp, p) is None
 p = "47.82.11.03"
 assert re.match(pp, p) is None
def test_filter():
 s = ips.read_ips()
 sout = ips.filter_ips(s)
 assert len(sout) in (68, 69, 70)
```



Tic-Tac-Toe

A complete Tic-Tac-Toe AI implemented for the CS50 AI course, rendered visually with pygame. The game logic is split across seven functions: initial_state(), player(), actions(), result(), winner(), terminal(), utility(), and the core minimax() function.

The Minimax algorithm works by recursively evaluating all possible game states. The maximising player (X) selects the action with the highest min_value() score; the minimising player (O) selects the action with the lowest max_value() score. Because the utility values are +1 (X wins), -1 (O wins), and 0 (draw), optimal play by both sides always results in a draw when starting from an empty board.

An additional explain_move() function provides natural-language explanations for each AI decision, categorising moves as: immediate win, blocking opponent's win, creating a fork, blocking a fork, or a Minimax-optimal play. The pygame runner handles all user interaction, board rendering, and game-over detection.

```
"""
Tic Tac Toe Player
"""
import copy
import math
# Constants representing the two players and an empty cell
X = "X"
O = "O"
EMPTY = None
def initial_state():
    # Returns starting state of the board.
    return [[EMPTY, EMPTY, EMPTY],
            [EMPTY, EMPTY, EMPTY],
            [EMPTY, EMPTY, EMPTY]]
# Returns player who has the next turn on a board
def player(board):

    count_X = 0
    count_O = 0
    # Count the number of X and O marks on the board
    for row in board:
        for cell in row:
            if cell == X:
                count_X += 1
            elif cell == O:
                count_O += 1
    # Determine whose turn it is# Determine whose turn it is
    if count_X == count_O:
        return X
    else:
        return O
# Returns set of all possible actions (i, j) available on the board.
def actions(board):

    possible_actions = set()
```

```
    # Iterate through the board and collect empty positions
    for i in range(3):
        for j in range(3):
            if board[i][j] == EMPTY:
                possible_actions.add((i, j))
    return possible_actions
# Returns the board that results from making move (i, j) on the board
def result(board, action):

    if action not in actions(board):
        raise Exception
    current_player = player(board)
    # Create a copy of the board to avoid mutating the original
    new_board = copy.deepcopy(board)
    i, j = action
    # Apply the move
    new_board[i][j] = current_player
    return new_board
# Returns the winner of the game, if there is one
def winner(board):

    # Check the rows
    for i in range(3):
        if board[i][0] == board[i][1] == board[i][2] != EMPTY:
            return board[i][0]

    # Check the columns
    for j in range(3):
        if board[0][j] == board[1][j] == board[2][j] != EMPTY:
            return board[0][j]
    # Check diagonals
    if board[0][0] == board[1][1] == board[2][2] != EMPTY:
        return board[0][0]
    if board[0][2] == board[1][1] == board[2][0] != EMPTY:
        return board[0][2]
    return None
# Returns True if game is over, False otherwise
def terminal(board):
    # If someone has won, the game is finished
    if winner(board) != None:
        return True
    # If there are still empty cells, the game continues
    for row in board:
        for cell in row:
            if cell == EMPTY:
                return False

    # No empty cells and no winner, so draw
    return True
# Returns 1 if X has won the game, -1 if O has won, 0 otherwise
def utility(board):

    w = winner(board)
    if w == X:
        return 1
    elif w == O:
        return -1
    else:
        return 0
# Returns the maximum value of the board
```

```
def max_value(board):
    if terminal(board):
        return utility(board)
    v = float("-inf")
    for action in actions(board):
        v = max(v, min_value(result(board, action)))
    return v
# Returns the minimum value of the board
def min_value(board):
    if terminal(board):
        return utility(board)
    v = float("inf")
    for action in actions(board):
        v = min(v, max_value(result(board, action)))
    return v
# Returns the optimal action for the current player on the board
def minimax(board):
    # If the game is already finished, no move can be made
    if terminal(board):
        return None
    current_player = player(board)
    best_action = None
    if current_player == X:
        best_score = float("-inf")
        for action in actions(board):
            score = min_value(result(board, action))
            if score > best_score:
                best_score = score
                best_action = action
    else:
        best_score = float("inf")
        for action in actions(board):
            score = max_value(result(board, action))
            if score < best_score:
                best_score = score
                best_action = action
    return best_action
# Returns the best action and its explanation for the current player on the board
def explain_move(board):
    if terminal(board):
        return None, "Game is already over."
    current = player(board)
    opponent = O if current == X else X
    best_action = minimax(board)
    # Helper: check if a move leads to a win
    def is_winning_move(board, action, p):
        new_board = result(board, action)
        return winner(new_board) == p
    # Immediate win
    if is_winning_move(board, best_action, current):
        return best_action, "This move wins the game immediately."
    # Block opponent win
    for action in actions(board):
        if is_winning_move(board, action, opponent):
            if action == best_action:
                return best_action, "This move blocks the opponent from winning on their n
ext turn."
    # Helper: count how many winning moves a player has after a move
    def count_winning_moves(board, p):
        count = 0
```

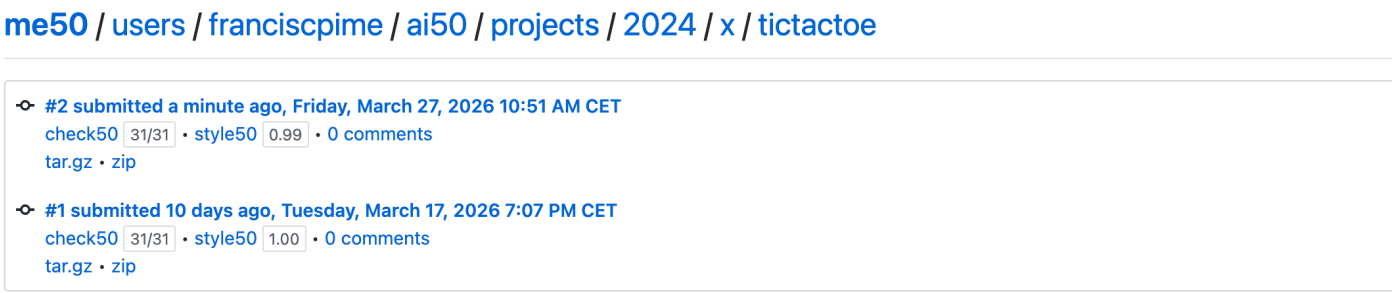
```
        for action in actions(board):
            if is_winning_move(board, action, p):
                count += 1
        return count
    # Create fork
    new_board = result(board, best_action)
    if count_winning_moves(new_board, current) >= 2:
        return best_action, "This move creates a fork, giving multiple ways to win."
    # Block opponent fork
    for action in actions(board):
        new_board = result(board, action)
        if count_winning_moves(new_board, opponent) >= 2:
            if action == best_action:
                return best_action, "This move blocks the opponent from creating a fork."
    # Minimax explanation
    score = None
    if current == X:
        score = max(
            min_value(result(board, action)) for action in actions(board)
        )
    else:
        score = min(
            max_value(result(board, action)) for action in actions(board)
        )
    if score == 1:
        return best_action, "This move leads to a guaranteed win with optimal play."
    elif score == 0:
        return best_action, "This move ensures a draw with optimal play."
    else:
        return best_action, "All moves lead to a loss, but this delays defeat as much as p
ossible."
```



Figure 52 - Tic-Tac-Toe | Win Result



Figure 53 - Tic-Tac-Toe | Tie Result



Current Challenges in AI - Based on User Interviews

After interviewing three people with different backgrounds and levels of tech familiarity, a few recurring themes stood out.

Trust and transparency came up in almost every conversation. People struggle to understand why AI gives a certain answer, which makes it hard to know when to trust it and when to question it. One person mentioned feeling unsure whether to use AI for important decisions because "it sounds confident even when it's wrong." This points to a real gap between how AI communicates and how reliable it actually is.

"Over-reliance against under-use", that was another tension. Some people avoid AI entirely because it feels too complicated or impersonal, while others use it for everything without critically evaluating the output. Neither extreme is healthy, and there doesn't seem to be much guidance on how to find a middle ground.

Personalization without privacy also came through clearly. People want AI tools that feel relevant to them, but they're uncomfortable with how much data that requires. There's a general feeling that you have to "give something up" to get a useful experience.

Finally, accessibility stood out as an underrated challenge. Most AI tools are built around people who are already tech-literate, speak English fluently, and have stable internet. For everyone outside that profile, the tools feel either useless or alienating.

Overall, the interviews revealed that the biggest challenges aren't purely technical. They're about how AI fits into people's real lives, their values, and their levels of trust. Any innovation in this space needs to address the human side just as much as the technology itself.

Week 7

Week 7 delves deeper into artificial intelligence, covering propositional logic (Knights and Knaves) and adversarial game-playing with an enhanced Minesweeper AI that introduces lie detection.

Knights

Solves four Knights and Knaves logic puzzles using a propositional logic engine (logic.py, provided by CS50 AI). Each puzzle defines a set of characters who are either knights (always truthful) or knaves (always lying). Their statements are encoded as logical implications: if a character is a knight, their statement must be true; if a knave, it must be false. The model_check() function then performs model checking, testing all possible truth assignments, to determine which assignments satisfy all constraints.

```
from logic import *
AKnight = Symbol("A is a Knight")
AKnave = Symbol("A is a Knave")
BKnight = Symbol("B is a Knight")
BKnave = Symbol("B is a Knave")
CKnight = Symbol("C is a Knight")
CKnave = Symbol("C is a Knave")
base_rules = And(
    # A is a Knight or a Knave, but not both
    Or(AKnight, AKnave),
    Not(And(AKnight, AKnave)),
    # B is a knight or a Knave, but not both
    Or(BKnight, BKnave),
    Not(And(BKnight, BKnave)),
    # C is a knight or a Knave, but not both
    Or(CKnight, CKnave),
    Not(And(CKnight, CKnave))
)
# Puzzle 0
# A says "I am both a knight and a knave."
knowledge0 = And(
    base_rules,
    # A is a knight implies A is both a knight and a knave
    Implication(AKnight, And(AKnight, AKnave)),
    # A is a knave implies A is not both a knight and a knave
    Implication(AKnave, Not(And(AKnight, AKnave)))
)
# Puzzle 1
# A says "We are both knaves."
# B says nothing.
knowledge1 = And(
    base_rules,
    # A is a knight implies A is both a knight and a knave
    Implication(AKnight, And(AKnave, BKnave)),
    # A is a knave implies A is not both a knight and a knave
    Implication(AKnave, Not(And(AKnave, BKnave)))
)
# Puzzle 2
```

```
# A says "We are the same kind."
# B says "We are of different kinds."
knowledge2 = And(
    base_rules,
    # Declaration A
    Implication(AKnight, Or(And(AKnight, BKnight), And(AKnave, BKnave))),
    Implication(AKnave, Not(Or(And(AKnight, BKnight), And(AKnave, BKnave)))),
    # Declaration B
    Implication(BKnight, Or(And(AKnight, BKnave), And(AKnave, BKnight))),
    Implication(BKnave, Not(Or(And(AKnight, BKnave), And(AKnave, BKnight))))
)
# Puzzle 3
# A says either "I am a knight." or "I am a knave.", but you don't know which.
# B says "A said 'I am a knave'."
# B says "C is a knave."
# C says "A is a knight."
knowledge3 = And(
    base_rules,
    # B says that A said "I am a knave"
    # If B is Knight (tells the truth), then A really said "I am a knave"
    #   >> If A is Knight, A only tells the truth, then AKnave would be true >> contradict
ion
    #   >> If A is Knave, A lies, then "I am a knave" would be false >> contradiction
    # So B can only be Knave
    Implication(BKnight, And(
        Implication(AKnight, AKnave),   # A said "I am a knave" and is knight >> contradic
tion
        Implication(AKnave, Not(AKnave)) # A said "I am a knave" and is knave >> lies >> c
ontradiction
    )),
    Implication(BKnave, Not(And(
        Implication(AKnight, AKnave),
        Implication(AKnave, Not(AKnave))
    ))),
    # B says "C is a knave"
    Implication(BKnight, CKnave),
    Implication(BKnave, Not(CKnave)),
    # C says "A is a knight"
    Implication(CKnight, AKnight),
    Implication(CKnave, Not(AKnight)),
)
def main() -> None:
    symbols = [AKnight, AKnave, BKnight, BKnave, CKnight, CKnave]
    puzzles = [
        ("Puzzle 0", knowledge0),
        ("Puzzle 1", knowledge1),
        ("Puzzle 2", knowledge2),
        ("Puzzle 3", knowledge3)
    ]
    for puzzle, knowledge in puzzles:
        print(puzzle)
        if len(knowledge.conjuncts) == 0:
            print("    Not yet implemented.")
        else:
            for symbol in symbols:
                if model_check(knowledge, symbol):
                    print(f"    {symbol}")
if __name__ == "__main__":
    main()
```

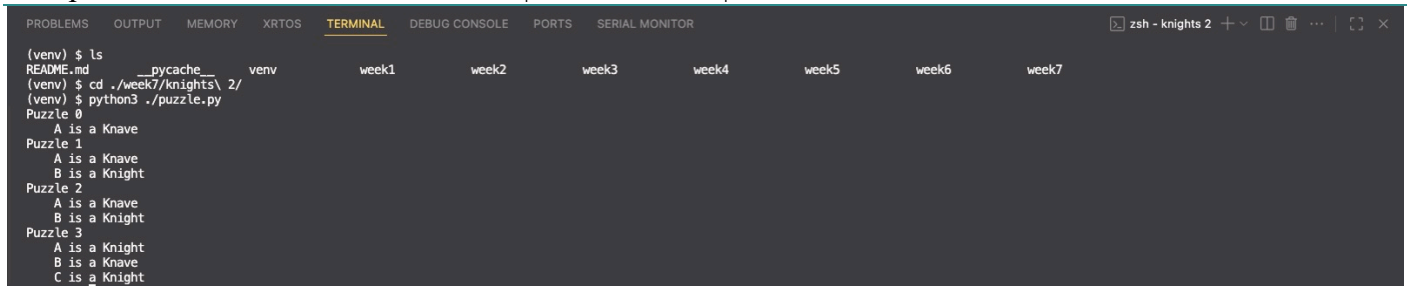


Figure 54 - Knights | Result

The Misleading Minesweeper

An extension of the CS50 AI Minesweeper project. The standard Minesweeper AI uses a knowledge base of logical sentences, each of the form "exactly N of these cells are mines", to deduce safe cells and mines. This variant introduces a twist: one randomly chosen safe cell reports a mine count that is one higher than the true count (a "lying cell").

The MinesweeperAI class is extended with a lie-detection subsystem. Every time a logical contradiction is detected (a sentence with a negative count, a count greater than the number of cells, or a cell appearing in both the safe and mine sets), a suspicion score is incremented for the cell that originated the contradicting sentence. Once any cell accumulates two or more suspicion points, it is promoted to "confirmed liar" and all sentences originating from it are quarantined (ignored in future reasoning). Random moves are additionally biased away from the liar's neighbourhood.

```
import itertools
import random
class Minesweeper():
    """
    Minesweeper game representation — with one lying cell.
    """
    def __init__(self, height=8, width=8, mines=8):
        self.height = height
        self.width = width
        self.mines = set()
        self.board = []
        for i in range(self.height):
            row = []
            for j in range(self.width):
                row.append(False)
            self.board.append(row)
        while len(self.mines) != mines:
            i = random.randrange(height)
            j = random.randrange(width)
            if not self.board[i][j]:
                self.mines.add((i, j))
                self.board[i][j] = True
        self.mines_found = set()
        # Pick one safe cell to be the liar (actual count + 1)
        safe_cells = [
            (i, j)
            for i in range(self.height)
```

```
            for j in range(self.width)
            if not self.board[i][j]
        ]
        self.lying_cell = random.choice(safe_cells)
        print(f"[DEBUG] Lying cell is: {self.lying_cell}")  # Remove for final submission
    def print(self):
        for i in range(self.height):
            print("--" * self.width + "-")
            for j in range(self.width):
                if self.board[i][j]:
                    print("|X", end="")
                else:
                    print("| ", end="")
            print("|")
        print("--" * self.width + "-")
    def is_mine(self, cell):
        i, j = cell
        return self.board[i][j]
    def nearby_mines(self, cell):
        count = 0
        for i in range(cell[0] - 1, cell[0] + 2):
            for j in range(cell[1] - 1, cell[1] + 2):
                if (i, j) == cell:
                    continue

                if 0 <= i < self.height and 0 <= j < self.width:
                    if self.board[i][j]:
                        count += 1
        # If this is the lying cell, return count + 1
        if cell == self.lying_cell:
            return count + 1
        return count
    def won(self):
        return self.mines_found == self.mines
class Sentence():
    """
    Logical statement about a Minesweeper game.
    Tracks which cell originated this sentence for lie detection.
    """
    def __init__(self, cells, count, origin=None):
        self.cells = set(cells)
        self.count = count
        self.origin = origin  # Which revealed cell created this sentence
    def __eq__(self, other):
        return self.cells == other.cells and self.count == other.count
    def __str__(self):
        return f"{self.cells} = {self.count} (from {self.origin})"
    def known_mines(self):
        if len(self.cells) == self.count and self.count != 0:
            return set(self.cells)
        return set()
    def known_safes(self):
        if self.count == 0:
            return set(self.cells)
        return set()
    def mark_mine(self, cell):
        if cell in self.cells:
            self.cells.remove(cell)
            self.count -= 1
    def mark_safe(self, cell):
```

```
        if cell in self.cells:
            self.cells.remove(cell)
class MinesweeperAI():
    """
    Minesweeper AI with lie detection and adaptive reasoning.
    """
    def __init__(self, height=8, width=8):
        self.height = height
        self.width = width
        self.moves_made = set()
        self.mines = set()
        self.safes = set()
        self.knowledge = []
        # Lie detection
        self.suspected_liar = None
        self.suspicion_scores = {}      # cell -> contradiction count
        self.ignored_sentences = set()  # indices of quarantined sentences
    def mark_mine(self, cell):
        self.mines.add(cell)
        for sentence in self.knowledge:
            sentence.mark_mine(cell)
    def mark_safe(self, cell):
        self.safes.add(cell)
        for sentence in self.knowledge:
            sentence.mark_safe(cell)
    def add_knowledge(self, cell, count):
        # 1) Mark move made
        self.moves_made.add(cell)
        # 2) Mark cell as safe
        self.mark_safe(cell)
        # 3) Build new sentence from unknown neighbors
        neighbors = set()
        adjusted_count = count
        for i in range(cell[0] - 1, cell[0] + 2):
            for j in range(cell[1] - 1, cell[1] + 2):
                if (i, j) == cell:
                    continue
                if 0 <= i < self.height and 0 <= j < self.width:
                    if (i, j) in self.mines:
                        adjusted_count -= 1
                    elif (i, j) not in self.safes:
                        neighbors.add((i, j))
        new_sentence = Sentence(neighbors, adjusted_count, origin=cell)
        self.knowledge.append(new_sentence)
        # 4) Update knowledge
        self._update_knowledge()
        # 5) Infer new sentences via subset method
        self._infer_new_sentences()
        # 6) Detect contradictions and identify liar
        self._detect_liar()
    def _update_knowledge(self):
        changed = True
        while changed:
            changed = False
            safes_to_mark = set()
            mines_to_mark = set()
            for i, sentence in enumerate(self.knowledge):
                if i in self.ignored_sentences:
                    continue
                safes_to_mark |= sentence.known_safes()
```

```
                mines_to_mark |= sentence.known_mines()
            for cell in safes_to_mark - self.safes:
                self.mark_safe(cell)
                changed = True
            for cell in mines_to_mark - self.mines:
                self.mark_mine(cell)
                changed = True
            self.knowledge = [s for s in self.knowledge if len(s.cells) > 0]
    def _infer_new_sentences(self):
        new_sentences = []
        active = [
            s for i, s in enumerate(self.knowledge)
            if i not in self.ignored_sentences
        ]
        for s1 in active:
            for s2 in active:
                if s1 == s2:
                    continue
                if s1.cells and s2.cells and s1.cells.issubset(s2.cells):
                    inferred_cells = s2.cells - s1.cells
                    inferred_count = s2.count - s1.count
                    # Negative count = logical contradiction
                    if inferred_count < 0:
                        self._flag_suspicion(s1.origin)
                        self._flag_suspicion(s2.origin)
                        continue
                    inferred = Sentence(inferred_cells, inferred_count,
                                        origin=s2.origin)
                    if inferred not in self.knowledge and inferred not in new_sentences:
                        new_sentences.append(inferred)
        self.knowledge.extend(new_sentences)
        if new_sentences:
            self._update_knowledge()
    def _detect_liar(self):
        """
        Scans knowledge base for impossible sentences:
        - Negative count
        - Count greater than number of cells
        Also checks for cells marked both safe and mine.
        Promotes top suspect to confirmed liar after 2+ contradictions.
        """
        for i, s in enumerate(self.knowledge):
            if i in self.ignored_sentences:
                continue
            if s.count < 0:
                self._flag_suspicion(s.origin)
                self.ignored_sentences.add(i)
            elif s.count > len(s.cells) and len(s.cells) > 0:
                self._flag_suspicion(s.origin)
                self.ignored_sentences.add(i)
        # Check for cells flagged as both mine and safe
        conflict = self.mines & self.safes
        if conflict:
            print(f"[AI] Conflict -> cell in both safe and mine sets: {conflict}")
            for cell in conflict:
                self._flag_suspicion(cell)
        # Promote top suspect if score is high enough
        if self.suspicion_scores and self.suspected_liar is None:
            top = max(self.suspicion_scores, key=self.suspicion_scores.get)
            if self.suspicion_scores[top] >= 2:
```

```
                self.suspected_liar = top
                print(f"[AI] Liar identified: {self.suspected_liar}")
                self._quarantine_liar()
    def _flag_suspicion(self, cell):
        if cell is None:
            return
        self.suspicion_scores[cell] = self.suspicion_scores.get(cell, 0) + 1
        print(f"[AI] Suspicion += {cell} "
              f"(score: {self.suspicion_scores[cell]})")
    def _quarantine_liar(self):
        """Ignores all sentences that came from the suspected liar."""
        for i, s in enumerate(self.knowledge):
            if s.origin == self.suspected_liar:
                self.ignored_sentences.add(i)
        print(f"[AI] Quarantined sentences from {self.suspected_liar}")
    def make_safe_move(self):
        for cell in self.safes:
            if cell not in self.moves_made:
                return cell
        return None
    def make_random_move(self):
        all_cells = [
            (i, j)
            for i in range(self.height)
            for j in range(self.width)
        ]
        # If liar known, prefer cells away from liar's neighbors
        if self.suspected_liar:
            liar_neighbors = self._get_neighbors(self.suspected_liar)
            preferred = [
                c for c in all_cells
                if c not in self.moves_made
                and c not in self.mines
                and c not in liar_neighbors
            ]
            if preferred:
                return random.choice(preferred)
        available = [
            c for c in all_cells
            if c not in self.moves_made and c not in self.mines
        ]
        return random.choice(available) if available else None
    def _get_neighbors(self, cell):
        neighbors = set()
        for i in range(cell[0] - 1, cell[0] + 2):
            for j in range(cell[1] - 1, cell[1] + 2):
                if (i, j) == cell:
                    continue
                if 0 <= i < self.height and 0 <= j < self.width:
                    neighbors.add((i, j))
        return neighbors
```



Figure 55 - The Misleading Minesweeper | Start

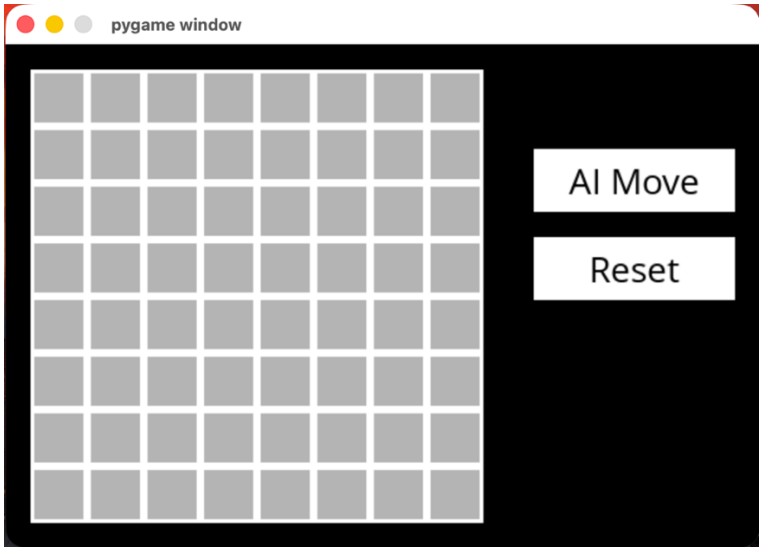


Figure 56 - The Misleading Minesweeper | Beginning

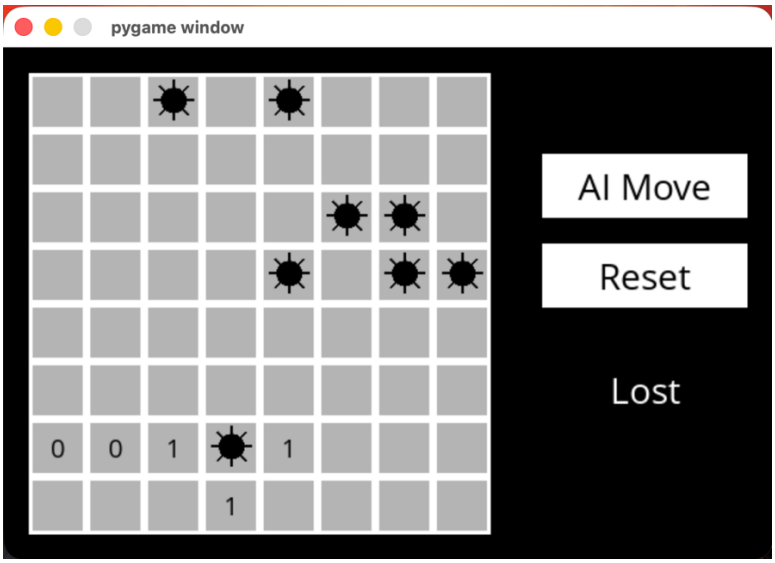


Figure 57 - The Misleading Minesweeper | Lost

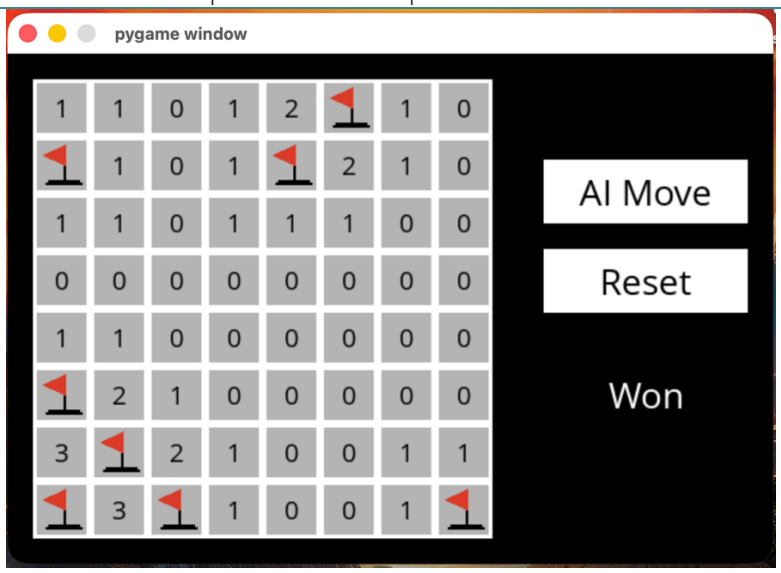


Figure 58 - The Misleading Minesweeper | Win

me50 / users / franciscpime / ai50 / projects / 2024 / x / minesweeper

#1 submitted a few seconds ago, Saturday, March 28, 2026 9:32 AM CET
check50 25/25 • style50 1.00 • 0 comments
tar.gz • zip

Design Thinking Assignment - Math Animator (Solo Innovator Challenge)

How Might We question

How might we help students understand why each step in an equation works, not just what the final answer is?

User profile 1

- 15 years old, 9th grade student
- Struggles with linear equations involving fractions, can follow steps on the board but freezes when trying to replicate them independently
- Re-watches tutorial videos multiple times without being able to pinpoint where his reasoning breaks
- Cannot verify whether his answer is correct on his own, which builds frustration over time

Needs: a step-by-step walkthrough he can pause and rewind, plain-language explanations alongside the math notation, and an automatic back-substitution so he can confirm the result himself.

User profile 2

- 34 years old, secondary school maths teacher with 25+ students per class
- Knows exactly which concepts her students struggle with but doesn't have time to produce custom visual materials for every topic
- Static slides don't convey the flow of a calculation, and the same "but why?" questions come up every single lesson
- Creating animations manually takes hours she doesn't have

Needs: a tool where she types any equation and gets a ready-to-play animation, with explanatory text already on screen so she can focus on the class rather than narrating every micro-step.

Week 8

Math Animator - Solo Project

Math Animator is a Python project that solves mathematical equations step by step and animates each operation using Manim, a mathematical animation library. The idea came from wanting to make equation solving more visual and intuitive, instead of just showing the final answer, the program walks through every arithmetic operation, from rearranging terms and finding common denominators to isolating the variable and verifying the solution. It currently supports first-degree equations with integers, fractions, and decimals. The architecture is built with separation of concerns in mind, the solver knows nothing about animation, and the renderer knows nothing about math, communicating through a simple Step model. Going forward, the plan is to extend it to second-degree equations and other equation types, without changing any of the existing logic.

Feedback from 3 people

Sunny: "Hi Francisco, your solo project idea is helping people to understand how math or physical formula works. In my opinion, your idea is fantastic and good. I remember you showed me your project and how it works. For example, users don’t understand PID formula (u(t)=K_p e(t)+K_i ∫_0^t▒〖e(t)" " dt+〗 K_d (de(t))/dt). When people set values in Kp, Ki, kd, your program can generate a video that how formula works. I will make a big suggestion to you for the next step. So, in the future, your project will be published on the Internet. So, you need to create APIs. Users can use these APIs to generate math or physical teaching videos. By the way, you can use AI to request these APIs which is related to math and physical teaching videos."

Vlad: "Francisco, I really like that you created a visualization for solving equations like 5x - 3 = 17. The step-by-step visual approach is especially strong. it makes the solving process much easier to follow. This is very helpful for anyone who is struggling to understand linear equations or simply wants to check their work.

One suggestion for improvement would be to make the user interface. For example, it could be useful to allow users to move back and forth between the solving steps. You might also consider adding AI-based explanations to further clarify each step.
Overall, the program does a great job, it effectively solves linear equations and presents them in a clear, visual way through a video."

Mae: "Very nicely done project, I like the animations and the UI is very clean, great job!"

Embedded Systems

This chapter provides a reference overview of the hardware components and software libraries used across all Embedded Systems assignments. Understanding these building blocks is essential for interpreting the circuit diagrams and source code in the following sections.

Hardware Platform

Raspberry Pi Pico 2W

The Raspberry Pi Pico 2W is a microcontroller development board based on the RP2350 chip. It features dual Arm Cortex-M33 cores running at up to 150 MHz, 520 KB of SRAM, and 2 MB of on-board flash memory. The "W" suffix indicates integrated 2.4 GHz Wi-Fi (via the CYW43439 chip), enabling network connectivity without additional hardware. All assignments were programmed using either MicroPython or CircuitPython via USB.

Key GPIO capabilities used in this portfolio include: digital output (LED control), analog input via three ADC channels (ADC0-ADC2 on GP26-GP28), I2C and SPI peripherals, USB HID emulation, and Wi-Fi station mode for web serving.

Pin Reference (assignments)

GP1-GP11: 7-segment display segments and digit enables | GP16-GP18: DS1302 RTC (CLK, DAT, RST) | GP26 (ADC0): LDR, LM35, joystick X-axis | GP27 (ADC1): joystick Y-axis | GP0/GP1: I2C SDA/SCL for LCD | GP1-GP3: RGB LED channels | GP22: joystick button

Software Environments

MicroPython

MicroPython is a lean Python 3 implementation optimised for microcontrollers. It provides access to hardware peripherals through the machine module (Pin, ADC, I2C, SPI), a utime module for timing, and a network module for Wi-Fi. All Weeks 1 to 5 and 6/7 embedded projects use MicroPython.

```
from machine import Pin, ADC, I2C
import utime
# Digital output
led = Pin(1, Pin.OUT)
led.value(1)          # HIGH
# Analog input (ADC0 = GP26)
sensor = ADC(26)
raw = sensor.read_u16()  # 0 - 65535 proportional to 0 - 3.3V
# I2C bus
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=100000)
```

CircuitPython

CircuitPython (by Adafruit) is used for the HID assignments in Week 4 because the adafruit_hid library is not available for MicroPython. The key difference in CircuitPython is the use of the board module for named pin references and the digitalio / analogio modules for I/O, replacing MicroPython's machine module.

```
import board
import analogio, digitalio
import usb_hid
from adafruit_hid.keyboard import Keyboard
from adafruit_hid.keycode import Keycode
ldr = analogio.AnalogIn(board.GP26)   # Analog input
kbd = Keyboard(usb_hid.devices)        # Virtual keyboard
kbd.send(Keycode.SPACE)                # Send a keypress
```

Eletronic Components

LED (Light Emitting Diode)

A standard 5mm LED (typically red or green, forward voltage ~2V) connected in series with a current-limiting resistor. The resistor value is calculated as: R = (Vcc − Vf) / I = (3.3V − 2.0V) / 0.01A = 130Ω. A 220Ω resistor is typically used to provide a safety margin. The LED anode connects to the GPIO pin; the cathode connects to GND.

```
# MicroPython - toggle LED state
led = Pin(1, Pin.OUT)
led_state = False
led_state = not led_state
led.value(led_state)
```

RGB LED (Common Cathode)

A common-cathode RGB LED packages three LEDs (red, green, blue) in a single component sharing a common ground pin. Each colour channel is driven from a separate GPIO pin through its own current-limiting resistor (~100Ω per channel). Colours are mixed by combining channels: red+blue = magenta, green+blue = cyan, red+green = yellow.

```
red   = Pin(1, Pin.OUT, value=0)
green = Pin(2, Pin.OUT, value=0)
blue  = Pin(3, Pin.OUT, value=0)
def set_color(r: int, g: int, b: int) -> None:
    red.value(r); green.value(g); blue.value(b)
set_color(1, 0, 1)  # Magenta
```

4-Digit 7-Segment Display

A 7-segment display uses seven LED segments (a-g) arranged to form digits, plus an optional decimal point (dp). The 4-digit variant adds four digit-enable pins, one per digit. Because driving all four digits simultaneously would require 11 GPIO pins at once and could cause segment conflicts, time multiplexing is used: only one digit is enabled at a time, cycling through all four at ~333 Hz (one digit every 3 ms), which appears continuous to the human eye.

```
# Segment patterns - (a,b,c,d,e,f,g)
LED_Bits = {
    "0": (1,1,1,1,1,1,0),
    "1": (0,1,1,0,0,0,0),
    "8": (1,1,1,1,1,1,1),
}
# Multiplexing loop - one digit at a time
for i in range(4):
    disable_digits()
    set_segments(str(D[i]), dot=(i==1))  # decimal after 2nd digit
    digits[i].value(1)
    utime.sleep_us(3000)
```

LM35 Temperature Sensor

The LM35 is a linear temperature sensor that outputs an analog voltage proportional to temperature: 10 mV per °C. At 25°C it outputs 250 mV. The sensor is powered from the Pico's 3.3V rail and its output pin connects to an ADC input. To reduce ADC noise, ten readings are averaged before conversion.

```
total = 0
for _ in range(10):
    total += temp_sensor.read_u16()
    utime.sleep_ms(2)
adc_avg = total / 10
voltage = adc_avg * 3.3 / 65535       # Convert to volts
temperature = voltage / 0.01           # 10 mV per °C >> divide by 0.01
```

DS1302 Real-Time Clock (RTC)

The DS1302 is a low-power RTC chip that maintains time and date (seconds, minutes, hours, day, month, year) using a coin-cell battery backup. It communicates over a 3-wire serial interface (CLK, DAT, RST) using a custom bit-banged protocol, handled by the ds1302 MicroPython library. The date_time() method reads or writes a list of seven integer values: [year, month, day, weekday, hour, minute, second].

```
import ds1302
rtc = ds1302.DS1302(Pin(16), Pin(17), Pin(18))  # CLK, DAT, RST
# Set time
rtc.date_time([2026, 3, 6, 4, 12, 37, 51])
# Read time
year, month, day, weekday, hour, minute, second = rtc.date_time()
```

16×2 I2C LCD Display

A 16-character by 2-line LCD display driven over I2C via a PCF8574 I/O expander backpack, reducing the connection to just two data lines (SDA, SCL). The I2cLcd MicroPython library abstracts the HD44780 LCD controller commands. Custom characters (up to 8) can be defined as 5×8 pixel bitmaps using the custom_char() method.

```
from machine import I2C, Pin
from i2c_lcd import I2cLcd
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=100000)
lcd = I2cLcd(i2c, 0x27, 2, 16)  # address, rows, cols
# Custom clock character (5×8 bitmap)
clock_char = [0b01110, 0b10101, 0b10101, 0b10001, 0b10001, 0b01110, 0,0]
lcd.custom_char(0, clock_char)
lcd.move_to(0, 0)
lcd.putchar(chr(0))   # Display custom clock icon
lcd.putstr("12:37:51")
```

LDR (Light Dependent Resistor)

An LDR is a passive component whose resistance decreases as light intensity increases. In the Dino Cheater project, it forms a voltage divider with a fixed resistor: as the screen (the dinosaur game) darkens (obstacle approaching), the voltage at the ADC pin drops below a threshold, triggering a keypress. The THRESHOLD constant controls sensitivity and can be tuned by monitoring printed ADC values.

```
THRESHOLD = 54500   # Tune by observing printed ADC values
value = ldr.value          # Read current light level
is_dark = value < THRESHOLD
if is_dark and not was_dark:   # Rising edge only
    kbd.send(Keycode.SPACE)    # Jump
was_dark = is_dark
```

Analog Joystick

A thumb joystick provides two analog outputs (X and Y axes, each 0 - 65535) and one digital output (button, active LOW with internal pull-up). In the Tetris controller project, the X axis maps to LEFT/RIGHT arrow keys, the Y axis to DOWN, and the button to UP (rotate). A DEADZONE of ±5000 around the centre (~32000) prevents jitter from registering as movement, and a minimum time delta of 150 ms limits key repeat speed.

```
DEADZONE = 5000
CENTER   = 32000
x = x_axis.value
if x < CENTER - DEADZONE:
    if now - last_move_time > 0.15:
        keyboard.send(Keycode.LEFT_ARROW)
        last_move_time = now
```

Voltage Divider

A voltage divider is used in the 7-Segment Voltmeter to measure voltages up to 10V while staying within the ADC's 3.3V input limit. Two resistors in series (R1 = 10kΩ, R2 = 1kΩ) divide the input voltage: Vout = Vin * R2 / (R1 + R2). The ADC reads Vout and the firmware reconstructs Vin: Vreal = Vadc * (R1 + R2) / R2.

```
r1 = 10000   # 10 kΩ
r2 = 1000    # 1 kΩ
adc_value   = adc.read_u16()
adc_voltage = adc_value * 3.4 / 65535       # Pico ADC reference ~3.4V
real_voltage = adc_voltage * (r1 + r2) / r2  # Scale back up
```

RFID Module (RC522)

The RC522 is an NFC/RFID reader that communicates over SPI. It can read the unique UID from passive MIFARE cards or key fobs.

```
# Conceptual SPI setup for RC522
from machine import SPI, Pin
import mfrc522
spi = SPI(0, baudrate=2_000_000)
rfid = mfrc522.MFRC522(spi, cs=Pin(5), rst=Pin(6))
stat, tag_type = rfid.request(rfid.REQIDL)
if stat == rfid.OK:
    stat, uid = rfid.SelectTagSN()
    print("Card UID:", uid)
```

Web Server (Socket-Based)

The Controlling Stuff project (Week 5) implements a minimal HTTP server directly in MicroPython using the socket module, without any web framework. The Pico W connects to Wi-Fi in station mode, binds a TCP socket to port 80, and enters an accept/respond loop. Each request's first line is parsed for the route; the server responds with a full HTTP/1.1 response including headers and an HTML body. A non-blocking timeout on accept() allows the main loop to simultaneously poll a physical button.

```
# Connect to Wi-Fi
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect("SSID", "password")
while not wifi.isconnected(): utime.sleep(1)
# Bind socket
server = socket.socket()
server.bind(socket.getaddrinfo("0.0.0.0", 80)[0][-1])
server.listen(1)
# Non-blocking accept to allow button polling
server.settimeout(0.05)
try:
    conn, addr = server.accept()
    request = conn.recv(1024).decode()
    # Parse route and send HTML response
except OSError:
    pass  # Timeout - go back to button check
```

Embedded Systems - Assignements

The following sections document each embedded systems assignment in weekly order. For each project the full source code is provided together with a description of the circuit and the observable behaviour. Circuit diagrams and real-world photographs are included inline.

Week 1

Blink

```
# ------------------------------------------------------
# LED BLINK USING SOFTWARE TIMER
# ------------------------------------------------------
# This program blinks an LED connected to a Raspberry Pi Pico
# using a simple software-based timer.
#
# The loop simulates elapsed time by accumulating small time
# intervals. When the counter reaches 1 second, the LED state
# toggles (ON/OFF), creating a blinking effect.
#
# Author: Francisco Pimentel
# Date: February 2026
# ------------------------------------------------------
from machine import Pin
import utime
LED = Pin(1, Pin.OUT)
timer = 0                                       # Time counter
led_state = 0                                   # 0 = OFF, 1 = ON
while True:

    timer += 0.05                               # Accumulate elapsed time
    if timer >= 1:                              # If 1 second has passed
        led_state = not led_state               # Toggle LED state
        LED.value(led_state)                    # Apply new LED state
        print("Blink")
        timer = 0                               # Reset time counter
```

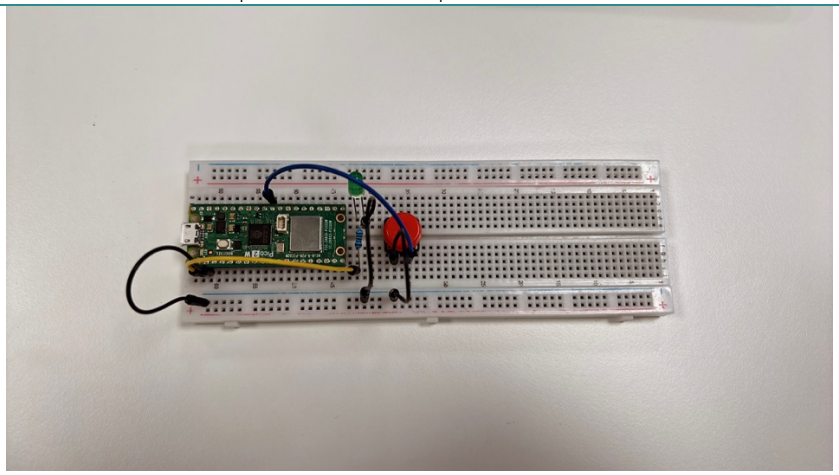


Figure 59 - Blink | Real View

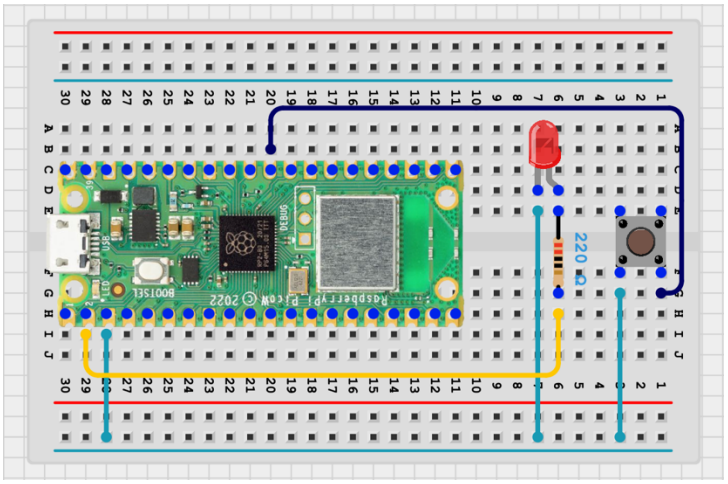


Figure 60 - Blink | Circuit View

Mario Pico

```
# ------------------------------------------------------------
# DOUBLE PYRAMID GENERATOR
# ------------------------------------------------------------
# This program asks the user for a height value between 1 and 8
# and then prints two side-by-side pyramids using "#" characters.
#
# The program ensures valid input using a loop. Once a correct
# value is provided, it builds each row by calculating the number
# of spaces and hashes needed to properly align the pyramids.
#
# The left pyramid is right-aligned, and the right pyramid is
# left-aligned, separated by two spaces.
#
# Author: Francisco Pimentel
# Date: February, 2026
# ------------------------------------------------------------
while True:
    n = int(input("Height: "))                    # Asks to input a number
    if 1 <= n <= 8:                               # Should be between [1, 8]
        break                                     # Get out of the loop
    else:
        print("Number out of range. Try again.")  # Print to try again
# Draw 2 pyramids
for i in range(n):                                # E.g: n = 5; i from 0 to 4
    num_spaces = n - i - 1                        # 5 - 0 - 1 = 4 spaces
    num_hashes = i + 1                            # 0 + 1 = 1 hashe
    # 1st Pyramid
    for space in range(num_spaces):               # Print spaces until i = 3 (0 to 3)
        print(" ", end="")                        # Print space
    for hash in range(num_hashes):                # i = 4; num_hashes = 5
        print("#", end="")                        # Print hash
    print("  ", end="")                           # Print spaces between pyramids
    # 2nd Pyramid
    for hash in range(num_hashes):                # i = 4; num_hashes = 5
        print("#", end="")                        # Print hash
    print()                                       # New line
```

```
Height: 0
Number out of range. Try again.
Height: 9
Number out of range. Try again.
Height: 6
     #  #
    ##  ##
   ###  ###
  ####  ####
 #####  #####
######  ######
```

Figure 61 - Mario More | Pico Version | Result

Week 2

7-Segment Voltmeter (0...10V)

```
# ------------------------------------------------------------
# 4-DIGIT 7-SEGMENT VOLTMETER
# ------------------------------------------------------------
# This program reads an analog voltage from pin GP26 (ADC0)
# and displays the value in the format nn.nn on a 4-digit
# 7-segment display.
#
# The display uses time multiplexing, meaning only one digit
# is turned ON at a time, but all digits appear lit due to
# rapid switching.
#
# The measured voltage is scaled using a voltage divider and
# converted into individual digits for display. A decimal point
# is placed after the second digit.
#
# The system continuously updates the display while periodically
# refreshing the voltage reading.
#
# Author: Francisco Pimentel
# Date: March, 2026
# ------------------------------------------------------------
from machine import Pin, ADC
import utime
# Segments: a,b,c,d,e,f,g,dp
# Digits: 4 individual display enable pins
LED_Segments = [0,1,3,2,4,5,6,11]
LED_Digits   = [7,8,9,10]
# Holds the 4 digits currently shown on display
D = [0,0,0,0]
# Segment patterns for numbers
# 1 = ON ; 0 = OFF
# Order: a,b,c,d,e,f,g
LED_Bits = {
' ':(0,0,0,0,0,0,0),
'0':(1,1,1,1,1,1,0),
'1':(0,1,1,0,0,0,0),
'2':(1,1,0,1,1,0,1),
'3':(1,1,1,1,0,0,1),
'4':(0,1,1,0,0,1,1),
'5':(1,0,1,1,0,1,1),
'6':(1,0,1,1,1,1,1),
'7':(1,1,1,0,0,0,0),
'8':(1,1,1,1,1,1,1),
'9':(1,1,1,1,0,1,1)
}
# Configure GPIO pins as outputs
segments = [Pin(p, Pin.OUT) for p in LED_Segments]
digits   = [Pin(p, Pin.OUT) for p in LED_Digits]
# Analog input (ADC0 = GP26)
adc = ADC(26)
```

```
r1 = 10000
r2 = 1000
def main():
    refresh = 0
    try:
        while True:
            # Only one digit on at a time
            for i in range(4):
                disable_digits()
                # Put decimal point after second digit -> nn.nn
                set_segments(str(D[i]), i == 1)
                digits[i].value(1)
                utime.sleep_us(3000)
            # Every 100 ms
            refresh += 1
            if refresh >= 140:
                blank_display()
                utime.sleep_ms(5)
                voltage = read_voltage()
                number_to_digit(voltage)
                refresh = 0
    except KeyboardInterrupt:
        disable_digits()
        for s in segments:
            s.value(0)
# Turn OFF all digits
def disable_digits():
    for d in digits:
        d.value(0)
# Set the segments to display a character
# dot=True enables decimal point
def set_segments(char, dot):
    pattern = LED_Bits[char]

    # Set segments a-g
    for i in range(7):
        segments[i].value(pattern[i])
    # Set decimal point
    segments[7].value(1 if dot else 0)
# Convert float voltage >> 4 digits nn.nn
# Example: 3.27 V >> [0,3,2,7]
def number_to_digit(value):
    n = int(value * 100)                    # keep 2 decimal places
    # Clamp to display range
    if n > 9999:
        n = 9999
    if n < 0:
        n = 0
    # Split into digits
    D[0] = n // 1000
    n %= 1000
    D[1] = n // 100
    n %= 100
    D[2] = n // 10
    n %= 10
    D[3] = n
# Read ADC voltage
# ADC range: 0 - 65535 corresponds to 0 - 3.3V
def read_voltage():
    adc_value = adc.read_u16()
```

```
    adc_voltage = adc_value * 3.4 / 65535
    real_voltage = adc_voltage * (r1 + r2) / r2
    print("RAW ADC :", adc_value)
    print("ADC V   : {:.3f} V".format(adc_voltage))
    print("REAL V  : {:.3f} V".format(real_voltage))
    print("--------------------------")
    return real_voltage
def blank_display():
    disable_digits()
    for s in segments:
        s.value(0)
main()
```

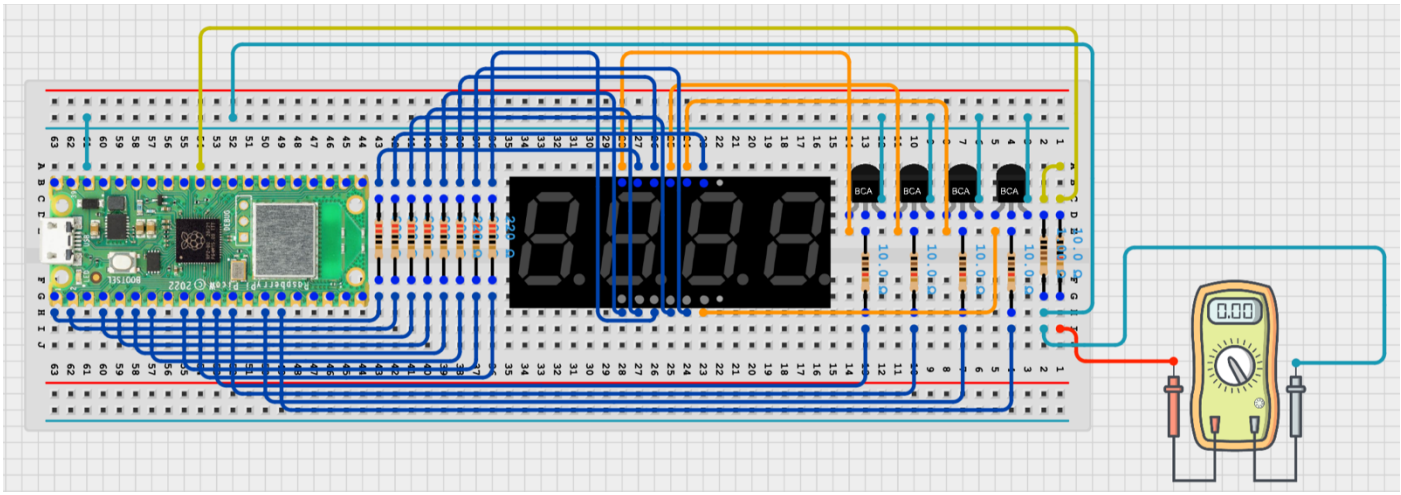


Figure 62 - 7-Segment Voltmeter | Circuit View

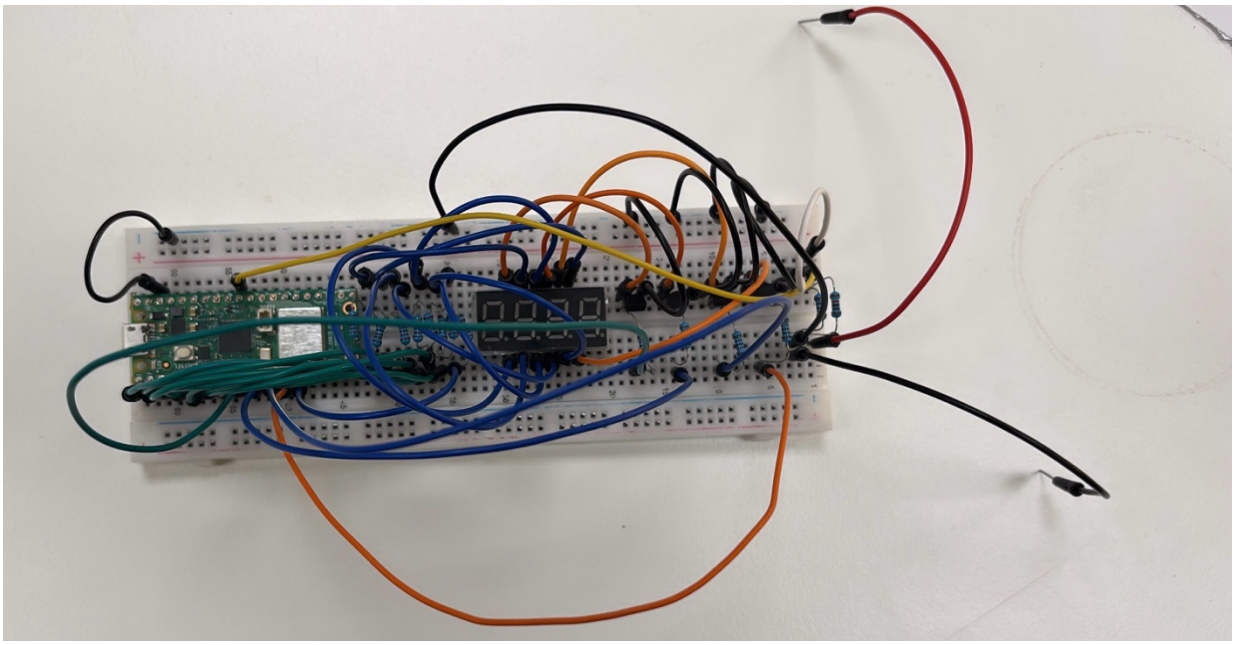


Figure 63 - 7-Segment Voltmeter | Real View

Week 3

RTC and Temperature on LCD-display

```
# ------------------------------------------------------------
# REAL-TIME CLOCK AND TEMPERATURE MONITOR
# ------------------------------------------------------------
# This program reads the current date and time from a DS1302
# Real Time Clock (RTC) module and the ambient temperature
# from an LM35 analog temperature sensor.
#
# The data is displayed on a 16x2 I2C LCD screen and updated
# every second. A custom clock icon is used to visually enhance
# the interface.
#
# The temperature is calculated by averaging multiple ADC
# readings to reduce noise, then converting the voltage into
# degrees Celsius based on the LM35 characteristics (10 mV/ºC).
#
# The user can optionally set the RTC date and time at startup.
#
# Author: Francisco Pimentel
# Date: March, 2026
# ------------------------------------------------------------
from machine import I2C, Pin, ADC
from i2c_lcd import I2cLcd
import utime
import ds1302
LCD_ADDR = 0x27
LCD_ROWS = 2
LCD_COLS = 16
# Standard I2C speed for stability with LCD modules
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=100000)
lcd = I2cLcd(i2c, LCD_ADDR, LCD_ROWS, LCD_COLS)
rtc = ds1302.DS1302(Pin(16), Pin(17), Pin(18))
# LM35 output connected to GP26 (ADC0)
temp_sensor = ADC(26)
# Custom LCD character
clock_char = [
    0b01110,
    0b10101,
    0b10101,
    0b10101,
    0b10101,
    0b10001,
    0b10001,
    0b01110,
]
lcd.custom_char(0, clock_char)
def main():
    utime.sleep_ms(200)
    set_clock()
    lcd.clear()
    lcd.backlight_on()
    last_time = utime.time()
```

```
    while True:
        now = utime.time()
        if now - last_time >= 1:
            last_time = now
            display_clock()
def format_time(hour, minute, second):
    # Return formatted time string >> hours:minutes:seconds
    return "{:02}:{:02}:{:02}".format(hour, minute, second)
def format_date(day, month, year):
    # Return formatted date string >> day/month/year
    return "{:02}/{:02}/{}".format(day, month, year)
def read_temperature():

    # Read LM35 temperature, then
    # uses averaging to reduce ADC noise.
    # LM35 output: 10 mV per 1°C

    total = 0
    for i in range(10):
        value = temp_sensor.read_u16()
        total += value
        utime.sleep_ms(2)
    adc_value = total / 10
    # Convert ADC value to voltage
    voltage = adc_value * 3.3 / 65535
    # Convert voltage to temperature
    temperature = voltage / 0.01
    print("Average ADC:", adc_value)
    print(f"Voltage: {voltage:.2f} V       ({voltage*1000:.0f} mV)")
    print(f"Temperature: {temperature:.1f} C")
    print("-" *30)
    return temperature
def set_clock():
    answer = input("Say 'ok' to set date and time: ").upper()
    if answer == "OK":
        print("Example: 2007, 2, 7, 4, 14, 30, 0")
        year = int(input("Year: "))
        month = int(input("Month: "))
        day = int(input("Day: "))
        weekday = int(input("Week num: "))
        hour = int(input("Hour: "))
        minute = int(input("Minute: "))
        second = int(input("Second: "))
        date_time = [
            year, month, day, weekday,
            hour, minute, second
        ]
        rtc.date_time(date_time)
        print("Clock set successfully.")
    else:
        print("Using current RTC time.")
def display_clock():
    # RTC reading
    year, month, day, weekday, hour, minute, second = rtc.date_time()
    time_str = format_time(hour, minute, second)
    date_str = format_date(day, month, year)
    temperature = read_temperature()
    temp_str = "{:.1f}C".format(temperature)
    print("Hour:", time_str)
    print("Date:", date_str)
```

```
    print("=" *30)
    # First row: Clock icon, time and temperature
    lcd.move_to(0, 0)
    lcd.putchar(chr(0))                 # custom clock icon
    lcd.putstr(":")
    lcd.move_to(2, 0)
    lcd.putstr(time_str)
    lcd.move_to(11, 0)
    lcd.putstr("{:<5}".format(temp_str))
    # Second row: the rest
    lcd.move_to(0, 1)
    lcd.putstr("Date:")
    lcd.move_to(6, 1)
    lcd.putstr(date_str)
main()
```

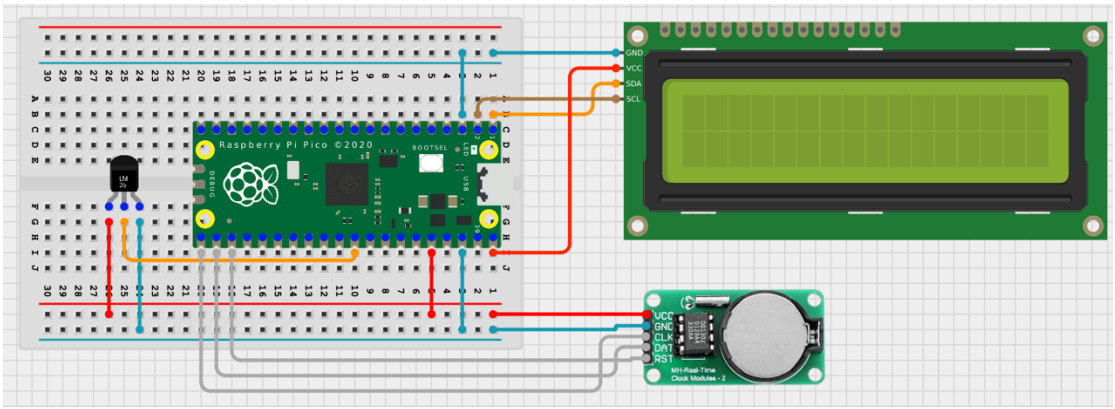


Figure 64 - RTC and Temperature | Circuit View

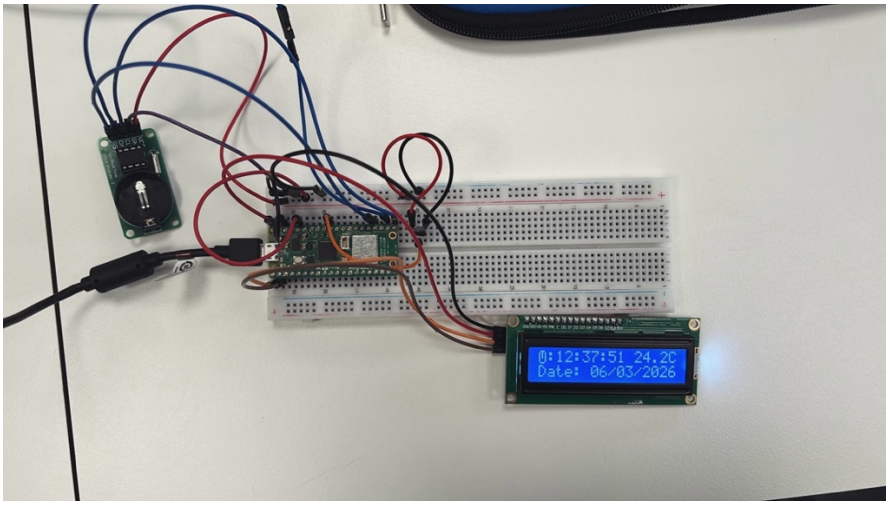


Figure 65 - RTC and Temperature | Real View

Week 4

Dino Cheater (HID)

```
# ------------------------------------------------------------
# LIGHT-TRIGGERED KEYBOARD INPUT
# ------------------------------------------------------------
# This program uses an LDR (Light Dependent Resistor) connected
# to an analog pin (GP26) to detect changes in ambient light.
#
# When the light level drops below a defined threshold (darkness),
# the system sends a keyboard input (SPACE key) to the computer
# using USB HID emulation.
#
# The trigger only activates when the environment transitions
# from light to dark, preventing repeated key presses while
# remaining in darkness.
#
# The sensitivity can be adjusted by changing the threshold value.
#
# Author: Francisco Pimentel
# Date: March, 2026
# ------------------------------------------------------------
import time
import board
import analogio
import usb_hid
from adafruit_hid.keyboard import Keyboard
from adafruit_hid.keycode import Keycode
# Creates an object to read the LDR sensor connected to the GP26 pin, so
# ldr.value will return me values between 0 and 65535
ldr = analogio.AnalogIn(board.GP26)
# Initializes the "virtual keyboard"
kbd = Keyboard(usb_hid.devices)
# Defines the threshold for distinguishing light from darkness
# High value >> more sensitive to light
# Low value >> more sensitive to darkness
THRESHOLD = 54500
# Check if it is dark
# If value < threshold >> dark - True
# Otherwise >> light - False
was_dark = False
while True:
    value = ldr.value

    # Check if it is dark based on the threshold
    is_dark = value < THRESHOLD
    print("Value:", value, "| Dark:", is_dark)
    # Trigger only when it just becomes dark
    if is_dark and not was_dark:
        print("Jump!")
        kbd.send(Keycode.SPACE)
    was_dark = is_dark
    time.sleep(0.01)
```

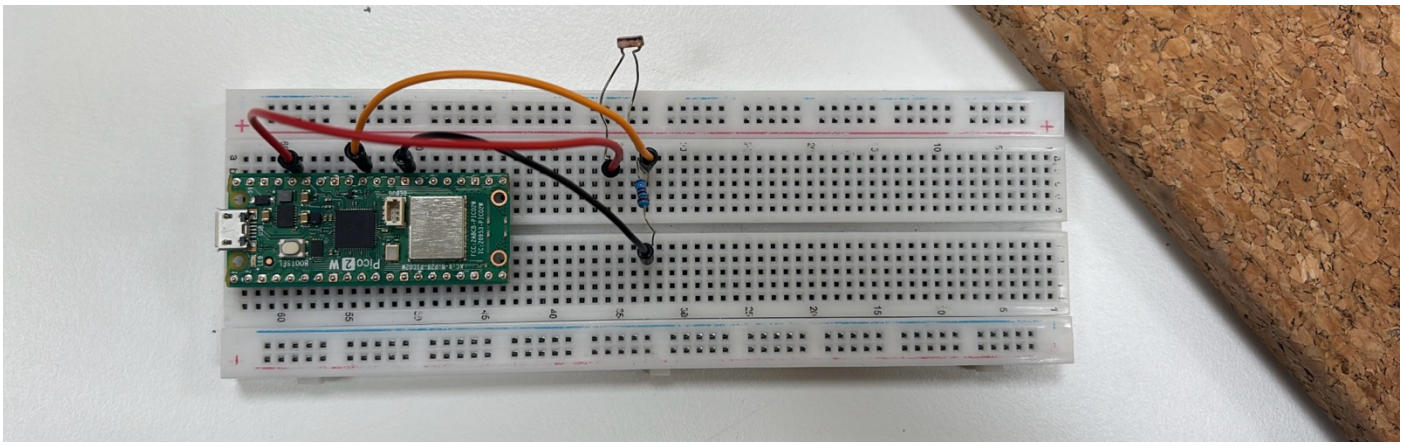


Figure 66 - Dino Cheater (HID) | Real View

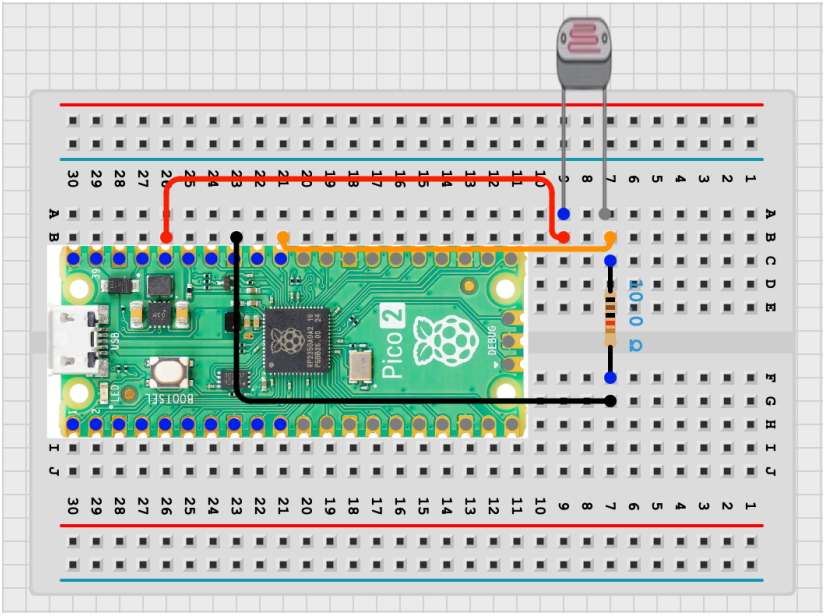


Figure 67 - Dino Cheater (HID) | Circuit View

Analog Joystick (HID)

```
import time
import board
import analogio
import digitalio
import usb_hid
from adafruit_hid.keyboard import Keyboard
from adafruit_hid.keycode import Keycode
# Axes (analog inputs from joystick)
x_axis = analogio.AnalogIn(board.GP26)      # X-axis (left/right)
y_axis = analogio.AnalogIn(board.GP27)      # Y-axis (up/down)
# Button setup
button = digitalio.DigitalInOut(board.GP22)
button.direction = digitalio.Direction.INPUT     # Set as input
button.pull = digitalio.Pull.UP        # Enable internal pull-up resistor
keyboard = Keyboard(usb_hid.devices)
DEADZONE = 5000            # Range around center where no movement is detected
CENTER = 32000            # Approximate center value of joystick
last_move_time = 0           # Stores last time a horizontal move was made
while True:
    x = x_axis.value           # Read X-axis value
    y = y_axis.value           # Read Y-axis value
    print("X:", x, "Y:", y)
    now = time.monotonic()         # Get current time (in seconds)
    # Left movement
    if x < CENTER - DEADZONE:         # If joystick pushed left
        if now - last_move_time > 0.15:      # Limit repeat speed
            keyboard.send(Keycode.LEFT_ARROW)      # Send left arrow key
            print("LEFT")
            last_move_time = now        # Update last move time
    # Right movement
    elif x > CENTER + DEADZONE:        # If joystick pushed right
        if now - last_move_time > 0.15:      # Limit repeat speed
            keyboard.send(Keycode.RIGHT_ARROW)      # Send right arrow key
            print("RIGHT")
            last_move_time = now        # Update last move time
    # Down movement
    if y < CENTER - DEADZONE:         # If joystick pushed down
        keyboard.send(Keycode.DOWN_ARROW)       # Send down arrow key
        print("DOWN")
        time.sleep(0.05)          # Small delay to control speed
    # Button press (rotate piece)
    if not button.value:          # Button is pressed (active LOW)
        keyboard.send(Keycode.UP_ARROW)       # Send up arrow key (rotate)
        print("ROTATE")
        time.sleep(0.2)          # Debounce delay
    time.sleep(0.01)           # Small loop delay to reduce CPU usage
```

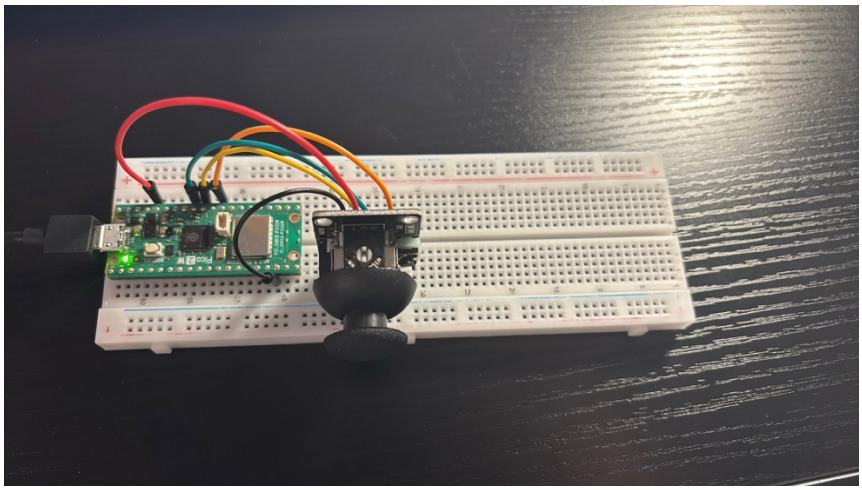


Figure 68 - Analog Joystick (HID) | Real View

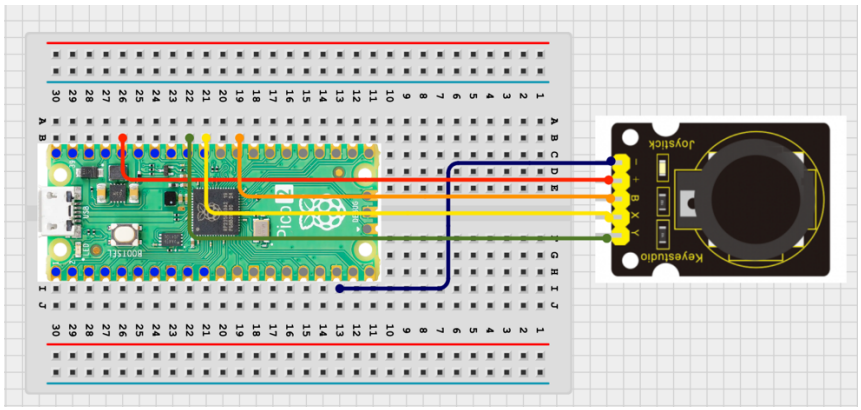


Figure 69 - Analog Joystick (HID) | Circuit View

Week 5

WiFi Scanner

```python
import utime
import network
# Create a WLAN interface in station mode
wifi_interface = network.WLAN()
# Activate the Wi-Fi interface so it can scan for networks
wifi_interface.active(True)
# Scan for nearby Wi-Fi networks
# The scan returns a list of tuples with the following structure:
# (ssid, bssid, channel, rssi, security, hidden)
networks = wifi_interface.scan()
# Sort the networks by signal strength (RSSI)
sorted_networks = sorted(networks, key=lambda network: network[3], reverse=True)
security_types = {
    0: "Open",
    1: "WEP",
    2: "WPA-PSK",
    3: "WPA2-PSK",
    4: "WPA/WPA2",
    5: "WPA2-Enterprise",
    6: "WPA3"
}
i = 1
for network in sorted_networks:
    ssid = network[0].decode()

    if not ssid:
        ssid = "Hidden Network"
    bssid = network[1]
    mac = ":".join("{:02x}".format(byte) for byte in bssid)
    channel = network[2]
    signal = network[3]
    security = network[4]
    print(f"{i:2} | Network: {ssid:20} | Signal: {signal:4} dBm")
    i += 1
```

```
PROBLEMS  OUTPUT  MEMORY  XRTOS  TERMINAL  DEBUG CONSOLE  PORTS  ···   MicroPico vREPL

 1 | Network: Cumpose              | Signal:  -59 dBm
 2 | Network: Campplus             | Signal:  -60 dBm
 3 | Network: Hidden Network       | Signal:  -70 dBm
 4 | Network: Campplus             | Signal:  -71 dBm
 5 | Network: KPNF930C6            | Signal:  -72 dBm
 6 | Network: Hidden Network       | Signal:  -74 dBm
 7 | Network: BudgetThuis-8E0E28   | Signal:  -77 dBm
 8 | Network: Hidden Network       | Signal:  -78 dBm
 9 | Network: Campplus             | Signal:  -78 dBm
10 | Network: Odido-3B1C65         | Signal:  -80 dBm
11 | Network: Odido-1F40B0         | Signal:  -87 dBm
12 | Network: Girl wanna have fun  | Signal:  -89 dBm
```

Figure 70 - WiFi Scanner | Result

Controlling Stuff (Webserver)

```
# ------------------------------------------------------------
# WIFI RGB LED WEB SERVER
# ------------------------------------------------------------
# This program turns a Raspberry Pi Pico W into a simple web
# server that allows remote control of an RGB LED via a browser.
#
# After connecting to a WiFi network, the microcontroller hosts
# an HTTP server on port 80. When a client accesses the server,
# a web page with buttons is displayed.
#
# Each button corresponds to a specific URL route (e.g. /red,
# /green, /blue). When a route is requested, the server parses
# the HTTP request and updates the RGB LED color accordingly.
#
# The system uses socket programming for communication and
# basic HTTP response handling to serve the control interface.
#
# Author: Francisco Pimentel
# Date: March 2026
# ------------------------------------------------------------
import utime
import network
import socket
from machine import Pin
red = Pin(1, Pin.OUT, value=0)
green = Pin(2, Pin.OUT, value=0)
blue = Pin(3, Pin.OUT, value=0)
button = Pin(4, Pin.IN, Pin.PULL_UP)
# State
led_on = False
current_color = (0, 0, 0)
last_button_state = 1
def main() -> None:
    global led_on, current_color, last_button_state
    set_color(0, 0, 0)
    wifi = network.WLAN(network.STA_IF)
    wifi.active(True)
    wifi.connect("iPhone", "francisco")
    while not wifi.isconnected():
        print("Connecting...")
        utime.sleep(1)
    print("Connected:", wifi.ifconfig()[0])
    addr = socket.getaddrinfo('0.0.0.0', 80)[0][-1]
    server = socket.socket()
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(addr)
    server.listen(1)
    print("Server running...")
    try:
        while True:
            # Verify button state
            button_state = button.value()       # reads the button: 1=unpressed, 0=pressed
            if button_state == 0 and last_button_state == 1:  # when pressed
                led_on = not led_on
                apply_color()
```

```
                utime.sleep_ms(50)
            last_button_state = button_state
            # Allow connection without blocking
            server.settimeout(0.05)         # don't block too long waiting
            try:
                connection, address = server.accept()
            except OSError:
                continue                    # timeout, go back to checking the button
            connection.settimeout(1.0)
            try:
                request = connection.recv(1024).decode()
                request_line = request.split('\n')[0]
                print("REQUEST:", request_line)
                # Status route - responds only with JSON
                if "GET /status " in request_line:
                    state = '{"on":true}' if led_on else '{"on":false}'
                    response = "HTTP/1.1 200 OK\r\nContent-
Type: application/json\r\nConnection: close\r\n\r\n" + state
                    connection.sendall(response.encode())
                    connection.close()
                    continue
                routes = {
                    "/red":     (1, 0, 0),
                    "/green":   (0, 1, 0),
                    "/blue":    (0, 0, 1),
                    "/magenta": (1, 0, 1),
                    "/cyan":    (0, 1, 1),
                    "/yellow":  (1, 1, 0),
                }
                if "GET /off " in request_line:
                    led_on = False
                    set_color(0, 0, 0)
                elif "GET /on " in request_line:
                    led_on = True
                    apply_color()
                else:
                    for route, color in routes.items():
                        if f"GET {route} " in request_line:
                            current_color = color
                            led_on = True
                            apply_color()
                            break
                html = """<!DOCTYPE html>
                            <html>
                            <head>
                                <title>RGB LED</title>
                                <script>
                                    function updateStatus() {
                                        fetch('/status')
                                            .then(r => r.json())
                                            .then(data => {
                                                document.getElementById('indicator').textC
ontent = data.on ? 'ON' : 'OFF';
                                            });
                                    }
                                    setInterval(updateStatus, 500);
                                    window.onload = updateStatus;
                                </script>
                            </head>
                            <body>
```

```
                                <h1>RGB LED Control</h1>
                                <div id="indicator"
                                    style="
                                        width:80px;
                                        height:80px;
                                        border-radius:8px;
                                        background:gray;
                                        display:flex;
                                        align-items:center;
                                        justify-content:center;
                                        font-size:20px;
                                        font-weight:bold;
                                    ">
                                    OFF
                                </div>
                                <br>
                            <a href="/red"><button style="background:red;">Red</button></a>
                        <a href="/green"><button style="background:green;">Green</button></a>
                          <a href="/blue"><button style="background:blue;">Blue</button></a>
                <a href="/magenta"><button style="background:magenta;">Magenta</button></a>
                          <a href="/cyan"><button style="background:cyan;">Cyan</button></a>
                    <a href="/yellow"><button style="background:yellow;">Yellow</button></a>
                                </body>
                                </html>"""
                response = (
                    "HTTP/1.1 200 OK\r\n"                    # status line
                    "Content-
Type: text/html\r\n"           # says to the browser that it will receive HTML
                    "Connection: close\r\n"          # closes the connection after sending
                    "\r\n"           # mandatory empty line >> separates headers from body
                    + html                                   # the HTML content of the page
                )
                connection.sendall(response.encode())
            except Exception as e:
                print("Error:", e)
            finally:
                connection.close()
    finally:
        server.close()
def set_color(r: int, g: int, b: int) -> None:
    red.value(r)
    green.value(g)
    blue.value(b)
def apply_color() -> None:
    if led_on:
        set_color(*current_color)
    else:
        set_color(0, 0, 0)
if __name__ == "__main__":
    main()
```

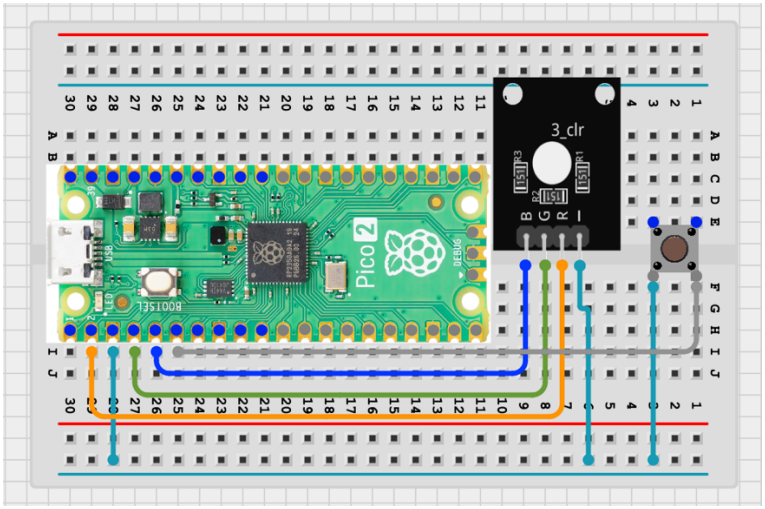


Figure 71 - Controlling Stuff (Webserver) | Circuit View

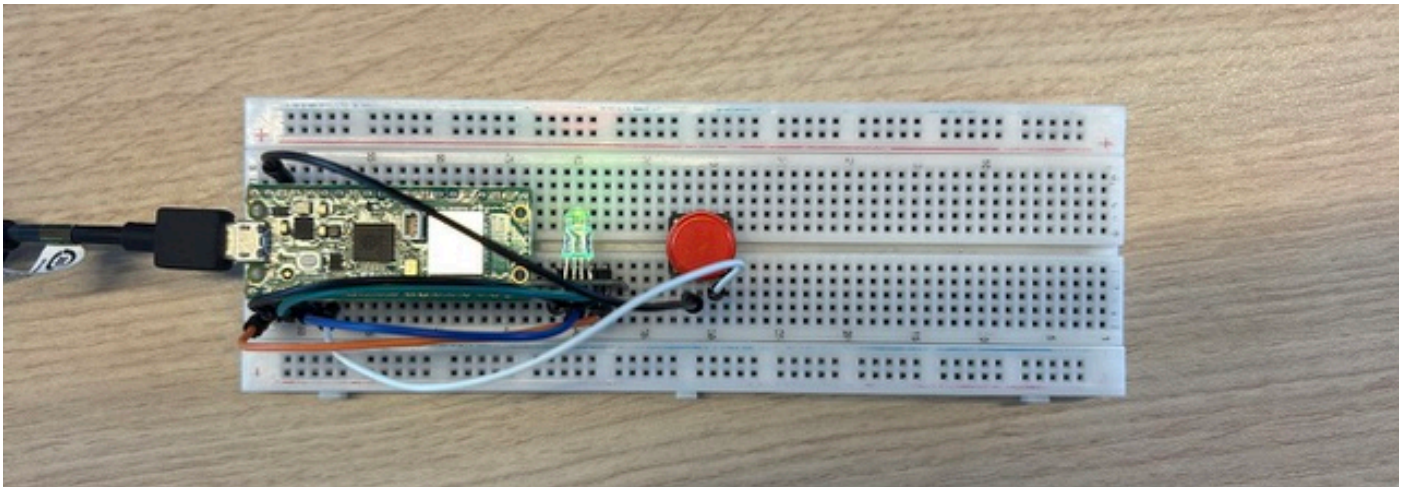


Figure 72 - Controlling Stuff (Webserver) | Real View

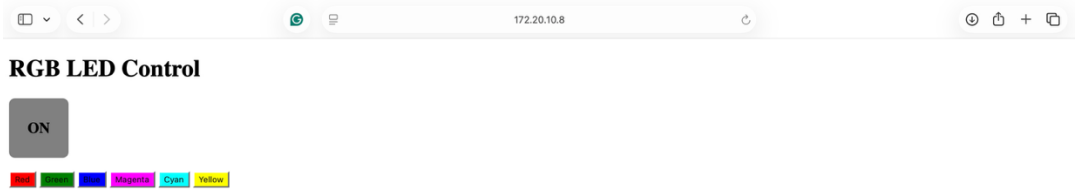


Figure 73 - Controlling Stuff (Webserver) | Web page

Weeks 6 / 7

Attendance (RFID, RTC, Web and LCD)

```
# ------------------------------------------------------------
# RFID ATTENDANCE REGISTER
# ========================
# Raspberry Pi Pico W based attendance system using RFID.
# Tracks clock-in and clock-out times, displays status on
# an LCD, and serves a live attendance log via a web server.
#
# Author: Francisco Pimentel
# Date: March 2026
# ------------------------------------------------------------
import machine
import utime
import network
import socket
import ds1302
from machine import I2C, Pin, SPI
from i2c_lcd import I2cLcd
from mfrc522 import MFRC522
# Initial RTC time: [year, month, day, weekday, hour, minute, second]
START_TIME = [2026, 3, 30, 2, 13, 22, 0]
# Authorized card UID converted to a single integer
KIKO_UID = int.from_bytes(bytes([33, 219, 41, 170]), "little", False)
# Wi-Fi credentials
WIFI_SSID = "iPhone"
WIFI_PASSWORD = "francisco"
# Minimum time (ms) between two reads of the same card
DEBOUNCE_TIME = 2000
# RGB LED pins
pin_red = machine.Pin(10, machine.Pin.OUT)
pin_green = machine.Pin(11, machine.Pin.OUT)
pin_blue = machine.Pin(12, machine.Pin.OUT)
# Buzzer on GP13 using PWM, starts silent
buzzer = machine.PWM(machine.Pin(13))
buzzer.duty_u16(0)
# LCD via I2C on GP8 (SDA) and GP9 (SCL), address 0x27, 2 rows x 16 cols
i2c = I2C(0, sda=Pin(8), scl=Pin(9), freq=100000)
lcd = I2cLcd(i2c, 0x27, 2, 16)
# RTC DS1302 on GP16 (CLK), GP17 (DAT), GP18 (CS)
rtc = ds1302.DS1302(Pin(16), Pin(17), Pin(18))
# RFID reader via SPI0: SCK=GP2, MOSI=GP3, MISO=GP4, RST=GP6, CS=GP5
reader = MFRC522(sck=2, mosi=3, miso=4, rst=6, cs=5)
kiko_is_inside = False      # tracks whether Kiko is currently clocked in
records = []                # list of all attendance records
current_record = None       # reference to the active (open) record
clock_in_seconds = 0       # clock-in time in seconds, used to calculate duration
last_uid = None            # UID of the last scanned card
last_uid_time = 0          # timestamp of the last scan in milliseconds
web_server = None          # socket server instance, None if Wi-Fi unavailable
def main() -> None:
    global last_uid, last_uid_time
```

```
    # Show startup message on LCD
    lcd.clear()
    lcd.backlight_on()
    lcd_write_line(0, "  Starting...   ")
    lcd_write_line(1, "  Please wait.. ")
    utime.sleep(1)
    # Set the RTC to the configured start time
    rtc.date_time(START_TIME)
    print("RTC time set!")
    print("RFID ready! Scan a card to see its UID.")
    # Connect to Wi-Fi and start the web server if successful
    ip = wifi_connect()
    if ip is not None:
        server_start()
        print("Visit: http://" + ip)
    else:
        print("No Wi-Fi - web server disabled.")
    # Show ready message on LCD
    lcd_write_line(0, "  System ready! ")
    lcd_write_line(1, "Scan your card..")
    utime.sleep(1)
    while True:
        # Update clock on line 0 every loop
        lcd_write_line(0, "Time: " + rtc_get_time_string())
        # Handle incoming web requests without blocking
        server_check(records)
        # Try to read an RFID card
        uid_int = rfid_read_uid()
        if uid_int is not None:
            now_ms = utime.ticks_ms()
            # Ignore the same card if scanned too recently (debounce)
            if uid_int == last_uid:
                diff = utime.ticks_diff(now_ms, last_uid_time)
                if diff < DEBOUNCE_TIME:
                    utime.sleep_ms(50)
                    continue
            # Update last scan info
            last_uid      = uid_int
            last_uid_time = now_ms
            current_time  = rtc_get_time_string()
            # Handle known or unknown card
            if uid_int == KIKO_UID:
                if kiko_is_inside == False:
                    do_clock_in(current_time)
                else:
                    do_clock_out(current_time)
            else:
                unknown_card(uid_int)
        else:
            # No card detected, update line 1 based on current state
            if kiko_is_inside == False:
                lcd_write_line(1, "Scan your card..")
            else:
                lcd_write_line(1, "Inside :)")
        utime.sleep_ms(200)
# --------
#  RFID
# --------
def rfid_read_uid() -> int | None:
    # Initialize the reader and check for a card in range
```

```
    reader.init()
    status, tag_type = reader.request(reader.REQIDL)
    if status != reader.OK:
        return None                                          # no card detected
    # Read the card's serial number (UID)
    status, uid = reader.SelectTagSN()
    if status != reader.OK:
        return None                                          # failed to read UID
    # Convert the 4-byte UID list to a single integer
    return int.from_bytes(bytes(uid), "little", False)
# -------
#  RTC
# -------
def rtc_get_time_string() -> str:
    # Read current time from RTC and return as HH:MM:SS string
    year, month, day, weekday, hour, minute, second = rtc.date_time()
    return "{:02d}:{:02d}:{:02d}".format(hour, minute, second)
# ---------------
#  WEB SERVER
# ---------------
def wifi_connect() -> str | None:
    # Activate the Wi-Fi interface in station mode
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    # If already connected, return the IP immediately
    if wlan.isconnected():
        ip = wlan.ifconfig()[0]
        print("Wi-Fi connected! Open: http://" + ip)
        return ip
    # Attempt to connect, retrying up to 15 times
    print("Connecting to Wi-Fi:", WIFI_SSID)
    wlan.connect(WIFI_SSID, WIFI_PASSWORD)
    for i in range(15):
        if wlan.isconnected():
            break
        print("  trying... ({}/15)".format(i + 1))
        utime.sleep(1)
    # Return IP if connected, otherwise return None
    if wlan.isconnected():
        ip = wlan.ifconfig()[0]
        print("Wi-Fi connected! Open: http://" + ip)
        return ip
    print("Could not connect to Wi-Fi.")
    return None
def server_start() -> None:
    global web_server
    # Bind a TCP socket to port 80 on all interfaces
    addr = socket.getaddrinfo("0.0.0.0", 80)[0][-1]
    web_server = socket.socket()
    web_server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    web_server.bind(addr)
    web_server.listen(1)
    # Short timeout so accept() doesn't block the main loop
    web_server.settimeout(0.05)
    print("Web server started.")
def server_check(records: list[dict]) -> None:
    global web_server
    # Do nothing if the server was never started
    if web_server is None:
        return
```

```
    try:
        # Wait briefly for an incoming connection
        connection, address = web_server.accept()
    except OSError:
        return                                          # timeout >> no client, conti
nue normally
    # Give the client 1 second to send the request
    connection.settimeout(1.0)
    try:
        # Read and log the HTTP request line
        request = connection.recv(1024).decode()
        print("REQUEST:", request.split('\n')[0])
        # Build and send the HTML response
        content = build_html(records)
        response = (
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Connection: close\r\n"
            "\r\n"
            + content
        )
        connection.sendall(response.encode())
    except Exception as e:
        print("Server error:", e)
    finally:
        connection.close()                              # always close the connection
def build_html(records: list[dict]) -> str:
    if len(records) == 0:
        # Centered message when no records exist yet
        rows = '<tr><td colspan="3" style="text-
align:center;">No records yet - scan a card!</td></tr>'
    else:
        rows = ""
        for record in records:
            if record.get("unknown"):
                color = "#fcc"       # light red >> unknown card
                out = record["clock_out"]
            elif record["clock_out"] == "":
                color = "#c8f7c5"    # light green >> currently inside
                out = "&#8212;"
            else:
                color = "#fde8c8"    # light orange >> clocked out
                out = record["clock_out"]
            rows += (
                '<tr style="background:' + color + ';">'
                "<td>" + record["name"]     + "</td>"
                "<td>" + record["clock_in"] + "</td>"
                "<td>" + out           + "</td>"
                "</tr>"
            )
    return """<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="refresh" content="5">
  <title>Attendance Register</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 30px; background: white; }
    h1 { margin-bottom: 20px; }
    table { border-collapse: collapse; width: 100%; }
```

```
    th, td { border: 1px solid #ccc; padding: 10px 16px; text-align: left; }
    th { background: #f0f0f0; }
  </style>
</head>
<body>
  <h1>Attendance Register</h1>
  <table>
    <thead><tr><th>Name</th><th>Clock In</th><th>Clock Out</th></tr></thead>
    <tbody>""" + rows + """</tbody>
  </table>
</body>
</html>"""
# -------------------------
#  CLOCK-IN / CLOCK-OUT
# -------------------------
def do_clock_in(time: str) -> None:
    global kiko_is_inside, current_record, clock_in_seconds
    kiko_is_inside = True
    # Store clock-in time in seconds to calculate duration later
    h = int(time[0:2])
    m = int(time[3:5])
    s = int(time[6:8])
    clock_in_seconds = h * 3600 + m * 60 + s
    # Create a new open record and add it to the list
    current_record = {"name": "Kiko", "clock_in": time, "clock_out": ""}
    records.append(current_record)
    # Show welcome message on LCD
    lcd_write_line(0, "Welcome Kiko!")
    lcd_write_line(1, "In: " + time)
    # Green LED and two-tone clock-in sound
    led_green()
    sound_clock_in()
    utime.sleep_ms(2000)
    led_off()
    print("CLOCK-IN ->", time)
def do_clock_out(time: str) -> None:
    global kiko_is_inside, current_record, clock_in_seconds
    kiko_is_inside = False
    # Calculate total time inside
    h = int(time[0:2])
    m = int(time[3:5])
    s = int(time[6:8])

    clock_out_seconds = h * 3600 + m * 60 + s
    diff = clock_out_seconds - clock_in_seconds
    # Handle midnight rollover
    if diff < 0:
        diff += 86400
    # Format duration as seconds, minutes, or hours
    if diff < 60:
        duration = "{}s".format(diff)
    elif diff < 3600:
        duration = "{}m".format(diff // 60)
    else:
        duration = "{}h".format(diff // 3600)
    # Close the current record with the clock-out time
    if current_record is not None:
        current_record["clock_out"] = time
    # Show goodbye message and duration on LCD
    lcd_write_line(0, "Goodbye Kiko!")
```

```
    lcd_write_line(1, "O:" + time + " T:" + duration)
    # Yellow LED and two-tone clock-out sound
    led_yellow()
    sound_clock_out()
    utime.sleep_ms(2000)
    led_off()
    print("CLOCK-OUT ->", time, "| Duration:", duration)
def unknown_card(uid: str) -> None:
    # Log the unknown card attempt with a red-styled name
    records.append({
        "name": '<span style="color:#a32d2d;font-weight:bold;">Unknown</span>',
        "clock_in":  rtc_get_time_string(),
        "clock_out": "&#8212;",
        "unknown": True
    })
    # Show error on LCD
    lcd_write_line(1, "Unknown card!")
    # Red LED and triple beep error sound
    led_red()
    sound_error()
    utime.sleep_ms(2000)
    led_off()
    print("Unknown card:", uid)
# --------
#  LCD
# --------
def lcd_write_line(line: int, text: str) -> None:
    # Pad the text with 16 spaces to ensure any previous content is overwritten,
    # then truncate to exactly 16 characters to fit the LCD width
    text = (text + " " * 16)[:16]
    lcd.move_to(0, line)
    lcd.putstr(text)
# --------
#  LED
# --------
def led_off() -> None:
    pin_red.value(0)                # red off
    pin_green.value(0)              # green off
    pin_blue.value(0)               # blue off
def led_green() -> None:
    pin_red.value(0)                # red off
    pin_green.value(1)              # green on
    pin_blue.value(0)               # blue off
def led_yellow() -> None:
    pin_red.value(1)                # red on
    pin_green.value(1)              # green on
    pin_blue.value(0)               # blue off
def led_red() -> None:
    pin_red.value(1)                # red on
    pin_green.value(0)              # green off
    pin_blue.value(0)               # blue off
# --------
#  BUZZER
# --------
def beep(frequency: int, duration_ms: int) -> None:
    buzzer.freq(frequency)
    buzzer.duty_u16(32768)          # 50% duty cycle >> full volume
    utime.sleep_ms(duration_ms)     # hold the tone
    buzzer.duty_u16(0)              # silence
def sound_clock_in() -> None:
```

```
    beep(880, 100)
    utime.sleep_ms(60)
    beep(1320, 150)
def sound_clock_out() -> None:
    beep(1320, 100)
    utime.sleep_ms(60)
    beep(660, 200)
def sound_error() -> None:
    beep(400, 80)
    utime.sleep_ms(80)
    beep(400, 80)
    utime.sleep_ms(80)
    beep(400, 80)
main()
```

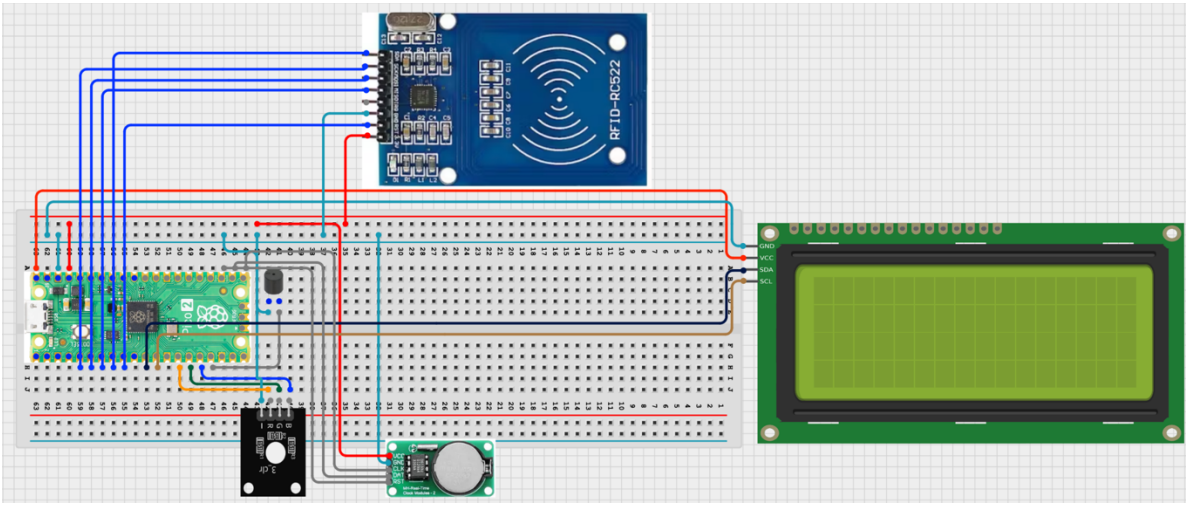


Figure 74 - Attendance (RFID, RTC, Web and LCD) | Circuit View

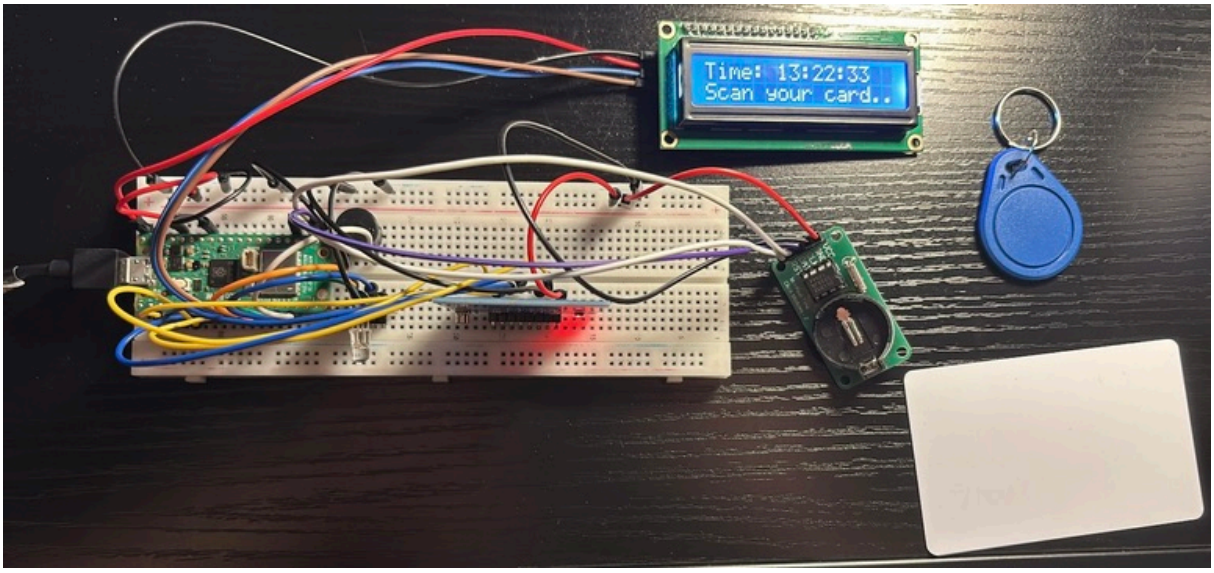


Figure 75 - Attendance (RFID, RTC, Web and LCD) | Real View

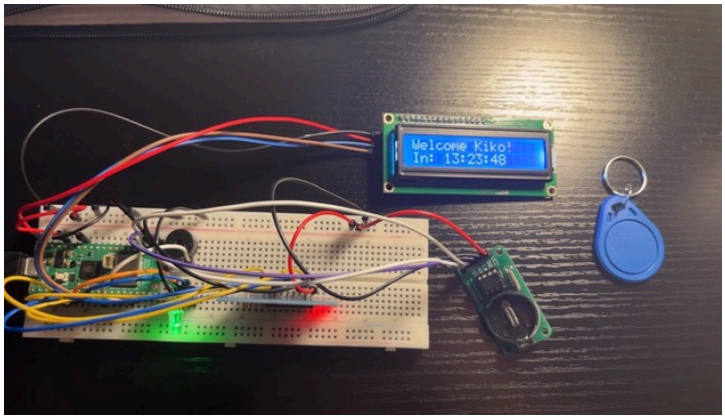


Figure 76 - Attendance (RFID, RTC, Web and LCD) | Clock-in

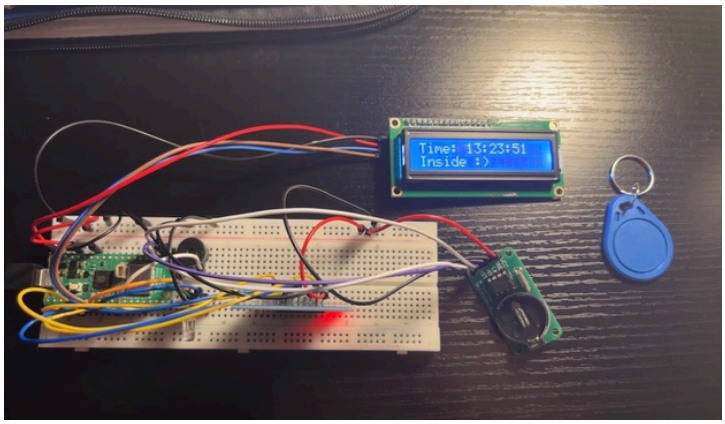


Figure 77 - Attendance (RFID, RTC, Web and LCD) | While Clock-in

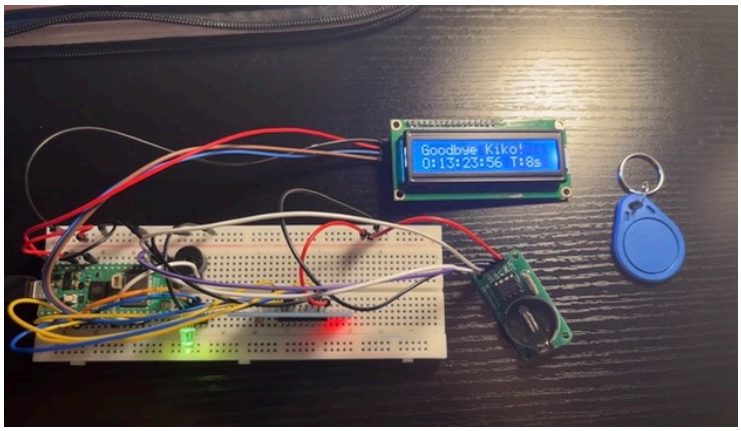


Figure 78 - Attendance (RFID, RTC, Web and LCD) | Clock-out

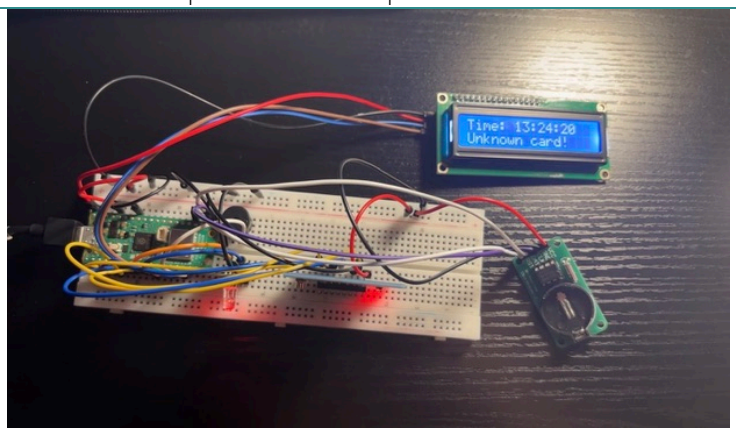


Figure 79 - Attendance (RFID, RTC, Web and LCD) | Error

Attendance Register

Name	Clock In	Clock Out
Kiko	13:22:12	—

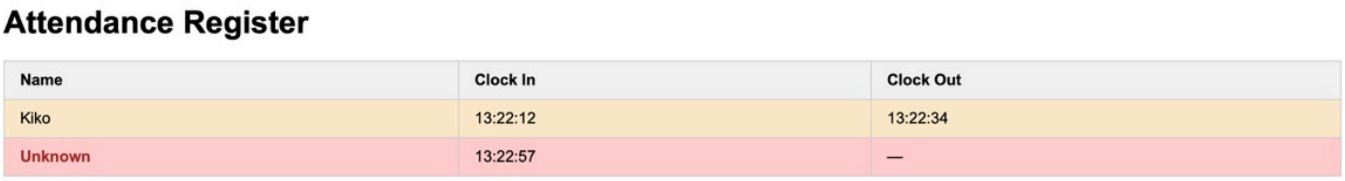
Figure 80 - Attendance (RFID, RTC, Web and LCD) | Clock-in



Attendance Register

Name	Clock In	Clock Out
Kiko	13:22:12	13:22:34

Figure 81 - Attendance (RFID, RTC, Web and LCD) | Clock-out



Attendance Register

Name	Clock In	Clock Out
Kiko	13:22:12	13:22:34
Unknown	13:22:57	—

Figure 82 - Attendance (RFID, RTC, Web and LCD) | Unknown

Self-Reflection

Looking back on this semester, I can say with confidence that it was a period of genuine growth, not because everything came easily, but because I approached every assignment with curiosity and a real desire to understand what I was building.

The most challenging part of the semester was undoubtedly the work involving artificial intelligence. Projects like Tic-Tac-Toe and Knights required a different kind of thinking, moving away from straightforward input/output logic and towards reasoning about game states, logical implications, and recursive algorithms. The Tic-Tac-Toe assignment in particular demanded the most time and effort. Implementing the Minimax algorithm correctly, ensuring it evaluated all possible game states and always returned the optimal move, was a process that pushed me to think more carefully about recursion and abstract problem-solving than any previous assignment had.

On the Embedded Systems side, this was already my third period working with hardware, so the components themselves were familiar. What made this semester different was the complexity of the projects. The one I am most proud of is the Attendance System, combining RFID, a real-time clock, an LCD display, and a web server into a single cohesive project felt like a genuine engineering challenge, and seeing it work as intended was very rewarding.

What I noticed throughout both areas is that nothing truly surprised me, not because the work was easy, but because I stayed consistent. I worked steadily through each week with focus and genuine interest, which meant challenges were met as they came rather than becoming obstacles. That consistency is something I recognise as a strength in my way of working: I maintain a good method of organising both tasks and time, and that kept me on track throughout the semester.

If I had to identify something to improve, it would be pushing myself further into the areas that challenge me most, particularly AI, rather than consolidating what I already know well. The foundation is solid; the next step is to be more ambitious with it.

Links

Assignements

link: https://github.com/franciscpime/NHL_CS_3rd_Period?tab=readme-ov-file

Solo Project

link: https://github.com/franciscpime/math_animator

Youtube Video

Before the link, I'm sorry for the bad quality and possible bad explanation. I think it was good and understandable, but at the same time I felt that some things were missing, but I can explain during interview without problem.

link: https://youtu.be/2j1eEj0RExk

P3. Computer Science

Name	Student nr.	Group	Year
Francisco Ponte Pimentel	5708915		2026

The student is responsible for safe keeping of this card

Assignments

Wk	Subject	Date	Signature
3.1	Smart Café Helper - works with venv. Good. Prof for manic	16 FEB 2026	
	Vanity Plates goes Flask with DB & history?	16 FEB 2026	
3.2	Manic Monday Functions and Variables	16 FEB 2026	
	Survival Simulator Looks really nice	2 MAR 2026	
3.3	Manic Monday	2 MAR 2026	
	Testing my twttr	2 MAR 2026	
	~~plates~~	2 MAR 2026	
	Fuel	2 MAR 2026	
	Bank	2 MAR 2026	
3.4	Presentation	4 MAR 2026	
3.5	Scavenger hunt 9⁶	12 MAR 2026	
	Shirt	16 MAR 2026	
3.6	Manic Monday	16 MAR 2026	
	~~Scavenger hunt Pt II~~	—	—
	Product goes OOP	16 MAR 2026	
	Tic-Tac-Toe		
3.7	The Misleading Minesweeper		
3.8	Manic Monday RESIT	—	—
	Presentation Solo Innovator Project		
	Check-Check-Triple Check		
T3	**Assessment**		

This card is proof that the above assignments are properly completed and should be:

- signed off by the appropriate teacher(s) during the guided ateliers, the same week
- a digital attachment of your portfolio and
- physically handed over at the beginning of the assessment

NHL STENDEN university of applied sciences

P3. Embedded Systems

Name	Student nr.	Group	Year
Francisco Ponte Pimentel	5708915		2026

The student is responsible for safe keeping of this card

Assignments

Wk	Ch	Subject	Rating	Date	Signature
3.1	1/3	Blink with external hardware reset	(V) O	5 FEB 2026	M. Baron
	2/3	Mario Pico	(V) O	5 FEB 2026	M. Baron
3.2	4/5	7-Segments Voltmeter *(0…10V)*	G (V) O	9 MAR 2026	M. Baron
3.3	5	RTC and temperature on LCD-display	G (V) O	6 MAR 2026	M. Baron
3.4	-	Dino cheater *(HID)*	G (V) O	18 MAR 2026	M. Baron
		Analog joystick *(HID)*	G (V) O	19 MAR 2026	M. Baron
3.5	11	Wifi scanner	G (V) O	16 MAR 2026	M. Baron
		Controlling stuff *(Webserver)*	G (V) O	25 MAR 2026	M. Baron
3.6	12	Attendance *(RFID, RTC, web and LCD)*	G (V) O	31 MAR 2026	M. Baron
3.7					
3.8	-	Solo Pico Project *(not mandatory)*	U G V O	—	—
3.8	-	Completing Portfolio			
T3	-	**Assessment**			

U = Ultimate, G = Good, V = Valid, O = Off standard

Feedback:

Uitdaging

31 MAR 2026

M. Baron

This card is proof that the above assignments are completed and should be:

- signed off during guided ateliers by the assigned teacher(s), in the same week,
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment